
Nemsys, una plataforma web colaborativa para compartir y acceder a recursos académicos

Nemsys, a collaborative web platform for sharing and accessing academic resources



Trabajo de Fin de Grado

Curso 2025–2026

Autor

Marco Antonio Pérez Neira

Director

Ramón González del Campo Rodríguez Barbero

Ingeniería de Software
Facultad de Informática
Universidad Complutense de Madrid

Resumen

Nemtsy, una plataforma web colaborativa para compartir y acceder a recursos académicos

Hasta hace relativamente poco, en España no existían plataformas bien integradas donde compartir apuntes, exámenes u otros recursos académicos; los estudiantes dependían de intercambios en persona o de grupos de mensajería sin persistencia real. En 2015 se lanzó Wuolah, una plataforma que sí se integraba con los centros educativos y permitía subir apuntes vinculados específicamente a una asignatura de un grado concreto. Sin embargo, esta plataforma presenta limitaciones significativas de rendimiento y un modelo de monetización basado en publicidad intensiva que deteriora la experiencia del usuario.

Este Trabajo de Fin de Grado describe el diseño e implementación de una aplicación web alternativa que permite a estudiantes compartir, buscar y descargar apuntes sin interferencias, con un énfasis en la rapidez y la ligereza desde el inicio. La plataforma está diseñada para integrarse con las asignaturas, estudios y centros educativos tanto de España como del extranjero. Debido a las limitaciones de tiempo y al carácter tedioso del proceso más que dificultad técnica, la integración completa solo se ha realizado sobre la UCM, aunque la arquitectura está preparada para su expansión a otros centros: únicamente sería necesario desarrollar un *web crawler* adaptado a la página de cada uno.

Para obtener toda la información necesaria se ha utilizado una combinación de *web scraping* y el desarrollo de un *script* que utiliza JetBrains SWOT, una lista de dominios de correo académicos, para conseguir todos los centros educativos y sus dominios. Los usuarios que completan el proceso de *onboarding* podrán ver todas las asignaturas de su grado separadas por año y ver los apuntes que otros usuarios han compartido. Además, podrán buscar globalmente entre todos los apuntes de la plataforma gracias a la utilización de *Full Text Search* que permite una búsqueda instantánea aun con millones de apuntes.

La plataforma está construida siguiendo una arquitectura cliente-servidor: un backend desarrollado en Go, elegido por su rendimiento y la solidez de su ecosistema para la construcción de APIs, un frontend en SvelteKit que proporciona una interfaz

limpia y responsiva y PostgreSQL como base de datos lo que facilita ciertas funcionalidades.

Palabras clave

Aplicación Web, Plataforma Colaborativa, Recursos Académicos, Go, SvelteKit, PostgreSQL, OAuth2, Docker.

Abstract

Nemsys, a collaborative web platform for sharing and accessing academic resources

Until relatively recently, there were no well-integrated platforms in Spain where students could share notes, exams or other academic resources; they relied on in-person exchanges or messaging groups with no real persistence. In 2015 Wuolah was launched, a platform that did integrate with educational institutions and allowed users to upload notes linked specifically to a subject within a degree programme at their university. However, this platform suffers from significant performance limitations and an advertising-heavy monetisation model that degrades the user experience.

This Bachelor's Thesis describes the design and implementation of an alternative web application that allows students to share, search and download notes without interference, with an emphasis on speed and lightness from the outset. The platform is designed to integrate with subjects, degree programmes and educational institutions both in Spain and abroad. Due to time constraints and the tedious nature of the process rather than technical difficulty, full integration has only been completed for the UCM, although the architecture is ready for expansion to other institutions: only a *web crawler* adapted to each institution's website would be needed.

To achieve this, a combination of *web scraping* and the development of a script that uses JetBrains SWOT, a list of academic email domains, was used to obtain all educational institutions and their domains. Users who complete the onboarding process can see all the subjects in their degree organised by year and access notes shared by other students. They can also search globally across all notes on the platform thanks to *Full Text Search*, allowing searches to remain instant even with millions of resources.

The platform is built following a client-server architecture: a backend developed in Go, chosen for its performance and the robustness of its ecosystem for building APIs, a frontend in SvelteKit that provides a clean and responsive interface, and PostgreSQL as the database which facilitates certain functionalities.

Keywords

Web Application, Collaborative Platform, Academic Resources, Go, SvelteKit, PostgreSQL, OAuth2, Docker.

Agradecimientos

A mi director, Ramón González del Campo Rodríguez Barbero, por su orientación y por haber confiado en este proyecto desde el inicio.

A mi familia, por el apoyo incondicional durante estos años de carrera.

A mis compañeros del Grado en Ingeniería de Software de la UCM, con quienes he compartido apuntes, exámenes y tantos quebraderos de cabeza, y sin cuya colaboración habría sido prácticamente imposible llegar hasta aquí. Este trabajo es, en cierto modo, un intento de devolver algo de lo mucho que me han aportado.

Índice general

Resumen	3
Nemsys, una plataforma web colaborativa para compartir y acceder a recursos académicos	3
Palabras clave	4
Abstract	6
Nemsys, a collaborative web platform for sharing and accessing academic resources	6
Keywords	7
1 Introducción	1
1.1 Motivación	1
1.2 Objetivos	3
1.3 Plan de trabajo	4
1.4 Estructura del documento	5
1 Introduction	6
1.1 Motivation	6
1.2 Objectives	8
1.3 Work Plan	9
1.4 Document Structure	9
2 Estado de la Cuestión	10
2.1 Plataformas existentes	10
2.1.1 Wuolah	10
2.1.2 Studocu	11
2.1.3 Docsity	12
2.1.4 Conclusiones	13
3 Descripción del Trabajo	14
3.1 Tecnologías evaluadas	14
3.1.1 Backend	14
3.1.2 Frontend	17
3.1.3 Base de datos	18
3.1.4 Almacenamiento	20
3.1.5 Despliegue	20
3.2 Arquitectura del sistema	21
3.2.1 Frontend	22
3.2.2 Backend	23

3.2.3	Autenticación	24
3.2.4	Despliegue	24
3.3	Diseño de la interfaz de usuario	27
3.3.1	Evolución del diseño	27
3.3.2	Sistema de diseño	28
3.3.3	Tipografía	29
3.3.4	Paleta de color	29
3.4	Diseño de la base de datos	31
3.4.1	Entidades principales	32
3.4.2	Evolución mediante migraciones	33
3.4.3	Búsqueda de texto completo	34
3.4.4	Índices adicionales	35
3.5	Implementación del backend	35
3.5.1	Estructura del proyecto	35
3.5.2	Arranque del servidor	37
3.5.3	Generación de código con <code>sqlc</code>	39
3.5.4	Creación de un recurso: transacciones y compensación	41
3.5.5	Descarga de archivos con redirección y empaquetado en <i>streaming</i>	44
3.5.6	Detalles internos de la autenticación	46
3.5.7	Sistema de reportes y moderación	50
3.6	Implementación del frontend	52
3.6.1	Estructura del proyecto	52
3.6.2	Ciclo de vida de una página	54
3.6.3	Reactividad con runes de Svelte 5	55
3.6.4	Comunicación con el backend	57
3.6.5	Subida de recursos: formulario multipaso	59
3.6.6	Búsqueda con <i>debounce</i> y desplazamiento infinito	60
3.6.7	Responsividad	61
3.6.8	Sistema de componentes	63
3.6.9	Panel de administración	63
3.7	Pruebas y validación	64
3.7.1	Tests unitarios del backend	65
3.7.2	Tests unitarios del frontend	66
3.7.3	Tests end to end con Playwright	67
3.7.4	Validación de rendimiento	69
4	Conclusiones y Trabajo Futuro	72
4.1	Conclusiones	72
4.2	Objetivos alcanzados	73
4.3	Trabajo futuro	73
4	Conclusions and Future Work	75
4.1	Conclusions	75
4.2	Objectives Achieved	76
4.3	Future Work	76

Bibliografía	78
Apéndice A Guía de despliegue	80
A.1 Requisitos previos	80
A.2 Configuración	80
A.3 Levantamiento del <i>stack</i>	81
A.4 Carga inicial del catálogo	81
A.5 Promoción de un usuario a administrador	82
Apéndice B Glosario y acrónimos	83

Índice de figuras

Figura 1	Análisis del rendimiento de wuolah.com con Google Lighthouse.	2
Figura 2	Captura de pantalla de wuolah.com mostrando la interfaz actual.	3
Figure 3	Performance analysis of wuolah.com with Google Lighthouse.	7
Figure 4	Screenshot of wuolah.com showing the current interface.	8
Figura 5	Pantalla de aceptación de la política de privacidad de Wuolah.	11
Figura 6	Barra de navegación de Wuolah.	11
Figura 7	Captura de pantalla de Studocu.	12
Figura 8	Captura de pantalla de Docsity para la UCM.	13
Figura 9	Frameworks y tecnologías web más populares según la encuesta de Stack Overflow 2025 [1].	15
Figura 10	Diagrama de componentes de la arquitectura de Nemsy.	22
Figura 11	Arquitectura de despliegue en el VPS con Docker Compose.	25
Figura 12	Primera versión de la interfaz de Nemsy (enero 2026).	27
Figura 13	Interfaz actual de Nemsy en modo desplegado y modo compacto.	28
Figura 14	Logotipo de Nemsy.	29
Figura 15	Paleta de color de Nemsy basada en colores por defecto de Tailwind CSS.	29
Figura 16	Página de búsqueda global de Nemsy.	30
Figura 17	Página de inicio para usuarios no autenticados.	30
Figura 18	Selección manual de universidad durante el <i>onboarding</i>	31
Figura 19	Vistas móviles: listado de recursos, búsqueda global y perfil de usuario.	31
Figura 20	Diagrama entidad-relación del esquema de Nemsy.	32
Figura 21	Salida del comando <code>tree</code> sobre la raíz del backend.	36
Figura 22	Fases del arranque del servidor en <code>cmd/server/main.go</code>	37
Figura 23	Creación de un recurso con varios archivos, mostrando el camino de éxito y la rama de compensación ante un fallo en S3.	42
Figura 24	Diagrama de secuencia del flujo de autenticación con Google OAuth2.	47
Figura 25	Ciclo de vida de un reporte, desde la denuncia del usuario hasta la resolución por parte de un administrador.	51
Figura 26	Salida de <code>tree src -L 4 -I node_modules --dirsfirst</code> sobre la raíz del frontend.	53

Figura 27	Fases de la primera carga de una página y transición a navegación cliente.	54
Figura 28	Camino de una petición SSR autenticada hasta la API.	57
Figura 29	Formulario de creación de un recurso en escritorio y en móvil.	60
Figura 30	Misma vista de recurso en escritorio y en móvil.	62
Figura 31	Panel de administración con la lista de reportes pendientes.	64
Figura 32	Ejecución de la prueba de carga con k6 en el VPS de producción.	69
Figura 33	Comparativa de puntuaciones Lighthouse entre Wuolah y Nemsy.	70

Índice de tablas

Tabla 1	Comparativa de tecnologías para el backend.	16
Tabla 2	Comparativa de tecnologías para el frontend.	18
Tabla 3	Comparativa de sistemas de gestión de bases de datos.	19
Tabla 4	Rutas de la aplicación frontend.	22
Tabla 5	Paquetes del backend y su responsabilidad.	24
Tabla 6	Tablas del esquema y su responsabilidad.	32
Tabla 7	Historial de migraciones del esquema.	33
Tabla 8	Variables de entorno consumidas por el servidor.	38
Tabla 9	<i>Claims</i> incluidos en el JWT de sesión de Nemsy.	49
Tabla 10	Directorios principales del frontend y su responsabilidad.	53
Tabla 11	Runes de Svelte 5 utilizados en el frontend.	55
Tabla 12	Endpoints de la API consumidos desde el frontend.	57
Tabla 13	Visores de previsualización empleados en <code>/create</code> y en la consulta de recursos.	59
Tabla 14	<i>Breakpoints</i> de Tailwind utilizados en el frontend.	61
Tabla 15	Bibliotecas de UI utilizadas en el frontend.	63
Tabla 16	Latencias por endpoint en la prueba de carga con k6 (100 VUs, 0 % de errores).	69

Capítulo 1 Introducción

Resumen: En este capítulo se presenta la motivación del proyecto, los objetivos que se persiguen y la estructura del documento.

Talk is cheap. Show me the code.

— Linus Torvalds

1.1 Motivación

Como estudiante de ingeniería de software utilizo una gran cantidad de software distinto en mi día a día; algunos funcionan mejor y otros resultan frustrantes. Es precisamente uno de los segundos lo que me motivó a realizar este Trabajo de Fin de Grado.

Antes de Wuolah, las alternativas se limitaban a plataformas genéricas como Docsity o StudyLib, que no se habían orientado al mercado español y cuyo contenido era muy escaso o prácticamente inexistente según la plataforma, y que tampoco se integraban con centros educativos concretos, o a soluciones informales como grupos de mensajería y *drives* compartidos.

Wuolah, la plataforma para compartir apuntes mejor integrada y más conocida, solucionó un problema real al ver la necesidad de los estudiantes de tener un lugar donde compartir y encontrar apuntes relevantes para sus asignaturas concretas, de su grado o estudio, en su centro educativo. A pesar de su popularidad el modelo actual que sigue presenta una serie de problemas que perjudican tanto la experiencia de usuario como la calidad del intercambio de información.

Su principal problema es el rendimiento. Al realizar un análisis de la plataforma utilizando Lighthouse, una herramienta de código abierto de Google que audita el rendimiento, la accesibilidad, mejores prácticas y el SEO de cualquier página web (Figura 1), se observan unos resultados bastante concluyentes en cuanto a su rendimiento.

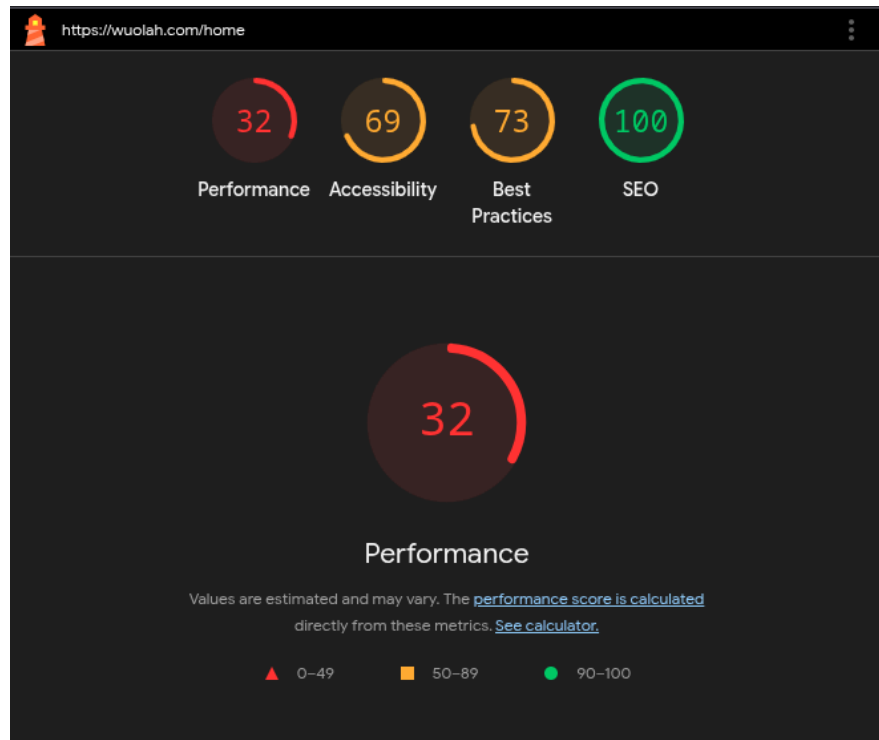


Figura 1: Análisis del rendimiento de wuolah.com con Google Lighthouse.

La plataforma obtiene una puntuación de rendimiento de 32 sobre 100, un resultado que muestra un gran problema de optimización y explica la lentitud que se siente al usarla. Esta baja puntuación se puede explicar por el uso de una arquitectura pesada en el lado del cliente y la gran cantidad de anuncios que se cargan en este. Esto empeora significativamente la experiencia del usuario y demuestra la necesidad de un enfoque de diseño más eficiente.

Además, su modelo de negocio basado en publicidad intensiva también perjudica la experiencia. Si bien es comprensible que la plataforma necesite monetizarse, la implementación actual penaliza en exceso al usuario gratuito con largos tiempos de espera para descargar cada recurso, a menos que se pague una suscripción mensual.

Por otro lado, los micropagos por descarga que ofrece Wuolah presentan otro problema fundamental: cuando se introduce una recompensa económica, especialmente cuando es mínima, se corre el riesgo de transformar la motivación de los usuarios; el deseo genuino de ayudar es reemplazado por un interés transaccional de bajo valor. Este enfoque ha sido demostrado como algo que puede llegar a ser contraproducente. La investigación sobre la motivación ha demostrado que las recompensas externas pueden disminuir el interés propio por realizar una tarea, un efecto conocido como «Desplazamiento Motivacional» [2], [3].

Además, este modelo de incentivos tiene otra consecuencia negativa para las carreras técnicas. Al limitar las recompensas a los archivos PDF, ya que en estos es en los que se puede incrustar anuncios, se desincentiva a que los estudiantes compartan información en otros formatos más adecuados para ciertas áreas como puede ser en el

caso de código, empobreciendo así la variedad y utilidad de los recursos disponibles en la plataforma.

Por último, cabe mencionar también cómo la interfaz actual refleja un problema común en plataformas que reciben un porcentaje significativo de uso desde dispositivos móviles: para facilitar la responsividad, la experiencia en escritorio se ha visto degradada. Como se puede observar en la Figura 2, con una resolución estándar de portátil de 1920x1080 al 100% de zoom, apenas la mitad de la pantalla se dedica al contenido, dejando las barras laterales sobrantes para anuncios. De esa mitad usada, algo más de un tercio está ocupado por una barra lateral con información de poco valor, dejando apenas una fracción del ancho total de la pantalla para lo que el usuario realmente quiere ver.

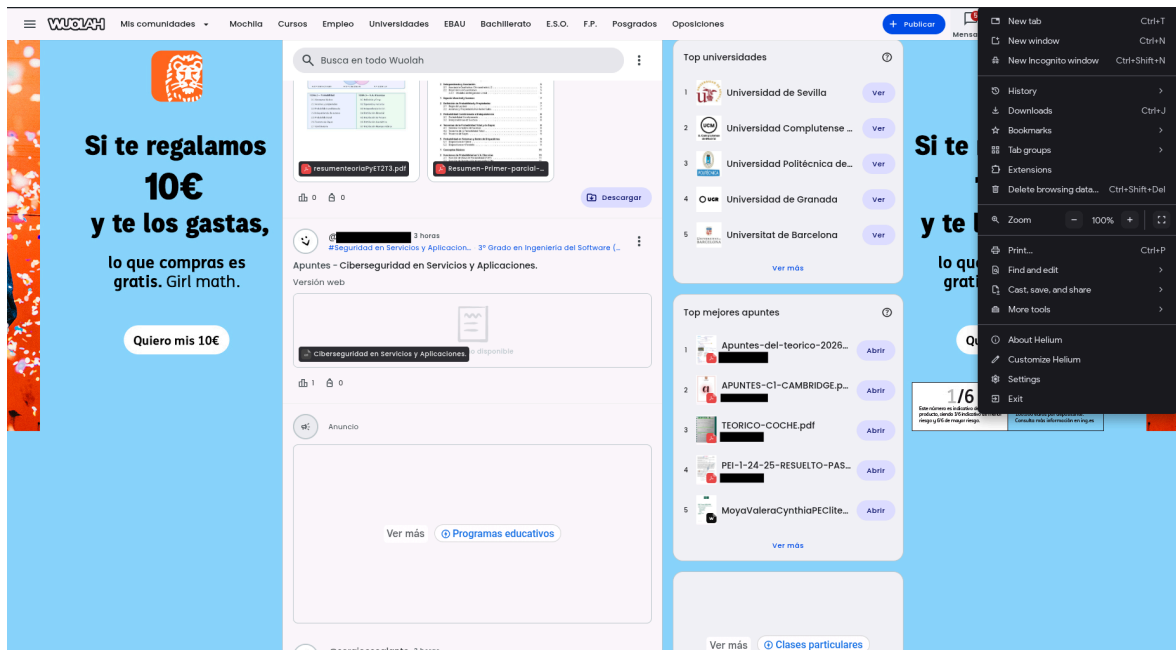


Figura 2: Captura de pantalla de wuolah.com mostrando la interfaz actual.

Este trabajo busca solucionar todos estos problemas y tiene como objetivo el diseño y desarrollo de un prototipo de plataforma alternativa. La solución propuesta mejorará el rendimiento usando una arquitectura ligera y eficiente con un *backend* en Go y un *frontend* con SvelteKit. Para reducir los costes de mantenimiento se usarán servidores privados virtuales (VPS) en vez de plataformas en la nube como AWS. Y se eliminará la publicidad invasiva y los incentivos monetarios para fomentar la colaboración genuina y la compartición de recursos en múltiples formatos.

1.2 Objetivos

El objetivo principal de este proyecto es crear y desplegar un prototipo de una aplicación para compartir recursos académicos como apuntes y ejercicios resueltos entre otros. Para realizarlo se han definido los siguientes objetivos:

1. Diseñar e implementar una API REST en Go que gestione usuarios, recursos y asignaturas.
2. Desarrollar una interfaz web con SvelteKit que ofrezca una experiencia rápida y sin publicidad.
3. Integrar autenticación con Google OAuth2 como método único de acceso, detectando automáticamente el centro educativo del usuario a partir de su dominio de correo.
4. Obtener automáticamente centros educativos y sus asignaturas mediante *web scraping* y JetBrains SWOT.
5. Utilizar almacenamiento de objetos compatible con S3 para gestionar los archivos subidos por los usuarios.
6. Implementar búsqueda global mediante *Full Text Search* de PostgreSQL.
7. Desplegar la plataforma de forma portable usando Docker Compose en un VPS, haciéndola accesible públicamente en nemsy.org.

1.3 Plan de trabajo

El desarrollo del proyecto se ha llevado a cabo siguiendo un enfoque iterativo e incremental con una metodología *API-first*: para cada funcionalidad se implementaba primero el *endpoint* en el *backend* y después la interfaz correspondiente. Para la gestión de tareas se utilizó un tablero Kanban en Taiga. Las fases principales del desarrollo fueron las siguientes:

1. **Prototipado inicial (septiembre - octubre 2025).** Elección de tecnologías, implementación de la autenticación con Google OAuth2, primeras iteraciones del diseño de la interfaz y configuración inicial de la base de datos y el flujo de *onboarding*.
2. **Diseño de la arquitectura e implementación de las funcionalidades principales (enero - febrero 2026).** Definición de la API REST, sustituyendo un enfoque inicial con GraphQL que resultó innecesario para el proyecto. Desarrollo del *scraper* de la UCM, implementación de la subida, descarga y previsualización de recursos, soporte para múltiples archivos por recurso y diseño del sistema de vistas.
3. **Funcionalidades avanzadas y refinamiento (marzo 2026).** Perfiles de usuario, búsqueda global con *Full Text Search*, integración de universidades mediante JetBrains SWOT y *web scraping*, y mejoras generales de la interfaz.
4. **Pruebas, despliegue y documentación (abril 2026).** Tests unitarios del backend, tests unitarios y *end to end* del *frontend* con Playwright, configuración de Docker y Docker Compose, despliegue en VPS y redacción de la memoria.

1.4 Estructura del documento

Este documento está estructurado de la siguiente manera:

- **Capítulo 2. Estado de la Cuestión:** Se analizan las plataformas existentes para compartir recursos académicos.
- **Capítulo 3. Descripción del Trabajo:** Se justifican las decisiones tecnológicas y se describe en detalle la arquitectura, el diseño y la implementación de la plataforma.
- **Capítulo 4. Conclusiones y Trabajo Futuro:** Se presentan las conclusiones y las posibles líneas de mejora.

Chapter 1 Introduction

Abstract: This chapter presents the motivation behind the project, the objectives pursued and the structure of the document.

Talk is cheap. Show me the code.
— Linus Torvalds

1.1 Motivation

As a software engineering student I use a large amount of different software in my day to day life; some work well and some are frustrating. It is precisely one of the latter that motivated me to undertake this Bachelor's Thesis.

Before Wuolah, the alternatives were limited to generic platforms such as Docsity or StudyLib, which had not been targeted at the Spanish market and therefore had very little or practically no content depending on the platform, and which also did not integrate with specific educational institutions, or to informal solutions such as messaging groups and shared drives.

Wuolah, the best known and most integrated platform for sharing notes, solved a real problem by recognising the need for students to have a place to share and find relevant notes for their specific subjects, in their degree programme and educational institution. Despite its popularity, its current model presents a series of problems that harm both the user experience and the quality of information exchange.

Its main problem is performance. By analysing the platform using Lighthouse, an open source Google tool that audits the performance, accessibility, best practices and SEO of any website (Figure 3), some fairly conclusive results are obtained regarding its performance.

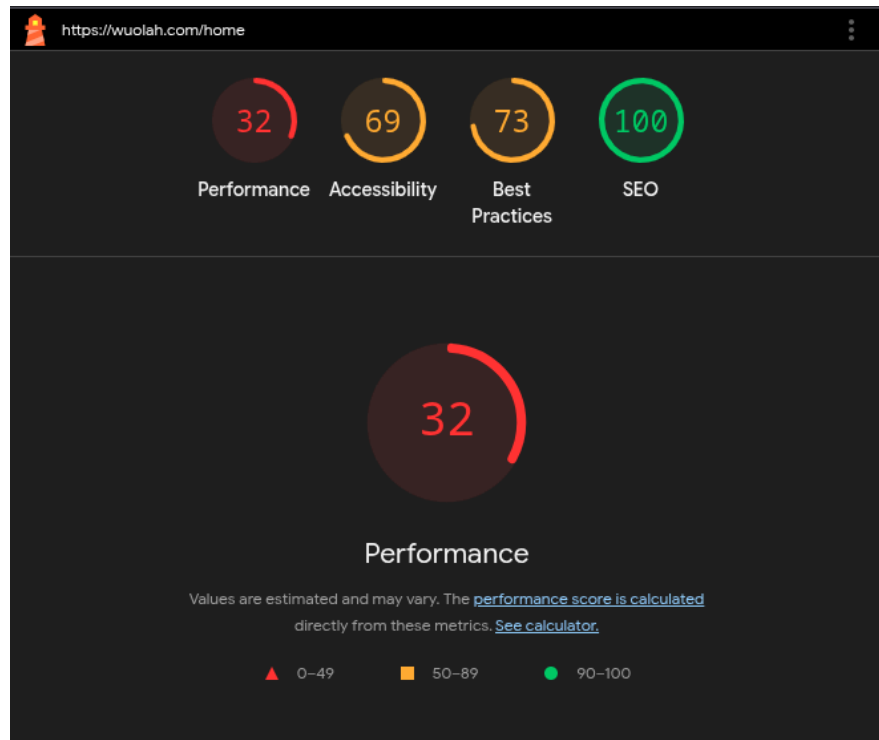


Figure 3: Performance analysis of wuolah.com with Google Lighthouse.

The platform achieves a performance score of 32 out of 100, a result that reveals a major optimisation problem and explains the slowness felt when using it. This low score can be explained by the use of a heavy client-side architecture and the large number of advertisements loaded on it. This significantly worsens the user experience and demonstrates the need for a more efficient design approach.

Furthermore, its business model based on intensive advertising also harms the experience. While it is understandable that the platform needs to monetise itself, the current implementation excessively penalises the free user with long waiting times to download each resource, unless a monthly subscription is paid.

On the other hand, the micropayments per download offered by Wuolah present another fundamental problem. When an economic reward is introduced, especially when it is minimal, there is a risk of transforming users' motivation; the genuine desire to help is replaced by a transactional interest of low value. This approach has been shown to be potentially counterproductive. Research on motivation has demonstrated that external rewards can diminish intrinsic interest in performing a task, an effect known as "Motivational Displacement" [2], [3].

Furthermore, this incentive model has another negative consequence for technical degrees. By limiting rewards to PDF files, since these are the ones in which advertisements can be embedded, it discourages students from sharing information in other formats more suitable for certain areas such as code, thus impoverishing the variety and usefulness of the resources available on the platform.

Finally, it is also worth mentioning how the current interface reflects a common problem in platforms that receive a significant percentage of usage from mobile

devices: to facilitate responsiveness, the desktop experience has been degraded. As can be seen in Figure 4, with a standard laptop resolution of 1920×1080 at 100% zoom, barely half of the screen is devoted to content, leaving the remaining side bars for advertisements. Of that half, slightly more than a third is occupied by a sidebar with low value information, leaving barely a fraction of the total screen width for what the user actually wants to see.

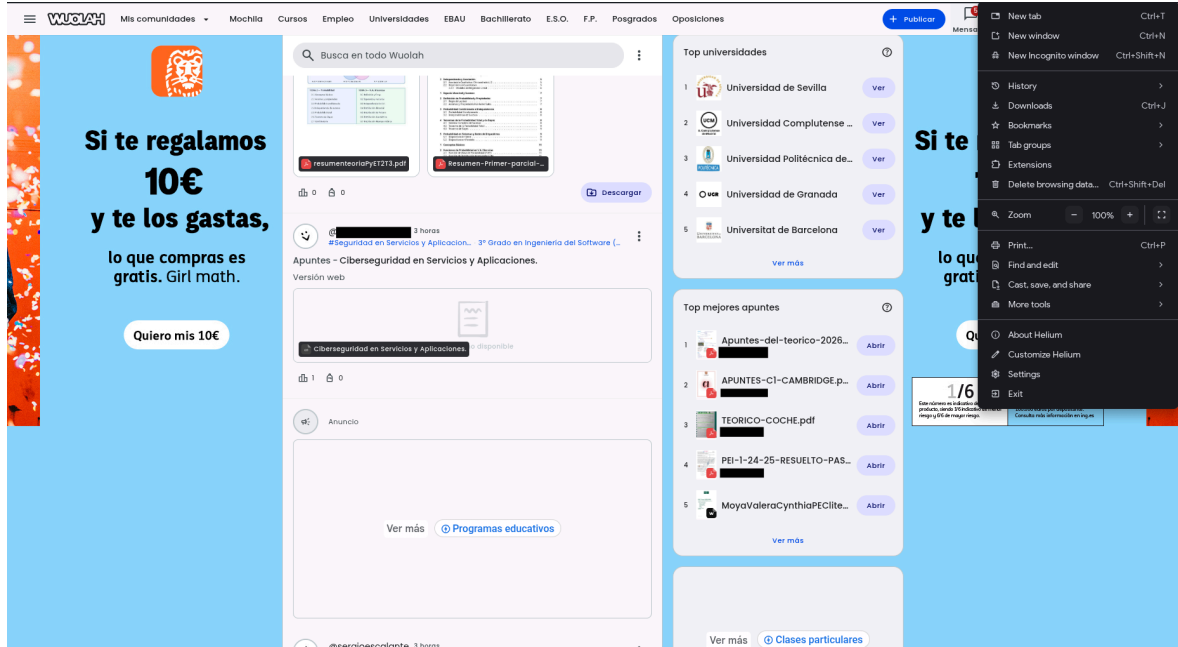


Figure 4: Screenshot of wuolah.com showing the current interface.

This work seeks to solve all these problems and aims to design and develop a prototype of an alternative platform. The proposed solution improves performance using a lightweight and efficient architecture with a Go backend and a SvelteKit frontend. To reduce maintenance costs, virtual private servers (VPS) are used instead of cloud platforms such as AWS. Intrusive advertising and monetary incentives are eliminated to foster genuine collaboration and the sharing of resources in multiple formats.

1.2 Objectives

The main objective of this project is to create and deploy a prototype of an application for sharing academic resources such as notes and solved exercises, among others. To achieve this, the following objectives have been defined:

1. Design and implement a REST API in Go that manages users, resources and subjects.
2. Develop a web interface with SvelteKit that offers a fast, advertisement free experience.

3. Integrate authentication with Google OAuth2 as the sole access method, automatically detecting the user's educational institution from their email domain.
4. Automatically retrieve educational institutions and their subjects through web scraping and JetBrains SWOT.
5. Use S3 compatible object storage to manage files uploaded by users.
6. Implement global search using PostgreSQL Full Text Search.
7. Deploy the platform in a portable manner using Docker Compose on a VPS, making it publicly accessible at nemsy.org.

1.3 Work Plan

The development of the project was carried out following an iterative and incremental approach with an API-first methodology: for each feature, the backend endpoint was implemented first, followed by the corresponding interface. For task management, a Kanban board on Taiga was used. The main development phases were as follows:

1. **Initial prototyping (September – October 2025).** Technology selection, implementation of Google OAuth2 authentication, first iterations of the interface design and initial configuration of the database and the onboarding flow.
2. **Architecture design and implementation of core features (January – February 2026).** Definition of the REST API, replacing an initial GraphQL approach that proved unnecessary for the project. Development of the UCM scraper, implementation of resource uploading, downloading and preview, support for multiple files per resource, and design of the view system.
3. **Advanced features and refinement (March 2026).** User profiles, global search with Full Text Search, university integration via JetBrains SWOT and web scraping, and general interface improvements.
4. **Testing, deployment and documentation (April 2026).** Backend unit tests, frontend unit and end to end tests with Playwright, Docker and Docker Compose configuration, VPS deployment and writing of the thesis report.

1.4 Document Structure

This document is structured as follows:

- **Chapter 2. State of the Art:** Existing platforms for sharing academic resources are analysed.
- **Chapter 3. Work Description:** The technological decisions are justified and the architecture, design and implementation of the platform are described in detail.
- **Chapter 4. Conclusions and Future Work:** Conclusions and possible lines of improvement are presented.

Capítulo 2 Estado de la Cuestión

Resumen: Este capítulo analiza el estado del arte de las plataformas colaborativas para compartir recursos académicos.

Adapta lo que sea útil, rechaza lo que sea inútil y añade lo que sea específicamente tuyo.

— Bruce Lee

2.1 Plataformas existentes

A continuación, se analizan las tres plataformas más relevantes para compartir recursos académicos, evaluando su integración con centros educativos, su modelo de monetización y la experiencia de usuario que ofrecen.

2.1.1 Wuolah

Wuolah es una plataforma española que ha sabido integrar las asignaturas, cursos y centros educativos para que sus usuarios puedan compartir y acceder a apuntes y exámenes verdaderamente relevantes para ellos. No se centra únicamente en estudios universitarios, también ofrece soporte para E.S.O, Bachillerato, EBAU, Ciclos y Oposiciones. Además, es la única de las plataformas analizadas que ofrece una compensación monetaria a los usuarios que publican sus apuntes, un aspecto cuyas implicaciones ya se han discutido en la motivación de este trabajo.

El proceso de registro es sencillo para lo que cubre la plataforma ya que intenta abarcar mucho más que solo estudios universitarios. Se puede iniciar sesión con correo y contraseña o mediante OAuth2, tras lo cual se selecciona el país, el tipo de estudio, el estudio concreto y el centro educativo. Sin embargo, el último paso requiere aceptar su política de privacidad, cuya presentación incluye patrones oscuros como lenguaje emocional y apelaciones a la urgencia del estudiante (Figura 5). Además, junto a la política se solicita consentimiento para recibir publicidad de empresas de sectores ajenos a la educación.

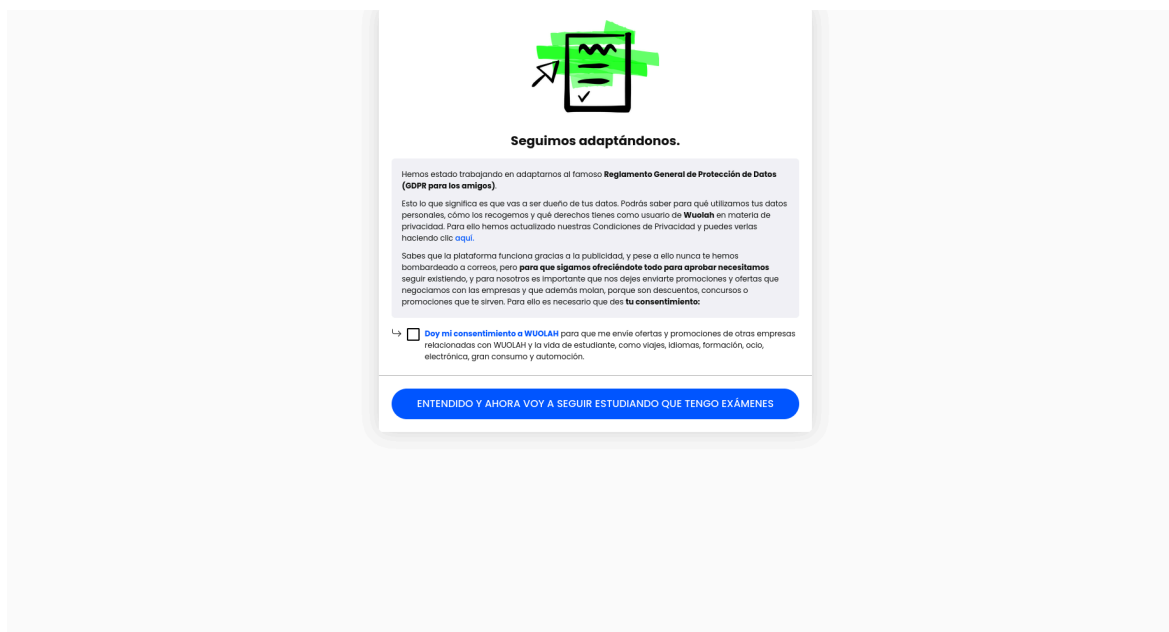


Figura 5: Pantalla de aceptación de la política de privacidad de Wuolah.

Esto va acompañado de los problemas anteriormente mencionados como el exceso de publicidad que interfiere con el uso de la plataforma, tiempos de espera de hasta medio minuto para que un usuario gratuito pueda descargar un apunte, y una interfaz de escritorio poco eficiente para facilitar la responsividad en móvil. A esto se suma una barra de navegación con demasiada información: si un usuario ya ha indicado que cursa un grado universitario, de poco le sirve tener acceso directo a los apartados de E.S.O, Bachillerato o EBAU (Figura 6).



Figura 6: Barra de navegación de Wuolah.

En conclusión, Wuolah es la plataforma que mejor ha resuelto el flujo de usuario para compartir recursos académicos y, con diferencia, la que mayor volumen de contenido tiene para universidades españolas. Sin embargo, sus defectos han ido acumulándose con el tiempo a medida que la plataforma ha crecido en funcionalidades y ha intensificado su modelo de monetización.

2.1.2 Studocu

Studocu es una plataforma fundada en 2013 en los Países Bajos que ha crecido hasta convertirse en la mayor plataforma de apuntes a nivel global. También ofrece integración con centros educativos, aunque algo más superficial que la de Wuolah, permite seleccionar universidad y asignaturas, pero no grado, por lo que asignaturas de mismo nombre compartidas entre grados se agrupan aunque puedan tener un temario diferente. Dejando ese detalle de lado, como se puede ver en la Figura 7, la interfaz de escritorio es excelente, aprovecha bien el ancho de la pantalla y tiene una buena densidad de información. Sin embargo, al no ser tan popular en España,

muchas asignaturas de grados como Ingeniería de Software no aparecen, y las que sí lo hacen tienen varias veces menos contenido que en Wuolah.

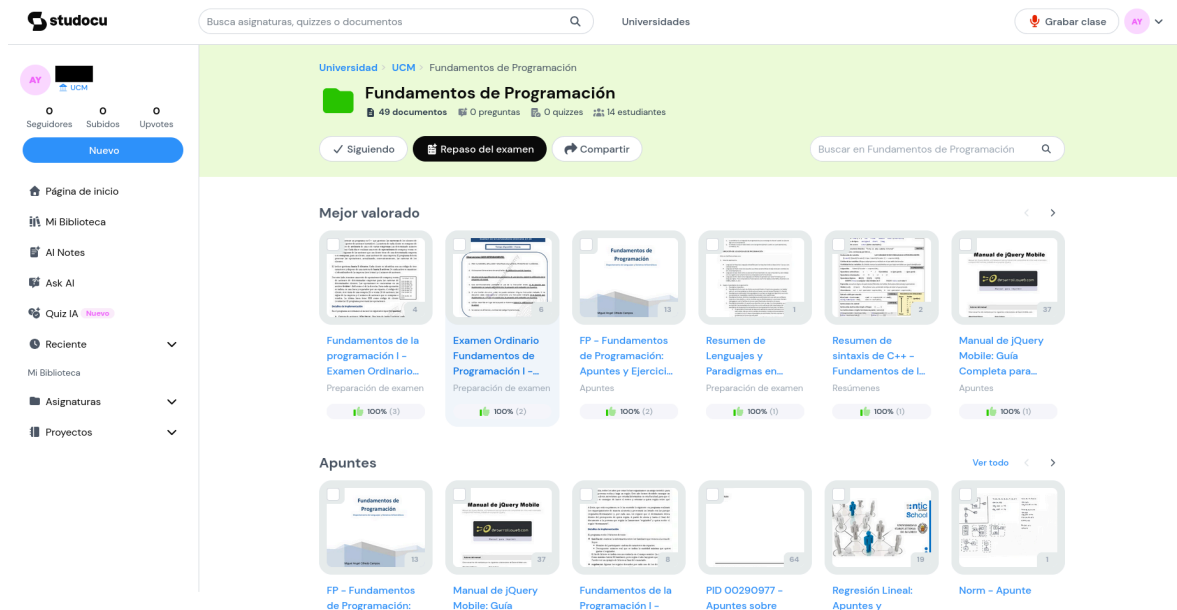


Figura 7: Captura de pantalla de Studocu.

Por último, aunque lejos del alcance de este TFG, cabe destacar que Studocu cuenta con una gran implementación de inteligencia artificial que recopila automáticamente la información de la asignatura y ayuda al estudiante a repasar con ese contexto. Wuolah también ofrece una funcionalidad similar, aunque todavía en fase beta y más limitada, siendo un *chatbot* al que se le pueden adjuntar apuntes concretos.

2.1.3 Docsity

Docsity es una plataforma italiana lanzada en 2010 que se abrió al mercado internacional en 2012. Ofrece una integración con centros educativos que incluye universidad, área de estudio, asignaturas y carrera, aunque al igual que Studocu no hay segregación de asignaturas por grado, por lo que la selección de carrera no tiene un efecto real en el contenido que se muestra.

La plataforma utiliza un sistema de puntos. Para incentivar las contribuciones los usuarios obtienen puntos al subir documentos o responder preguntas, y los gastan al descargar contenido de otros. Si bien evita la publicidad tan agresiva de Wuolah, este sistema de puntos puede resultar frustrante cuando un usuario quiere acceder a un recurso puntual y no tiene saldo suficiente.

En cuanto a presencia en España, Docsity cuenta con un volumen notable de contenido para algunas universidades como la UCM, aunque su distribución es desigual, carreras técnicas como Ingeniería de Software apenas tienen recursos disponibles, concentrándose la mayoría del contenido en áreas como periodismo, derecho o historia.

La interfaz (Figura 8) es limpia pero algo más confusa que las de Wuolah o Studocu ya que dentro de una asignatura solo los documentos populares muestran una miniatura, mientras que el resto se presenta en una lista sin previsualización, y la disposición general resulta menos intuitiva.

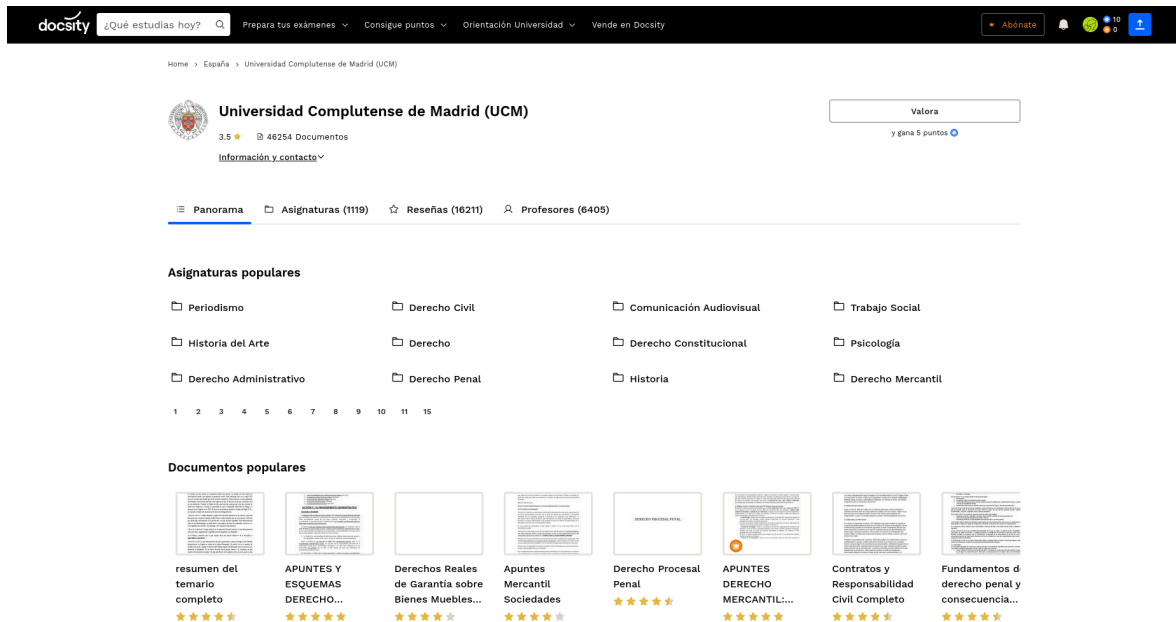


Figura 8: Captura de pantalla de Docsity para la UCM.

2.1.4 Conclusiones

Las tres plataformas analizadas comparten un objetivo común pero presentan carencias distintas. Wuolah ofrece la mejor integración con centros educativos españoles pero a costa del rendimiento y la experiencia de usuario. Studocu destaca por su interfaz y su alcance global, aunque su integración es más superficial y su contenido en España es limitado. Docsity, por su parte, tiene un modelo de incentivos menos intrusivo pero una organización del contenido poco precisa. Ninguna de las tres combina una integración completa con centros educativos, un buen rendimiento y una experiencia de usuario libre de publicidad o restricciones artificiales, lo que justifica la propuesta desarrollada en este trabajo.

Capítulo 3 Descripción del Trabajo

Resumen: En este capítulo se justifican las decisiones tecnológicas y se describe en detalle el desarrollo de la plataforma Nemsy.

La calidad de un sistema no está determinada por el poder de sus componentes individuales, sino por cómo se eligen y se ensamblan para satisfacer las necesidades.

— Grady Booch

3.1 Tecnologías evaluadas

En esta sección se justifican las principales decisiones tecnológicas del proyecto, comparando las opciones consideradas para cada componente de la arquitectura.

3.1.1 Backend

Para el backend se evaluaron tres alternativas con las que ya tenía experiencia: Node.js con Express, Spring Boot (Java) y Go con Chi, siendo esta última con la que más cómodo me sentía. Dos de ellas figuran entre los frameworks web más utilizados según la encuesta anual de Stack Overflow a desarrolladores [1], como se puede observar en la Figura 9.

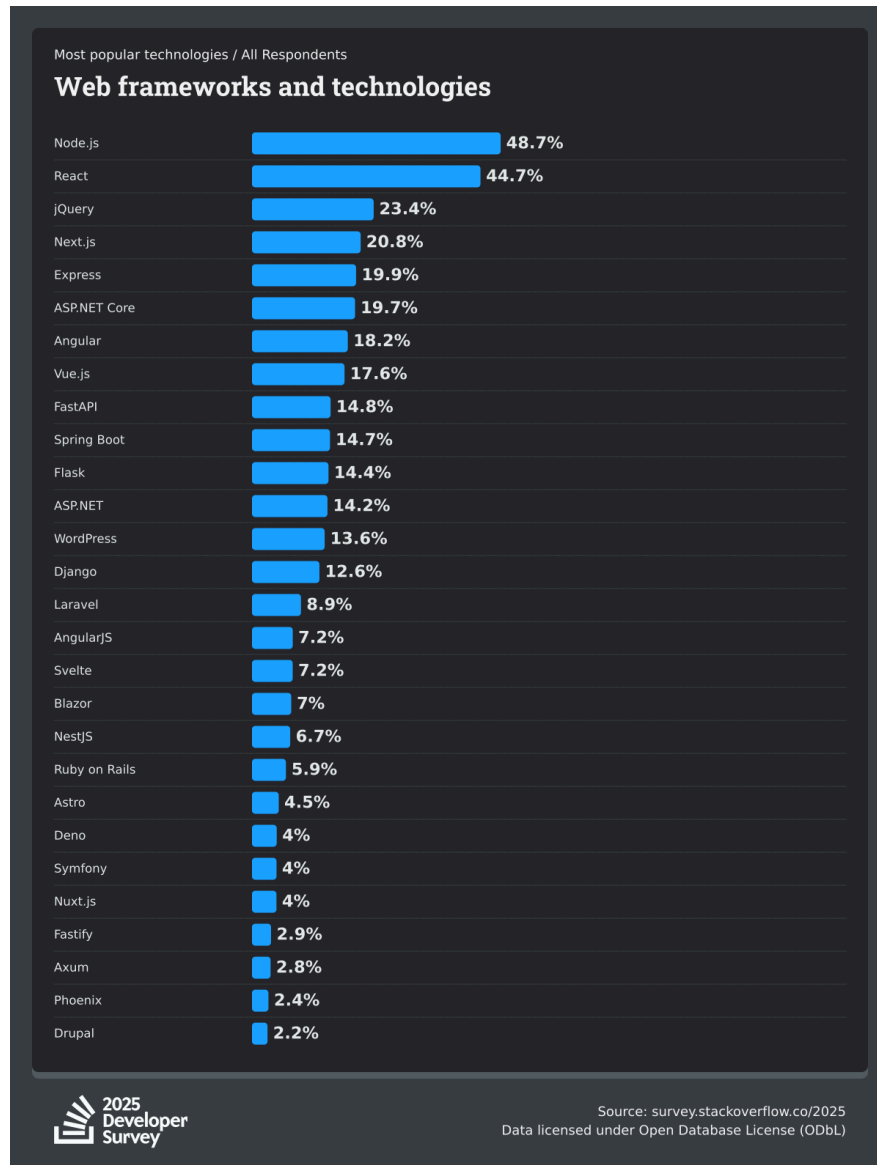


Figura 9: Frameworks y tecnologías web más populares según la encuesta de Stack Overflow 2025 [1].

3.1.1.1 Node.js (Express)

Express es un framework web minimalista escrito sobre Node.js. Como refleja la Figura 9, se sitúa entre los frameworks web más utilizados a nivel global, lo que se traduce en un ecosistema muy maduro y una gran cantidad de recursos disponibles. Además, es el framework utilizado en la asignatura de Aplicaciones Web del grado, por lo que ya se contaba con experiencia previa en su uso.

En cuanto a la experiencia de desarrollo, Express es excelente, es directo y sencillo, y su principal punto fuerte es la velocidad con la que se puede construir un servicio. Sin embargo, el objetivo de este proyecto no es maximizar la velocidad de desarrollo, sino ofrecer una plataforma que se sienta rápida para el usuario final, y Node.js, al ser interpretado y basarse en un único hilo de ejecución, no es la opción más adecuada para ello. Es por ello que fue descartada.

3.1.1.2 Spring Boot

Spring Boot es el framework más popular del ecosistema Java y también aparece en la Figura 9 con una presencia notable. Proporciona un entorno muy completo con inyección de dependencias, ORM integrado y una amplia cobertura para aplicaciones empresariales. Este es el framework utilizado en la asignatura de Ingeniería Web del grado, por lo que también se contaba con experiencia previa en su uso.

Su experiencia de desarrollo también es destacable una vez superado el boilerplate inicial, aunque con un matiz. La filosofía del ecosistema Java se apoya en múltiples capas de abstracción, lo cual puede dificultar la depuración cuando algo falla. Además, para un proyecto de este tamaño, contar con ORM e inyección de dependencias resulta más una carga que una ventaja. A esto se suma el elevado consumo de memoria de la JVM, un factor relevante al desplegar en un VPS con recursos limitados. Por todo ello, también fue descartada.

3.1.1.3 Go (Chi)

Go es un lenguaje de programación creado por Google, compilado y con un fuerte enfoque en servicios web y cloud [4]. Su librería estándar incluye el módulo `net/http`, lo suficientemente sólido como para construir un servidor HTTP sin dependencias externas. Chi es una extensión ligera sobre este módulo que añade utilidades como el enrutamiento con parámetros y middlewares encadenables [5]. Además, Go cuenta con un sistema de testing integrado en el propio lenguaje y un tooling moderno que cubre desde el formateo de código hasta la gestión de dependencias.

Frente a la filosofía de Java, donde las abstracciones ocultan los detalles de implementación, el ecosistema de Go apuesta por la generación de código y la transparencia. Herramientas como `sqlc` generan código Go a partir de consultas SQL escritas a mano, de forma que el desarrollador mantiene el control total sobre las queries sin depender de un ORM. Esto resulta especialmente relevante cuando se quieren aprovechar funcionalidades específicas del motor de base de datos elegido, algo que un ORM tiende a dificultar y cuya importancia se detallará en la sección de base de datos.

3.1.1.4 Comparativa

Criterio	Node.js 	Spring Boot 	Go 
Rendimiento	Medio (V8, single-thread)	Alto (JVM, JIT)	Alto (compilado, nativo)
Concurrencia	Event loop, un hilo	Threads del SO	Goroutines (ligeras)
Consumo de recursos	Medio	Alto (JVM)	Bajo (binario único)

Ecosistema HTTP	Muy amplio (Express)	Muy amplio (Spring MVC)	Sólido (net/http, Chi)
Despliegue	Requiere Node + node_modules	Requiere JVM + JAR	Binario único
Curva de aprendizaje	Ya conocido	Ya conocido	Ya conocido

Tabla 1: Comparativa de tecnologías para el backend.

Como se resume en la Tabla 1, Go destaca frente a las otras dos alternativas en consumo de recursos y despliegue, al compilar a un binario único sin dependencias externas. Además, su modelo de concurrencia basado en goroutines permite manejar operaciones simultáneas como descargas de archivos de forma nativa, sin configuración adicional. Dado que el foco de este TFG está muy ligado a la velocidad y el bajo consumo de recursos de la plataforma, Go resultó la opción más adecuada.

3.1.2 Frontend

Para el frontend se consideraron tres enfoques con filosofías distintas: un framework completo con React (Next.js), un framework ligero con Svelte (SvelteKit), y el enfoque clásico de servidor con templating y Bootstrap, utilizado en varias asignaturas del grado como Aplicaciones Web (EJS) e Ingeniería Web (Thymeleaf).

3.1.2.1 Servidor con templating + Bootstrap

El enfoque más directo y el más familiar por las asignaturas del grado. Con motores como EJS o Thymeleaf, el HTML se renderiza en el servidor y se envía completo al navegador, mientras que Bootstrap proporciona estilos predefinidos y componentes básicos. Es sencillo de entender y desplegar, pero presenta limitaciones claras para una aplicación interactiva: cada acción del usuario requiere una recarga completa de la página, la experiencia resulta lenta y poco fluida, y la personalización visual está limitada por los estilos de Bootstrap. Para una plataforma que aspira a competir en experiencia de usuario con aplicaciones modernas, este enfoque resulta insuficiente.

3.1.2.2 Next.js (React)

Next.js es el meta framework más popular del ecosistema React y uno de los más utilizados en la industria, como refleja la Figura 9. Ofrece renderizado del lado del servidor (SSR), generación estática, enrutamiento basado en ficheros y un ecosistema de componentes y librerías enorme. Sin embargo, React se basa en un virtual DOM que añade una capa de abstracción entre el código y el navegador [6], lo que resulta en un bundle más pesado. Además, no tenía experiencia previa con React, lo que habría añadido una curva de aprendizaje significativa al proyecto.

3.1.2.3 SvelteKit (Svelte)

SvelteKit es un meta-framework construido sobre Svelte, un compilador que transforma los componentes a JavaScript vanilla durante el build, eliminando la necesidad de un virtual DOM en tiempo de ejecución [7]. Esto resulta en bundles significativamente más pequeños y un mejor rendimiento en el navegador. Ya tenía experiencia con Svelte y SvelteKit, y además Svelte 5 introduce las runes, un sistema de reactividad más explícito y eficiente que el de versiones anteriores.

SvelteKit ofrece las mismas capacidades que Next.js (SSR, enrutamiento basado en ficheros, layouts anidados) pero con un enfoque más ligero. Al igual que con Go en el backend, la elección se alinea con el objetivo del proyecto de ofrecer una experiencia rápida y eficiente.

3.1.2.4 Comparativa

Criterio	Templating + Bootstrap 	Next.js (React) 	SvelteKit (Svelte) 
Rendimiento	Bajo (recarga completa)	Medio (virtual DOM)	Alto (compilado, sin virtual DOM)
Interactividad	Mínima (MPA)	Alta (SPA/SSR)	Alta (SPA/SSR)
Tamaño del bundle	Bajo (sin framework de JS)	Alto	Muy bajo
Personalización IU	Limitada (Bootstrap)	Total	Total
Ecosistema	Maduro pero limitado	Muy amplio	Creciente
Curva de aprendizaje	Ya conocido	Nuevo	Ya conocido

Tabla 2: Comparativa de tecnologías para el frontend.

Se eligió SvelteKit por su combinación de rendimiento, tamaño de bundle reducido y mi familiaridad con el framework. Al compilar a JavaScript vanilla, se alinea directamente con el objetivo del proyecto de ofrecer una plataforma ligera y rápida.

3.1.3 Base de datos

Los datos de Nemsy son inherentemente relacionales. Los usuarios, universidades, asignaturas, recursos y archivos están interconectados mediante claves foráneas. Por ello, la decisión entre modelos de datos se inclinó desde el inicio hacia un sistema relacional, pero se evaluaron igualmente las alternativas con las que se tenía experiencia.

3.1.3.1 MySQL

MySQL es el SGBD relacional de código abierto más extendido y el utilizado en asignaturas del grado como Modelado de Software y Aplicaciones Web. Es robusto, bien documentado y cuenta con un ecosistema enorme [8]. Sin embargo, su soporte de búsqueda de texto completo (FULLTEXT) es limitado, no permite asignar pesos a distintos campos, carece de un sistema de ranking comparable a `ts_rank`, y no dispone de un mecanismo nativo para ignorar acentos, algo imprescindible en una plataforma en español. Para conseguir una experiencia de búsqueda equivalente habría sido necesario recurrir a un servicio externo como Elasticsearch, añadiendo complejidad al despliegue.

3.1.3.2 MongoDB

MongoDB es una base de datos documental NoSQL que almacena los datos en formato BSON (similar a JSON). Es una opción popular para prototipos rápidos y esquemas que cambian con frecuencia [9], y fue uno de los SGBD utilizado en la asignatura de Ampliación de Bases de Datos del grado. Sin embargo, el modelo de datos de Nemsy se basa en relaciones claras entre entidades, algo que MongoDB no gestiona de forma nativa, las consultas que implican joins requieren múltiples peticiones o desnormalizar los datos, lo que complica el mantenimiento y compromete la consistencia. Además, `sqlc`, la herramienta de generación de código elegida para el backend, solo soporta bases de datos SQL.

3.1.3.3 PostgreSQL

PostgreSQL es un SGBD relacional de código abierto con una licencia permisiva (PostgreSQL License), con el que ya contaba con experiencia por proyectos personales. Su principal ventaja frente a MySQL para este proyecto es el sistema de búsqueda de texto completo integrado [10]. Mediante columnas de tipo `tsvector`, índices GIN y la función `ts_rank`, PostgreSQL permite construir búsquedas con pesos por campo y ranking de relevancia sin dependencias externas. Además, la extensión `unaccent` permite ignorar tildes en las consultas, algo fundamental para texto en español. Estas funcionalidades se integran directamente en el esquema mediante columnas generadas, de forma que el índice de búsqueda se actualiza automáticamente con cada inserción o modificación.

3.1.3.4 Comparativa

Criterio	MySQL	MongoDB	PostgreSQL
			
Modelo de datos	Relacional	Documental (NoSQL)	Relacional
Licencia	GPL	SSPL	PostgreSQL (permisiva)

Full-text search	Básico (FULLTEXT)	Básico	Avanzado (tsvector, ts_rank)
Soporte de acentos	No nativo	No nativo	Sí (unaccent)
Soporte en sqlc	Sí	No	Sí (nativo)
Curva de aprendizaje	Ya conocido	Ya conocido	Ya conocido

Tabla 3: Comparativa de sistemas de gestión de bases de datos.

Como se resume en la Tabla 3, PostgreSQL fue la elección natural. Es el único que resuelve la búsqueda de texto en español de forma nativa, sin añadir servicios externos al despliegue, y su compatibilidad con sqlc encaja directamente con la arquitectura del backend.

3.1.4 Almacenamiento

El almacenamiento de archivos es una pieza central en una plataforma como Nemsy, donde los usuarios suben y descargan documentos de forma continua. Se evaluaron dos enfoques para gestionar estos archivos.

La opción más simple es almacenar los archivos directamente en el sistema de ficheros del VPS. Es inmediata de implementar y no introduce dependencias externas, pero presenta limitaciones importantes, el espacio en disco de un VPS es limitado y caro por gigabyte, y más importante, los ficheros quedan ligados a una sola máquina y cualquier migración implica mover manualmente todo el contenido.

El almacenamiento de objetos es el enfoque estándar en la industria para este tipo de casos. AWS S3 es el servicio de referencia y su API se ha convertido en estándar de facto, pero su modelo de precios incluye costes de egreso que pueden resultar en facturas impredecibles en una plataforma orientada a descargas. Hetzner Object Storage ofrece la misma API compatible con S3 y un modelo de precios más predecible, con un coste fijo mensual por TB almacenado que incluye tráfico sin cargos adicionales por egreso. Al estar ubicado en centros de datos europeos, facilita además el cumplimiento del RGPD, un aspecto relevante para una plataforma que almacena documentos de usuarios de la Unión Europea.

La integración se realiza a través de la librería `minio-go`, que implementa el protocolo S3 de forma genérica y permite conectar con cualquier proveedor compatible mediante variables de entorno, sin cambios en el código.

3.1.5 Despliegue

La plataforma se despliega en un VPS con recursos limitados, por lo que la solución de despliegue debía ser ligera, reproducible y fácil de mantener por una sola persona.

Una alternativa habría sido desplegar los servicios manualmente, instalando Go, Node.js y PostgreSQL directamente en el sistema operativo del servidor y gestionando

los procesos con `systemd`. Es el enfoque más simple en términos de herramientas, pero también el más frágil, ya que el entorno del servidor se convierte en un estado implícito difícil de reproducir y actualizar una dependencia puede romper el resto.

Kubernetes es la solución estándar para orquestación de contenedores a escala, pero introduce una complejidad operativa desproporcionada para un proyecto de este tamaño. Requiere un clúster, recursos considerables y conocimientos específicos de administración, aspectos que no se justifican cuando la plataforma corre en una sola máquina, pero que sí serían valiosos si en el futuro fuese necesario escalar a múltiples máquinas.

Docker Compose permite definir todos los servicios de la aplicación (backend, frontend y base de datos) en un único fichero `docker-compose.yml`, levantarlos con un solo comando y garantizar que el entorno es idéntico en local y en producción. Cada servicio corre en su propio contenedor aislado, las variables de entorno se gestionan mediante un fichero `.env`, y el estado persistente de PostgreSQL se mantiene en un volumen nombrado. Es la opción que mejor equilibra reproducibilidad y simplicidad en el contexto de este proyecto.

3.2 Arquitectura del sistema

Nemsys sigue una arquitectura cliente-servidor de tres capas. El frontend es una *Single Page Application* (SPA) desarrollada con SvelteKit que se comunica con el backend exclusivamente a través de una API REST. El backend, implementado en Go con el router Chi, actúa como única puerta de entrada al sistema gestionando la autenticación, la lógica de negocio y el acceso a los dos sistemas de persistencia, PostgreSQL y un almacenamiento de objetos compatible con S3. El conjunto se despliega mediante Docker Compose en un VPS, de forma que el entorno de producción es reproducible e idéntico al de desarrollo local.

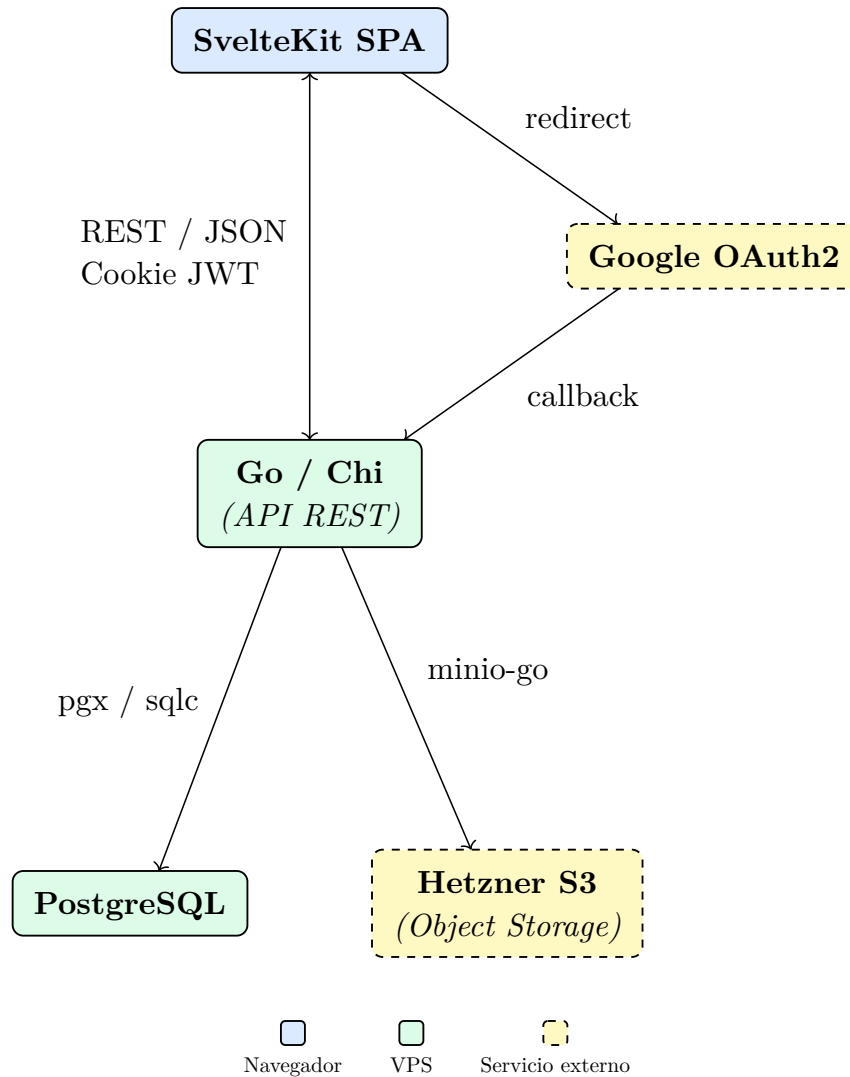


Figura 10: Diagrama de componentes de la arquitectura de Nemsy.

3.2.1 Frontend

El frontend está construido con SvelteKit, que organiza la aplicación en torno a un sistema de enrutamiento basado en el sistema de ficheros: cada carpeta dentro de `src/routes/` define una ruta, y los ficheros con nombres especiales (`+page.svelte`, `+layout.svelte`, `+page.server.ts`) tienen roles concretos dentro del ciclo de vida de la página. La Tabla 4 recoge las rutas de la aplicación.

Ruta	Descripción
/	Página de inicio: <i>hero</i> de bienvenida para usuarios no autenticados; lista de asignaturas y recursos para usuarios autenticados.
/search	Búsqueda global de recursos mediante Full Text Search.
/create	Formulario de subida de un nuevo recurso con sus archivos.
/user/[slug]	Perfil público de un usuario y sus recursos compartidos.
/auth	Pantalla de inicio de sesión con Google OAuth2.

Ruta	Descripción
/admin	Panel de administración para gestionar reportes (solo administradores).

Tabla 4: Rutas de la aplicación frontend.

El fichero `+layout.svelte` de la raíz actúa como *shell* de toda la aplicación y define dos disposiciones de navegación completamente distintas según el dispositivo. En escritorio se muestra una barra de navegación superior clásica con los enlaces principales centrados y el perfil de usuario a la derecha. En móvil esta barra desaparece y es sustituida por una barra de navegación fija en la parte inferior de la pantalla, siguiendo el patrón habitual en aplicaciones móviles, acompañada de un botón de acción flotante (FAB) para subir un recurso. Esta separación es un cambio de paradigma de navegación adaptado a cada contexto de uso. La interfaz también se adapta en función del estado de autenticación expuesto por `data.me`. La ruta `/auth` queda excluida de este shell mediante su propio `+layout.svelte` independiente, de forma que la pantalla de inicio de sesión se renderiza sin navegación.

Las páginas que necesitan datos del backend utilizan un fichero `+page.server.ts` con una función `load`, que SvelteKit ejecuta en el servidor antes de renderizar la página. La función accede a las cookies de sesión y al resultado del `load` del layout padre mediante `parent()`, evitando repetir la llamada de autenticación en cada página y garantizando que la primera carga llegue al navegador ya con los datos.

3.2.2 Backend

El backend es un servidor HTTP escrito en Go que expone la API REST en el puerto 8080. Está organizado en paquetes por dominio funcional, cada uno con su propio *handler*. Todos los *handlers* reciben una instancia del struct `App`, que agrupa las tres dependencias compartidas del sistema: el cliente de base de datos (`Queries`, generado por `sqlc`), el *pool* de conexiones a PostgreSQL (`DB`, para transacciones) y el cliente S3 (`Storage`). Este patrón evita la necesidad de un framework de inyección de dependencias, manteniendo el control explícito sobre las dependencias sin añadir capas de abstracción innecesarias.

```

1 type App struct {
2     DB      *pgxpool.Pool
3     Queries QuerierWithTx
4     Storage *storage.S3Client
5 }
```

go

Código 1: Struct `App` que agrupa las dependencias compartidas del backend.

Los paquetes del backend son los siguientes:

Paquete	Descripción
auth	Implementa el flujo OAuth2 con Google, la generación y validación de tokens JWT y el middleware de autenticación. También expone el middleware <code>AdminOnly</code> para restringir el acceso a rutas administrativas.
resources	Gestiona el ciclo de vida completo de los recursos: creación con subida de archivos a S3, consulta, descarga (tanto del paquete completo como de ficheros individuales), eliminación y sistema de reportes.
users	Expone el perfil del usuario autenticado y permite actualizar su universidad, estudio y asignaturas fijadas.
studies universities	Proporcionan endpoints de consulta y búsqueda por texto sobre estudios y universidades.
admin	Rutas protegidas exclusivamente para administradores que permiten revisar y gestionar los reportes de recursos.
storage	Encapsula el cliente <code>minio-go</code> para interactuar con cualquier proveedor S3-compatible, configurable mediante variables de entorno.
db/generated	Código Go generado automáticamente por <code>sqlc</code> a partir de las consultas SQL escritas a mano en el esquema.

Tabla 5: Paquetes del backend y su responsabilidad.

3.2.3 Autenticación

La autenticación se delega completamente a Google OAuth2, por lo que el usuario no gestiona credenciales propias en Nemsy, sino que inicia sesión con su cuenta de Google. Una vez autenticado, el backend mantiene la sesión mediante un JSON Web Token firmado y almacenado en una *cookie* del navegador, que se valida en cada petición por el *middleware* `AuthMiddleware`. Las rutas administrativas pasan además por un segundo *middleware* `AdminOnly` que comprueba el rol del usuario. Los detalles internos del flujo (anti-CSRF, validación del *ID Token*, estructura del JWT y cadena de *middlewares*) se desarrollan en Sección 3.5.6.

3.2.4 Despliegue

El diagrama de la Figura 10 describe la arquitectura lógica del sistema, pero no cómo se materializa en el servidor. En producción, los tres componentes alojados en el VPS (frontend, backend y base de datos) corren cada uno en su propio contenedor Docker, orquestados por un único fichero `docker-compose.yml`. La Figura 11 ilustra esta disposición.

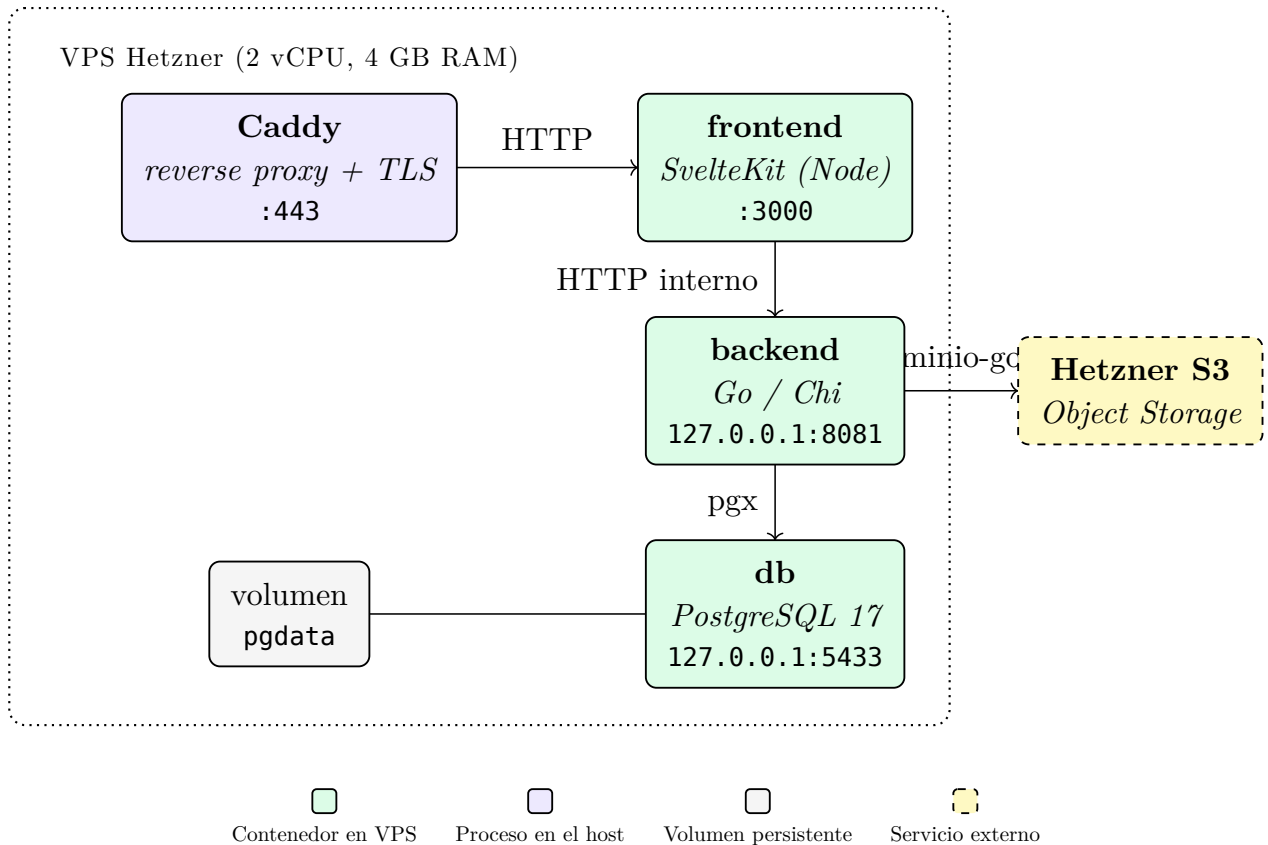


Figura 11: Arquitectura de despliegue en el VPS con Docker Compose.

Los tres servicios comparten una red interna gestionada por Docker, de forma que se comunican entre sí mediante el nombre del servicio (**db**, **backend**) sin exponer esos puertos al exterior. Los puertos del backend y la base de datos se publican únicamente en la interfaz de *loopback* (127.0.0.1:8081 y 127.0.0.1:5433), accesibles desde el propio VPS para tareas de administración pero no desde Internet. El tráfico HTTPS público llega al frontend a través de Caddy [11], un reverse proxy instalado en el host que termina TLS y reenvía las peticiones al contenedor correspondiente. Caddy fue elegido ante alternativas como Nginx por su gestión automática de certificados TLS mediante Let's Encrypt sin configuración adicional, de esta forma se consiguen renovar los certificados sin necesidad de intervención manual.

El estado persistente de PostgreSQL se almacena en un volumen nombrado **pgdata**, fuera del ciclo de vida del contenedor, permitiendo poder reemplazar los contenedores sin perder los datos. Los archivos subidos por los usuarios no se guardan en el VPS, sino en Hetzner Object Storage a través de la librería **minio-go**. Esto mantiene la máquina prácticamente sin estado.

Toda la configuración sensible como los credenciales de base de datos, el secreto JWT, o las claves de OAuth2 y S3, se inyectan a los contenedores mediante variables de entorno definidas en un fichero **.env** presente únicamente en el servidor. De este modo, el mismo **docker-compose.yml** sirve tanto para desarrollo local como para producción, cambiando solo el contenido del **.env**, y el entorno resulta reproducible con un único comando.

```
1  services:
2    db:
3      image: postgres:17-alpine
4      restart: unless-stopped
5      environment:
6        POSTGRES_USER: nemsy
7        POSTGRES_PASSWORD: ${DB_PASSWORD}
8        POSTGRES_DB: nemsy
9      volumes:
10       - pgdata:/var/lib/postgresql/data
11      ports:
12       - "127.0.0.1:5433:5432"
13
14    backend:
15      build: ./backend
16      restart: unless-stopped
17      depends_on: [db]
18      environment:
19        DATABASE_URL: postgresql://nemsy:${DB_PASSWORD}@db:5432/nemsy?
20          sslmode=disable
21        JWT_SECRET: ${JWT_SECRET}
22        # ... OAuth2 y credenciales S3
23      ports:
24       - "127.0.0.1:8081:8080"
25
26    frontend:
27      build:
28        context: ./frontend
29      args:
30        PUBLIC_API_BASE_URL: ${PUBLIC_API_BASE_URL}
31      restart: unless-stopped
32      depends_on: [backend]
33      ports:
34       - "3000:3000"
35
36    volumes:
37      pgdata:
```

Código 2: Fragmento del docker-compose.yml de Nemsy.

3.3 Diseño de la interfaz de usuario

La interfaz de Nemsy se ha diseñado en paralelo al desarrollo del backend, partiendo de una primera versión funcional pero poco refinada que ha ido madurando hasta el lenguaje visual actual. En esta sección se recoge esa evolución y los tres pilares que sostienen el diseño final, el sistema de componentes, la tipografía y la paleta de color, que en conjunto buscan ofrecer una interfaz sobria pero no fría, alineada con el objetivo de competir en experiencia de usuario con plataformas como Wuolah.

3.3.1 Evolución del diseño

El diseño de Nemsy evolucionó de forma iterativa a lo largo del desarrollo. La primera versión, visible en la Figura 12, ya establecía la estructura que se mantiene hoy, con islas blancas sobre fondo de color y retícula de tres columnas, pero el lenguaje visual era más informal, con bordes negros gruesos, esquinas redondeadas y fondo azul pálido. A medida que se añadían nuevas pantallas, el diseño fue derivando hacia una estética más sobria, sustituyendo los bordes gruesos por líneas finas, las esquinas redondeadas por rectas y el azul pálido por un gris neutro. La Figura 13 muestra el estado actual en los dos modos de visualización disponibles.

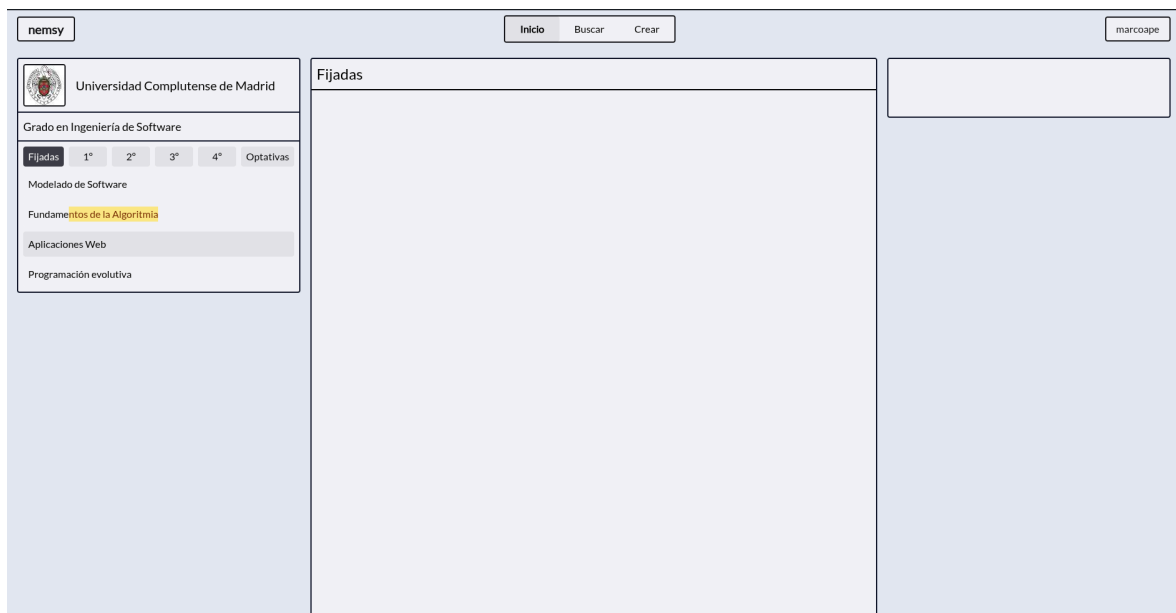


Figura 12: Primera versión de la interfaz de Nemsy (enero 2026).

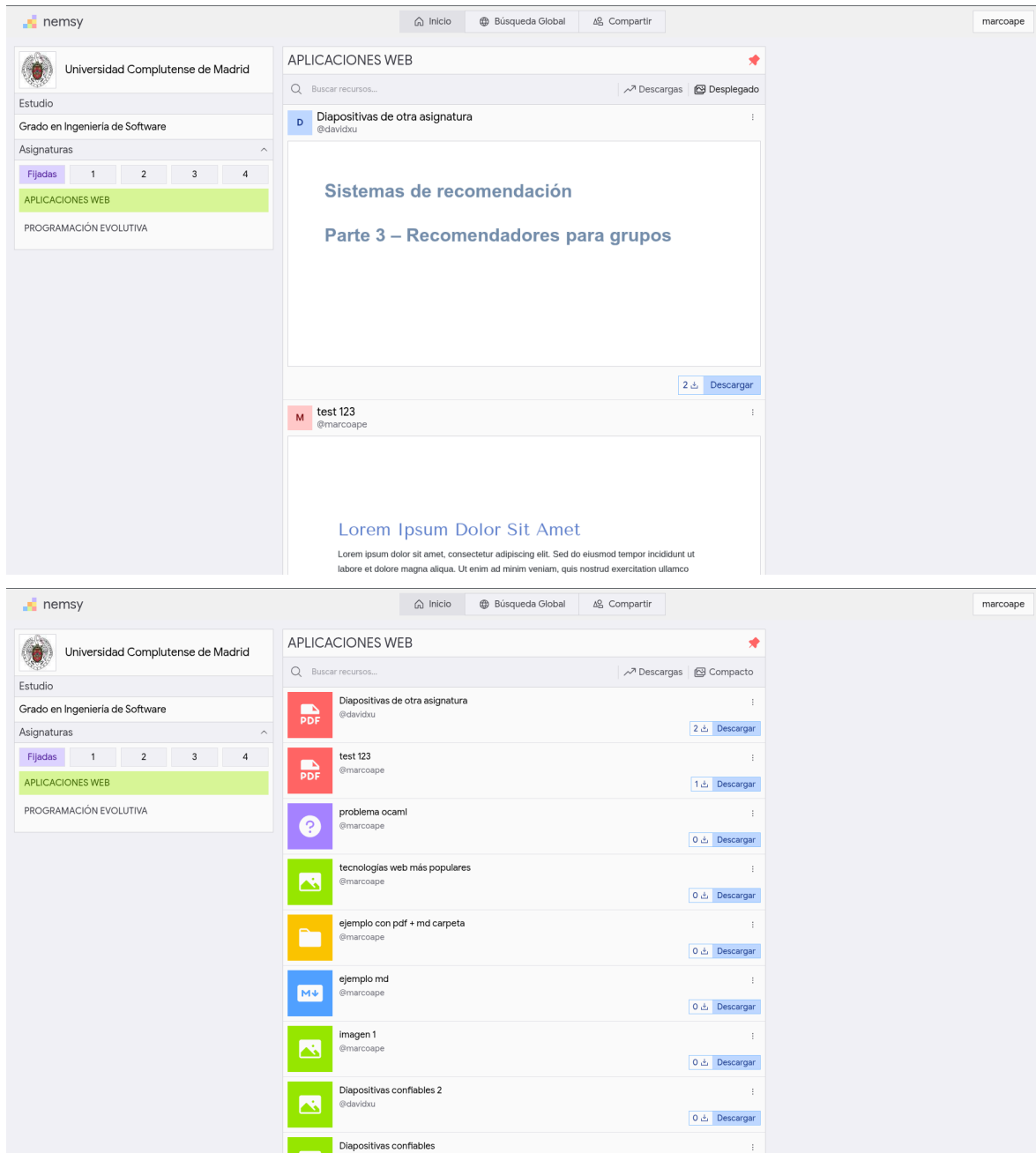


Figura 13: Interfaz actual de Nemsy en modo desplegado y modo compacto.

3.3.2 Sistema de diseño

La interfaz organiza el contenido en islas blancas sobre un fondo gris claro, con esquinas rectas y bordes finos, buscando transmitir una sensación más ordenada y sobria que la primera versión. El logotipo, mostrado en la Figura 14, es el elemento visualmente más llamativo y resume la identidad visual de la plataforma: seis cuadrados de colores dispuestos sobre una retícula de 3×3 que forman una escalera ascendente junto al nombre. Esos seis colores no se quedan en el logotipo, sino que se reutilizan como acentos a lo largo de toda la interfaz, lo que evita que la rigidez de la cuadrícula y los bordes rectos resulten monótonos.

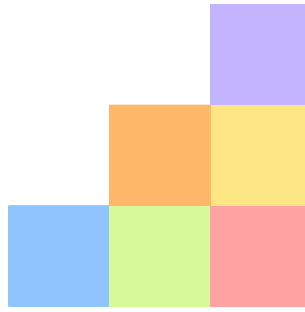


Figura 14: Logotipo de Nemsy.

3.3.3 Tipografía

La tipografía es *Google Sans Flex*, una fuente variable cargada desde Google Fonts que cubre todos los pesos y tamaños ópticos en un único fichero, evitando así la carga de múltiples variantes. Se eligió por su buena legibilidad a tamaños pequeños y porque sus formas ligeramente redondeadas contrastan con la rectitud de los componentes sin que la interfaz resulte fría.

3.3.4 Paleta de color

La paleta se divide en dos grupos. Por un lado, los colores base, una escala neutra de grises (zinc de Tailwind CSS) que estructura el fondo, las islas de contenido, los bordes y la jerarquía tipográfica. Por otro, los seis colores de acento heredados directamente del logotipo, también tomados de la paleta de Tailwind, que se aplican en pequeños detalles como fondos de elementos seleccionados, botones de acción e iconos de tipo de archivo. A diferencia del enfoque habitual de elegir un único color de acento, este reparto multicolor consigue que la interfaz, pese a su rigidez geométrica y su sobriedad, mantenga el suficiente contraste cromático como para no resultar aburrida de mirar ni de usar. La Figura 16 ilustra esta paleta en la página de búsqueda global.



Figura 15: Paleta de color de Nemsy basada en colores por defecto de Tailwind CSS.

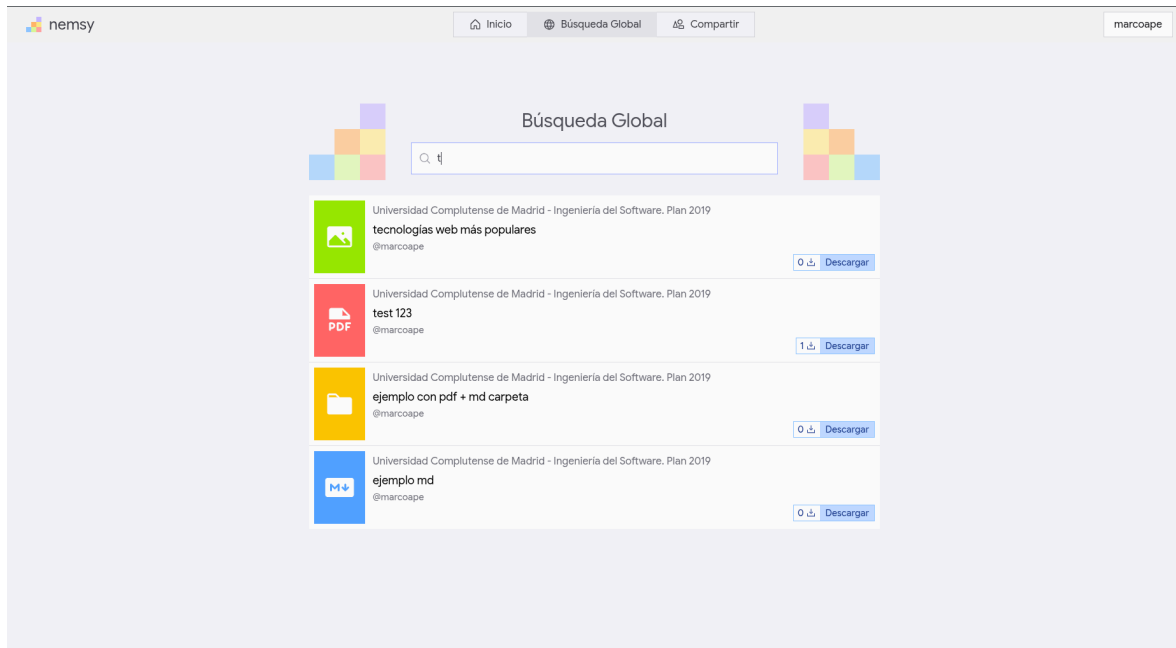


Figura 16: Página de búsqueda global de Nemsy.

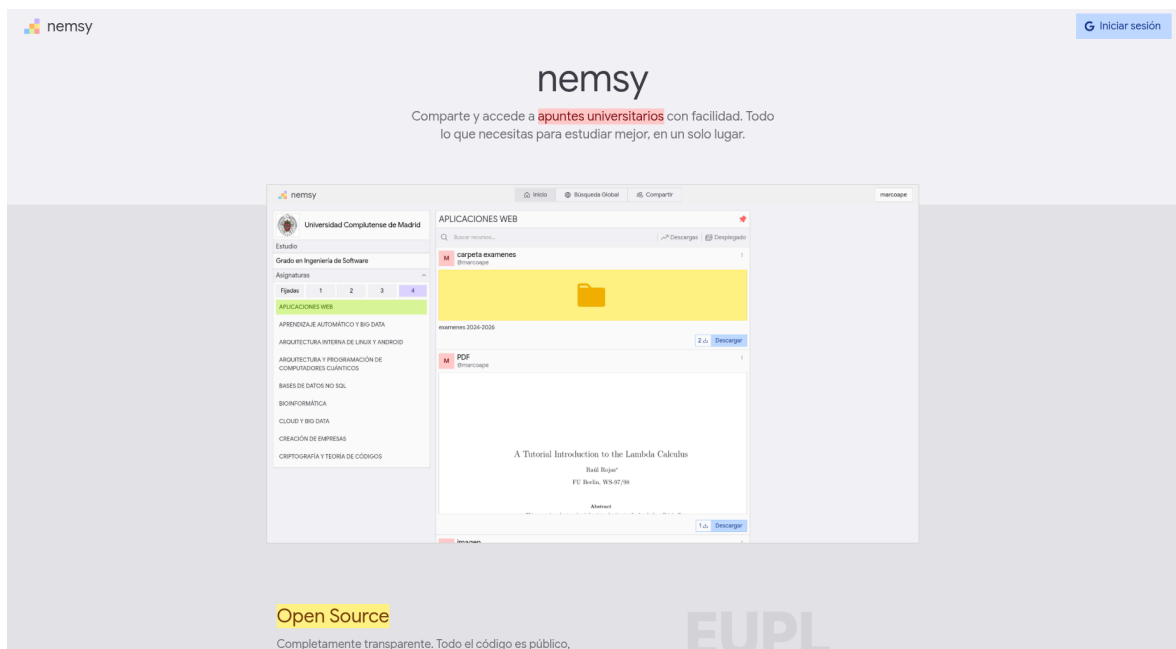


Figura 17: Página de inicio para usuarios no autenticados.

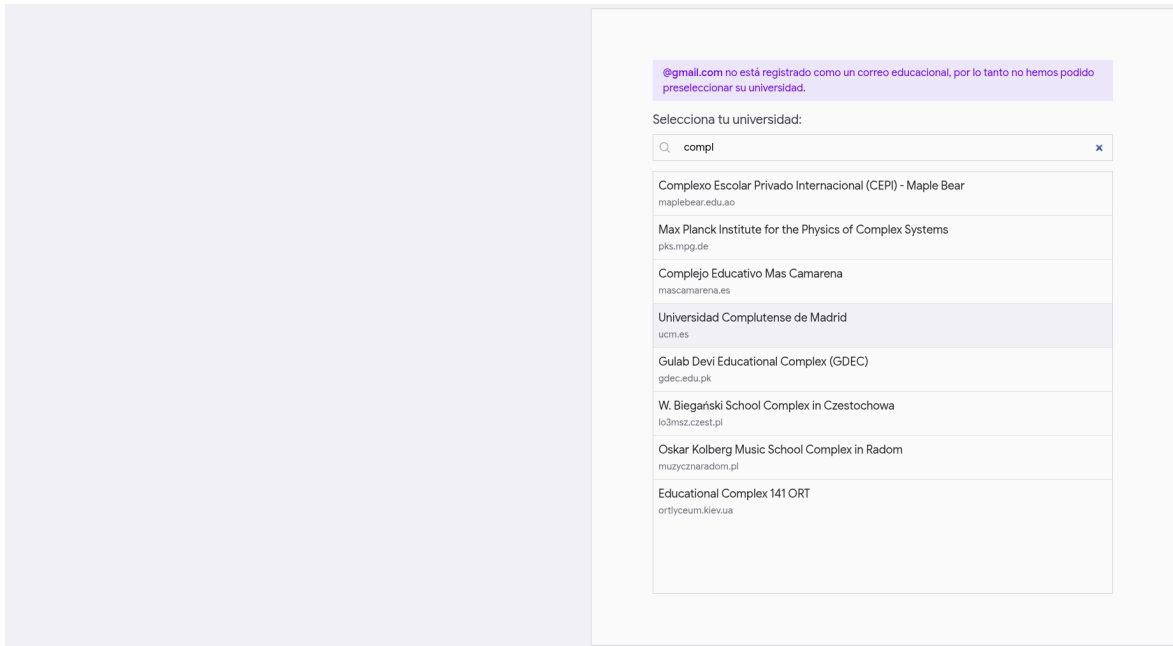


Figura 18: Selección manual de universidad durante el *onboarding*.

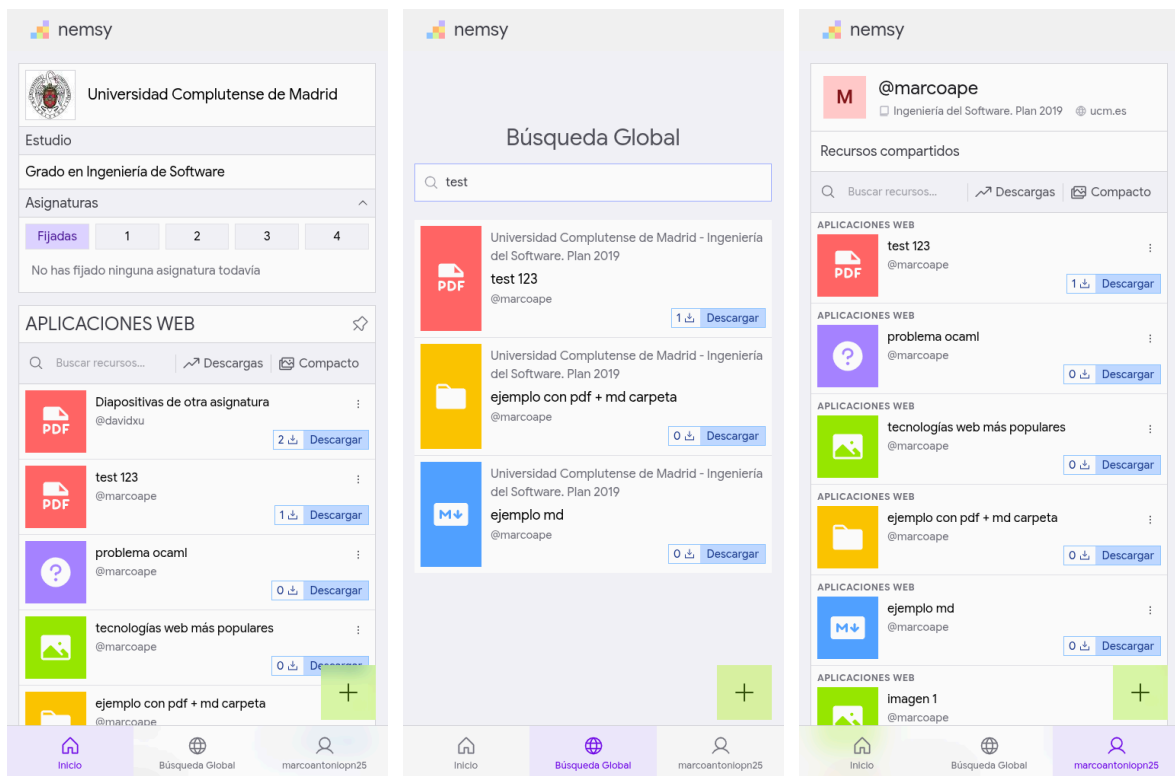


Figura 19: Vistas móviles: listado de recursos, búsqueda global y perfil de usuario.

3.4 Diseño de la base de datos

El esquema de Nemsy se apoya en nueve tablas relacionales que modelan las entidades principales de la plataforma y las relaciones entre ellas. El diseño sigue la tercera forma normal en lo que respecta a la información de dominio, pero introduce de forma

deliberada dos columnas desnormalizadas (`download_count` y `search_vector`) para evitar cálculos repetidos en cada consulta. La Figura 20 recoge el modelo entidad-relación resultante.

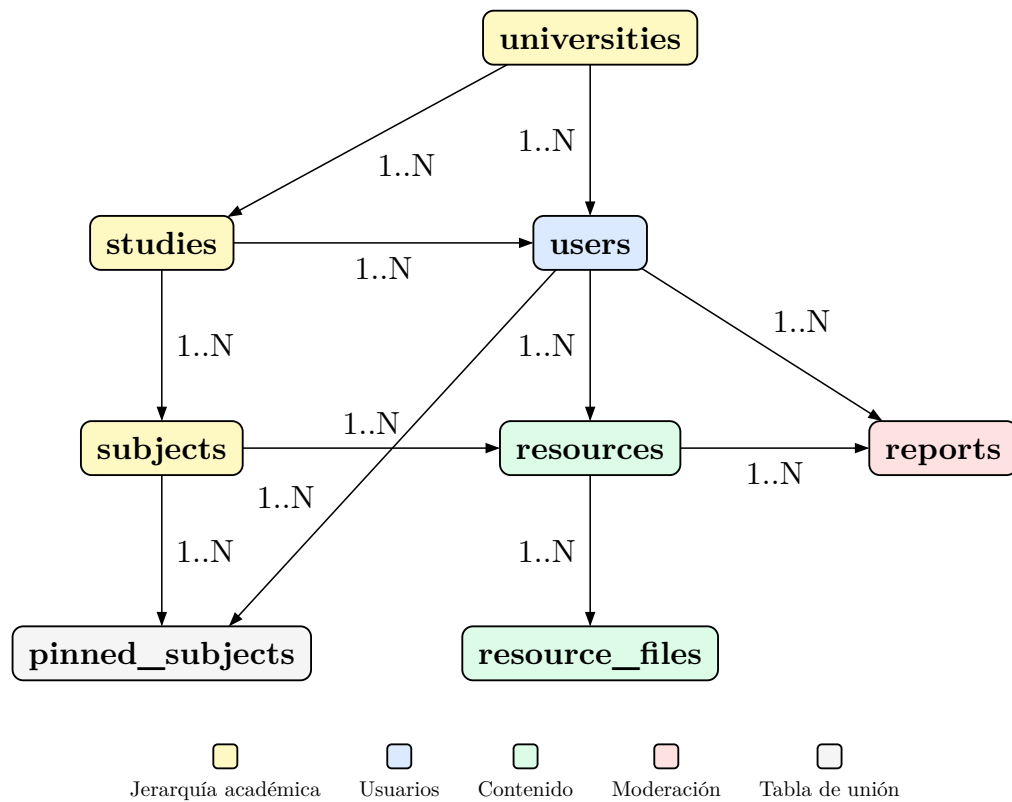


Figura 20: Diagrama entidad-relación del esquema de Nemsy.

3.4.1 Entidades principales

El esquema se articula en torno al eje educativo `universities` → `studies` → `subjects`, que modela la jerarquía académica, y al eje de contenido `users` → `resources` → `resource_files`, que modela la autoría y el almacenamiento. La tabla `pinned_subjects` materializa la relación N:M entre usuarios y asignaturas fijadas, y `reports` recoge las denuncias sobre recursos. La Tabla 6 resume el papel de cada tabla.

Tabla	Responsabilidad
<code>universities</code>	Lista de universidades con su dominio de correo (<code>ucm.es</code> , <code>upm.es</code> , ...), usado para asociar automáticamente al usuario al iniciar sesión con Google.
<code>studies</code>	Grados o titulaciones ofertados por una universidad. Cada estudio pertenece a una universidad mediante <code>university_id</code> .
<code>subjects</code>	Asignaturas que componen un estudio, junto con el curso (<code>year</code>) al que pertenecen.
<code>users</code>	Usuarios autenticados vía Google. Se almacena el <code>google_sub</code> como identificador, el <code>email</code> , el <code>hd</code> (<i>hosted domain</i>) del que se deduce la universidad, un <code>username</code> único generado a partir del correo y un campo <code>role</code> para distinguir administradores de usuarios corrientes.

Tabla	Responsabilidad
resources	Apuntes y demás archivos subidos por los usuarios. Cada recurso agrupa uno o varios archivos bajo un mismo <code>title</code> y <code>description</code> , y mantiene un contador <code>download_count</code> y un <code>search_vector</code> para la búsqueda de texto completo.
resource_files	Archivos individuales que componen un recurso, con su <code>s3_key</code> (clave en Object Storage), nombre original y tamaño.
pinned_subjects	Relación N:M entre <code>users</code> y <code>subjects</code> que representa las asignaturas fijadas por cada usuario en su página de inicio. Su clave primaria compuesta (<code>user_id</code> , <code>subject_id</code>) evita duplicados sin un índice adicional.
reports	Denuncias sobre recursos. La restricción <code>UNIQUE(resource_id, reporter_id)</code> impide que un mismo usuario denuncie dos veces el mismo recurso.

Tabla 6: Tablas del esquema y su responsabilidad.

Todas las claves foráneas se declaran con `ON DELETE CASCADE`, de forma que al eliminar un usuario se borran en cascada sus recursos, archivos, denuncias y asignaturas fijadas, por lo que no quedan filas huérfanas ni es necesario ninguna lógica de limpieza en el backend.

3.4.2 Evolución mediante migraciones

El esquema no se definió en un único fichero monolítico, sino que se construyó de forma incremental a medida que crecieron los requisitos de la plataforma. Para gestionar esta evolución se utilizó `golang-migrate` [12], una herramienta que aplica migraciones SQL numeradas y mantiene una tabla `schema_migrations` interna para saber qué versión está activa. Cada migración consta de dos ficheros, `NNN_nombre.up.sql` para aplicar el cambio y `NNN_nombre.down.sql` para revertirlo, lo que permite volver a una versión anterior sin intervención manual sobre la base de datos.

La Tabla 7 recoge el historial completo de migraciones del proyecto, que refleja la evolución del esquema desde la versión inicial hasta el estado actual con cada nueva funcionalidad que se requería modificar de este.

Migración	Cambio introducido
000001_init	Esquema inicial con <code>studies</code> , <code>users</code> , <code>subjects</code> y <code>resources</code> .
000002_multi_files	Extrae los archivos de <code>resources</code> a la tabla <code>resource_files</code> para permitir varios ficheros por recurso.
000003_pinned_subjects	Tabla <code>pinned_subjects</code> para asignaturas fijadas por cada usuario.
000004_username	Sustituye <code>full_name</code> y <code>pfp</code> por un <code>username</code> único generado del correo.
000005_download_count	Añade <code>download_count</code> a <code>resources</code> para evitar <code>COUNT(*)</code> en cada consulta.
000006_search_vector	Añade la columna generada <code>search_vector</code> e índice GIN para Full Text Search.
000007_universities	Introduce la tabla <code>universities</code> y enlaza <code>studies</code> y <code>users</code> con ella.
000008_unaccent_search	Integra la extensión <code>unaccent</code> en los <code>search_vector</code> para ignorar tildes.
000009_user_role	Añade el campo <code>role</code> a <code>users</code> y la tabla <code>reports</code> .

Tabla 7: Historial de migraciones del esquema.

3.4.3 Búsqueda de texto completo

Una de las decisiones de diseño más relevantes es el uso de las capacidades nativas de Full Text Search de PostgreSQL para implementar la búsqueda global de recursos, sin recurrir a servicios externos como Elasticsearch. La búsqueda se apoya en tres piezas que se combinan en el propio esquema: columnas generadas de tipo `tsvector`, la extensión `unaccent` envuelta en una función `IMMUTABLE`, e índices GIN.

La columna `search_vector` de `resources` es una columna generada y almacenada (`GENERATED ALWAYS ... STORED`) que PostgreSQL recalcula automáticamente en cada `INSERT` o `UPDATE`, eliminando la necesidad de triggers o lógica adicional en el backend. Su contenido combina el título y la descripción del recurso con pesos distintos ('A' y 'B'), de forma que una coincidencia en el título pesa más que una en la descripción al calcular el ranking con `ts_rank`.

Un problema importante que surge al tratar con texto en español son las tildes, que pese a ser frecuentes en el idioma, los usuarios no siempre tienden a escribirlas al buscar pero si esperan que la búsqueda lo encuentre. La extensión `unaccent` de PostgreSQL elimina los diacríticos, pero no puede usarse directamente dentro de una columna generada porque no está marcada como `IMMUTABLE`. La solución, introducida en la migración `000008`, consiste en envolverla en una función SQL propia (`immutable_unaccent`) que sí lo está, tal y como muestra el fragmento de la Código 3.

```

1  CREATE OR REPLACE FUNCTION immutable_unaccent(text)
2  RETURNS text AS $$
3      SELECT public.unaccent('public.unaccent', $1)
4  $$ LANGUAGE sql IMMUTABLE PARALLEL SAFE;
5
6  ALTER TABLE resources ADD COLUMN search_vector tsvector
7      GENERATED ALWAYS AS (
8          setweight(to_tsvector('spanish',
9              immutable_unaccent(coalesce(title, ''))), 'A') ||
10             setweight(to_tsvector('spanish',
11                 immutable_unaccent(coalesce(description, ''))), 'B')
12      ) STORED;
13
14  CREATE INDEX idx_resources_search ON resources USING GIN
    (search_vector);

```

Código 3: Definición del `search_vector` con pesos y `unaccent`.

Sobre esta columna se construye un índice GIN (*Generalized Inverted Index*), la estructura recomendada por PostgreSQL para datos `tsvector` [13]. Un índice GIN indexa cada *lexema* de forma independiente, de manera que una consulta con el

operador `@@` localiza los recursos coincidentes sin necesidad de escanear toda la tabla. La misma técnica se aplica a la tabla `universities`, aunque en su caso se usa la configuración `'simple'` en lugar de `'spanish'`, ya que los nombres propios no deben reducirse a su raíz como sí conviene hacer con el texto en prosa.

3.4.4 Índices adicionales

Además del índice GIN para la búsqueda, el esquema define varios índices B-tree sobre las claves foráneas más consultadas. El índice compuesto `idx_resources_subject_created` (`subject_id`, `created_at` DESC) es especialmente relevante ya que sirve para la consulta que lista los recursos de una asignatura ordenados por fecha, permitiendo que PostgreSQL resuelva la operación directamente desde el índice sin ordenación posterior. Los índices por `owner_id` y por `resource_id` en `resource_files` garantizan que las eliminaciones en cascada y las consultas de perfil se ejecuten en tiempo logarítmico, sin necesidad de recorrer la tabla completa.

3.5 Implementación del backend

Una vez establecidas la arquitectura general en la Figura 10 y las elecciones tecnológicas, esta sección entra en los detalles de la implementación. Se describen primero la organización del proyecto y el proceso de arranque del servidor, y a continuación los aspectos técnicamente más relevantes: la generación de código a partir de SQL, el tratamiento transaccional de las operaciones multi-paso, la gestión de archivos contra el almacenamiento de objetos, los detalles internos de la autenticación y, por último, el sistema de moderación.

3.5.1 Estructura del proyecto

El backend sigue el *layout* convencional en Go, que separa los puntos de entrada ejecutables del código reutilizable mediante dos directorios de primer nivel, `cmd/`, que contiene un subdirectorio por cada binario producido por el módulo, e `internal/`, que aloja los paquetes de la aplicación. El compilador de Go aplica una restricción especial sobre `internal/`, impidiendo que sus paquetes sean importados desde fuera del módulo [14], lo que garantiza que la API interna del backend no se filtre como dependencia accidental. La Figura 21 muestra el árbol resultante.

La Figura 21 reproduce la salida del comando `tree` ejecutado sobre la raíz del backend, con los directorios resaltados como en una terminal real.

```

├── cmd
│   ├── gen-token
│   │   └── main.go
│   ├── seed-studies
│   │   └── scrape_ucm.py
│   ├── seed-universities
│   │   └── main.go
│   └── server
│       └── main.go
├── Dockerfile
├── go.mod
├── go.sum
├── internal
│   ├── admin
│   │   ├── handler.go
│   │   └── handler_test.go
│   ├── app
│   │   └── app.go
│   ├── auth
│   │   ├── admin.go
│   │   ├── handler.go
│   │   ├── middleware.go
│   │   ├── statestore.go
│   │   ├── utils.go
│   │   └── utils_test.go
│   ├── db
│   │   ├── generated
│   │   │   ├── db.go
│   │   │   ├── models.go
│   │   │   ├── querier.go
│   │   │   ├── reports.sql.go
│   │   │   ├── resource_files.sql.go
│   │   │   ├── resources.sql.go
│   │   │   ├── studies.sql.go
│   │   │   ├── subjects.sql.go
│   │   │   ├── universities.sql.go
│   │   │   └── users.sql.go
│   │   ├── migrations
│   │   │   ├── 000001_init.down.sql
│   │   │   ├── 000001_init.up.sql
│   │   │   ├── 000002_multi_files.down.sql
│   │   │   ├── 000002_multi_files.up.sql
│   │   │   ├── 000003_pinned_subjects.down.sql
│   │   │   ├── 000003_pinned_subjects.up.sql
│   │   │   ├── 000004_username.down.sql
│   │   │   ├── 000004_username.up.sql
│   │   │   ├── 000005_download_count.down.sql
│   │   │   ├── 000005_download_count.up.sql
│   │   │   ├── 000006_search_vector.down.sql
│   │   │   ├── 000006_search_vector.up.sql
│   │   │   ├── 000007_universities.down.sql
│   │   │   ├── 000007_universities.up.sql
│   │   │   ├── 000008_unaccent_search.down.sql
│   │   │   ├── 000008_unaccent_search.up.sql
│   │   │   ├── 000009_user_role.down.sql
│   │   │   └── 000009_user_role.up.sql
│   │   ├── queries
│   │   │   ├── reports.sql
│   │   │   ├── resource_files.sql
│   │   │   ├── resources.sql
│   │   │   ├── studies.sql
│   │   │   ├── subjects.sql
│   │   │   ├── universities.sql
│   │   │   └── users.sql
│   │   └── sqlc.yaml
│   ├── resources
│   │   ├── handler.go
│   │   └── handler_test.go
│   ├── search
│   │   ├── query.go
│   │   └── query_test.go
│   ├── storage
│   │   └── s3.go
│   ├── studies
│   │   ├── handler.go
│   │   └── handler_test.go
│   ├── universities
│   │   ├── handler.go
│   │   └── handler_test.go
│   ├── users
│   │   ├── handler.go
│   │   └── handler_test.go
└── tools.go

```

Figura 21: Salida del comando `tree` sobre la raíz del backend.

El directorio `cmd/` aloja cuatro binarios independientes. El principal, `cmd/server`, es el servidor HTTP descrito en el apartado Sección 3.5.2. Los dos siguientes, `cmd/seed-universities` y `cmd/seed-studies`, son utilidades de línea de comandos que cargan respectivamente el listado de universidades españolas y los grados ofertados por cada una; se ejecutan una sola vez durante el despliegue inicial. El último, `cmd/gen-token`,

genera tokens JWT firmados con el mismo secreto que el servidor para usarlos en las pruebas de carga con k6 descritas en Sección 3.7.4.1, evitando así depender del flujo OAuth2 durante esas pruebas.

Dentro de `internal/`, los paquetes se organizan por dominio funcional. El paquete `app` define el `struct App` y la interfaz `QuerierWithTx` que comparten todos los `handlers`, ya introducidos en Código 1. El paquete `auth` concentra el flujo OAuth2 con Google, la generación y validación de tokens JWT y los dos `middlewares` de la cadena de autenticación, `AuthMiddleware` y `AdminOnly`. Los paquetes `resources`, `users`, `studies`, `universities` y `admin` agrupan los `handlers` HTTP correspondientes a cada dominio, siendo `resources` el más extenso por concentrar la lógica de creación de recursos con archivos, descargas, búsqueda y reportes.

Los tres paquetes restantes son de infraestructura. El paquete `storage` encapsula el cliente `minio-go` tras una pequeña fachada, lo que permite reemplazar el proveedor S3 sin tocar el resto del código. El paquete `search` aloja la función `PrefixQuery`, que transforma la entrada de usuario en una expresión `tsquery` compatible con el operador `@@` de PostgreSQL. Finalmente, el paquete `db` se subdivide en tres directorios complementarios: `queries/` con las consultas SQL escritas a mano, `migrations/` con las migraciones numeradas que aplica `golang-migrate` (véase Tabla 7), y `generated/` con el código Go que `sqlc` emite a partir de los dos anteriores.

3.5.2 Arranque del servidor

El binario `cmd/server` concentra la totalidad de la inicialización del backend en una única función `main`, siguiendo el principio de *composition root* [15]: todas las dependencias se construyen en el mismo punto y se inyectan explícitamente en los componentes que las necesitan, sin contenedores de inyección de dependencias ni configuración global oculta. La Figura 22 ilustra las cinco fases del arranque.

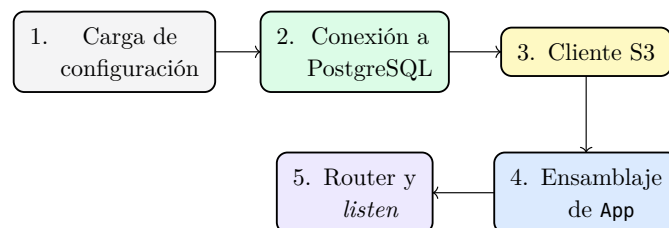


Figura 22: Fases del arranque del servidor en `cmd/server/main.go`.

La configuración se inyecta exclusivamente a través de variables de entorno, siguiendo la metodología *twelve-factor* [16] que recomienda mantener una separación estricta entre código y configuración. La Tabla 8 recoge las variables consumidas por el servidor; cualquier variable obligatoria ausente provoca el aborto inmediato del proceso mediante `log.Fatal`, evitando que el servidor arranque en un estado inconsistente.

Variable	Obligatoria	Propósito
DATABASE_URL	Sí	Cadena de conexión a PostgreSQL, consumida por <code>pgxpool</code> .
JWT_SECRET	Sí	Clave HMAC para firmar y verificar tokens JWT.
S3_ENDPOINT	Sí	Endpoint del proveedor de almacenamiento de objetos compatible con S3.
S3_ACCESS_KEY, S3_SECRET_KEY	Sí	Credenciales del bucket de almacenamiento.
S3_BUCKET	Sí	Nombre del bucket donde se almacenan los archivos de los recursos.
S3_USE_SSL	No	Habilita TLS contra el endpoint S3. Por defecto activado.
ALLOWED_ORIGIN	No	Origen permitido para CORS y <i>redirect</i> tras el <i>login</i> . Por defecto <code>http://localhost:5173</code> en desarrollo.
GOOGLE_CLIENT_ID, GOOGLE_CLIENT_SECRET	Sí	Credenciales del cliente OAuth2 registrado en Google Cloud.

Tabla 8: Variables de entorno consumidas por el servidor.

Tras validar la configuración, se inicializa el *pool* de conexiones a PostgreSQL mediante `pgxpool.New`, que abre y reutiliza un conjunto de conexiones físicas a la base de datos. Sobre este *pool* se construye el *querier* generado por `sqlc` (`db.New(pool)`), que es la implementación concreta de la interfaz `QuerierWithTx` introducida en Código 1. A continuación se instancia el cliente S3 y, con las tres dependencias disponibles, se ensambla el *struct App* que se inyectará en cada *handler*.

La etapa final construye el *router* Chi y registra las rutas en tres grupos diferenciados según el *middleware* que les aplica, como muestra la muestra de Código 4. Esta separación se apoya en la capacidad de Chi de anidar grupos de rutas con *middlewares* independientes, permitiendo que las rutas públicas de autenticación, las rutas protegidas por JWT y las rutas exclusivas de administración convivan en un mismo árbol sin condicionales repetidos en cada *handler*.


```

1 // rutas públicas (sin autenticación)
2 r.Get("/auth/login", authHandler.LoginHandler)
3 r.Get("/auth/callback", authHandler.CallbackHandler)
4 r.Post("/auth/logout", authHandler.LogoutHandler)
5
6 // rutas protegidas (JWT validado en cada petición)
7 r.Group(func(r chi.Router) {
8     r.Use(mw.Middleware)
9     r.Get("/api/me", usersHandler.MeHandler)
10    // ...
11
12    // solo administradores
13    r.Group(func(r chi.Router) {
14        r.Use(auth.AdminOnly)
15        r.Get("/api/admin/reports", adminHandler.ListReports)
16    })
17 })

```

Código 4: Estructura de grupos de rutas en Chi con *middlewares* anidados.

El servidor se lanza en una *goroutine* independiente para que la *goroutine* principal pueda quedar bloqueada a la espera de una señal del sistema operativo (SIGINT o SIGTERM). Al recibirla, el proceso cierra el servidor de forma ordenada antes de devolver el control, evitando interrumpir peticiones en curso y permitiendo que `defer pool.Close()` libere las conexiones a PostgreSQL.

3.5.3 Generación de código con sqlc

La capa de persistencia es la que más decisiones de diseño concentra del backend. La elección de sqlc [17] frente a un ORM tradicional responde a una preferencia por la transparencia, y a la filosofía de tratar el SQL como fuente de verdad y al código Go como un derivado de este. Este enfoque es bastante más habitual en proyectos Go que en el ecosistema Java, donde el ORM (típicamente JPA) es prácticamente la opción por defecto. La idea es que el desarrollador escriba las consultas como las escribiría en un cliente de PostgreSQL, anotándolas con una directiva especial que indica el nombre y la cardinalidad del resultado y la herramienta las lee y genera funciones Go fuertemente tipadas que las ejecutan, con *structs* para los parámetros y para las filas devueltas. A diferencia de un *query builder*, que va componiendo el SQL en tiempo de ejecución a partir de llamadas encadenadas, en sqlc el SQL ya está escrito tal cual se envía a PostgreSQL, lo único que se genera automáticamente es el código de pegamento que lo conecta con Go.

Entrada (queries/resources.sql):

```

1  -- name:
   CreateResource :one
2  INSERT INTO resources (
3      owner_id, subject_id,
4      title, description
5  ) VALUES (
6      $1, $2, $3, $4
7  )
8  RETURNING id, owner_id,
   subject_id,
9      title, description,
   created_at,
10     download_count;

```

Salida (generated/resources.sql.go):

```

1  type CreateResourceParams
   struct {
2      OwnerID      int32
3      SubjectID    int32
4      Title        string
5      Description  pgtype.Text
6  }
7
8  func (q *Queries)
   CreateResource(
9      ctx context.Context,
10     arg CreateResourceParams,
11 ) (CreateResourceRow, error)
   { /* ... */ }

```

Código 5: Consulta SQL anotada con `-- name: ... :one` y código Go generado por `sqlc` a partir de ella.

Este enfoque tiene tres ventajas frente a un ORM. La primera, ya mencionada en Tabla 3, es que permite aprovechar funcionalidades específicas de PostgreSQL como `tsvector`, columnas generadas o expresiones de tipos compuestos sin tener que envolverlas en abstracciones genéricas. La segunda es que el SQL queda visible en el repositorio, en lugar de oculto detrás de capas de *builders* que el desarrollador ha de mentalmente traducir a la consulta real que se ejecuta. La tercera es que cualquier error de sintaxis SQL o de tipos se detecta en el momento de generar el código, no en tiempo de ejecución, desplazando una clase entera de errores del entorno de producción al de desarrollo.

La opción `emit_interface: true` del fichero `sqlc.yaml` instruye a la herramienta para que, además de las funciones, emita la interfaz `Querier` que las agrupa, recogida parcialmente en la Código 6. Esta interfaz es el contrato que cumple la implementación generada (`*Queries`) y, fundamentalmente, el que cualquier *mock* de pruebas debe cumplir también. Esta decisión es la que habilita la estrategia de *testing* del paquete `auth` y de los *handlers* HTTP, descrita en Sección 3.7.1.

```

1 type Querier interface {
2     CreateResource(ctx context.Context, arg CreateResourceParams)
      (CreateResourceRow, error)
3     CreateResourceFile(ctx context.Context, arg
      CreateResourceFileParams) (ResourceFile, error)
4     DeleteResource(ctx context.Context, arg DeleteResourceParams) error
5     GetResourceWithOwner(ctx context.Context, id int32)
      (GetResourceWithOwnerRow, error)
6     ListResourcesBySubjectWithOwnerPaginated(ctx context.Context,
7         arg ListResourcesBySubjectWithOwnerPaginatedParams)
      ([]ListResourcesBySubjectWithOwnerPaginatedRow, error)
8     // ...
9 }

```

Código 6: Fragmento de la interfaz `Querier` generada por `sqlc`.

Sobre la interfaz `Querier` generada por `sqlc`, el paquete `app` define una extensión propia llamada `QuerierWithTx` que añade un único método, `WithTx(tx pgx.Tx) *Queries`, ya introducido en Código 1. Este método permite obtener una nueva instancia del *querier* asociada a una transacción concreta, manteniendo la misma API que la versión sin transacción. La utilidad de esta extensión se materializa en el siguiente apartado.

3.5.4 Creación de un recurso: transacciones y compensación

La creación de un recurso es la operación más compleja de la API porque atraviesa los dos sistemas de persistencia simultáneamente: PostgreSQL (que almacena los metadatos del recurso y el listado de archivos asociados) y el almacenamiento de objetos S3 (donde residen los bytes de cada archivo). Cualquier fallo a mitad del proceso ha de dejar la plataforma en un estado consistente, sin filas de `resource_files` apuntando a objetos inexistentes ni objetos huérfanos en el *bucket* que ningún registro referencie.

El problema fundamental es que PostgreSQL y S3 son dos sistemas independientes, sin un coordinador transaccional común que los abarque a ambos. La estrategia adoptada se inspira en el patrón de *saga* con compensación [18]: la operación se divide en pasos, y cualquier fallo en un paso intermedio dispara acciones compensatorias que deshacen los pasos previos. La Figura 23 ilustra el flujo completo.

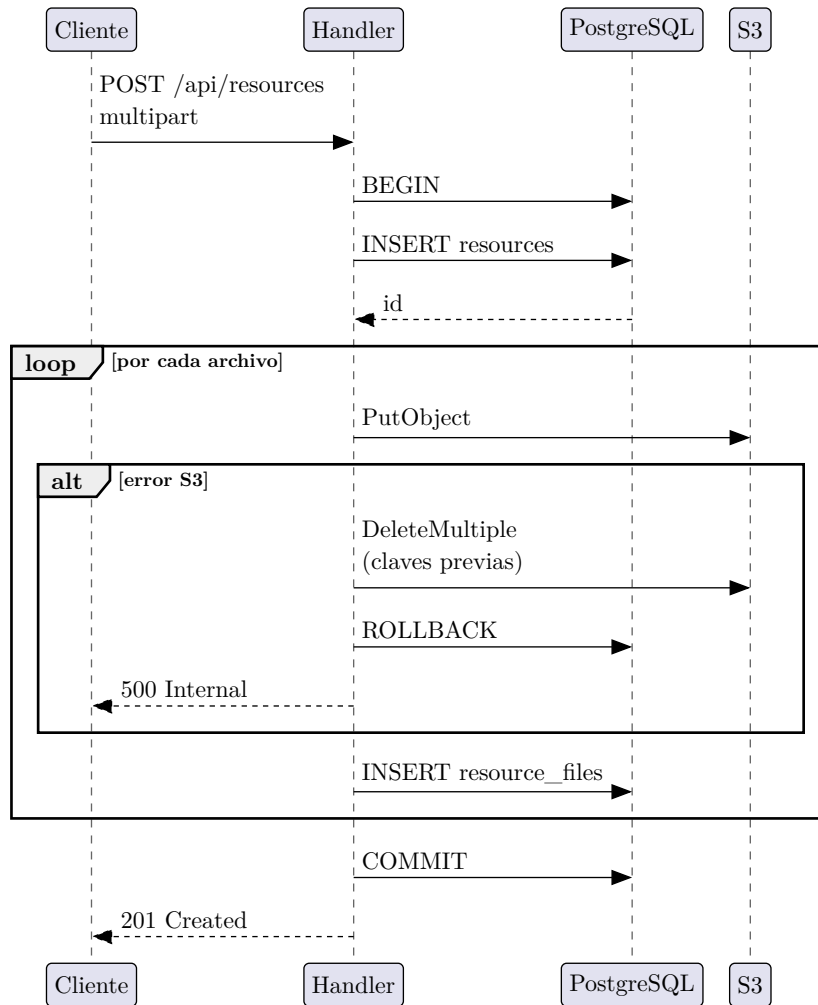


Figura 23: Creación de un recurso con varios archivos, mostrando el camino de éxito y la rama de compensación ante un fallo en S3.

El *handler* comienza abriendo una transacción con `h.app.DB.Begin` y registrando inmediatamente un `defer tx.Rollback`. Esta línea es fundamental: garantiza que, si la función retorna por cualquier camino antes del `Commit`, la transacción se descarta automáticamente. PostgreSQL ignora el `Rollback` posterior si el `Commit` ya se ha ejecutado con éxito, de forma que el `defer` actúa como red de seguridad sin penalizar el camino feliz.

```

1  tx, err := h.app.DB.Begin(r.Context())
2  if err != nil { /* 500 */ }
3  defer tx.Rollback(r.Context()) // red de seguridad
4  qtx := h.app.Queries.WithTx(tx)
5
6  resource, err := qtx.CreateResource(r.Context(),
7    db.CreateResourceParams{
8    // ...
9  })
10 if err != nil { /* 500 */ }
11
12 var uploadedKeys []string // journal para la compensación
13 for _, fh := range files {
14   s3Key := fmt.Sprintf("resources/%d/%s", resource.ID,
15     sanitizeFilename(fh.Filename))
16   if err := h.app.Storage.Upload(r.Context(), s3Key, f, fh.Size,
17     contentType); err != nil {
18     h.cleanupS3(r, uploadedKeys) // compensación
19     http.Error(w, "...", http.StatusInternalServerError);
20     return
21   }
22   uploadedKeys = append(uploadedKeys, s3Key)
23
24   if _, err := qtx.CreateResourceFile(r.Context(),
25     db.CreateResourceFileParams{
26     // ...
27   }); err != nil {
28     h.cleanupS3(r, uploadedKeys) // compensación
29     http.Error(w, "...", http.StatusInternalServerError);
30     return
31   }
32 }
33
34 if err := tx.Commit(r.Context()); err != nil {
35   h.cleanupS3(r, uploadedKeys) // compensación
36   http.Error(w, "...", http.StatusInternalServerError);
37   return
38 }

```

Código 7: Creación de un recurso con compensación en S3 (internal/resources/handler.go).

Sobre esa transacción se construye un *querier* alternativo mediante `WithTx(tx)`. Todas las consultas ejecutadas a través de `qtx` participan automáticamente en la

transacción, mientras que las llamadas que pudieran hacerse contra el *querier* original seguirían viendo el estado anterior al **Begin**. Este patrón permite escribir el resto del *handler* sin diferenciar entre operaciones transaccionales y no transaccionales: la API de ambos *queriers* es exactamente la misma gracias a la interfaz generada por `sqlc`.

El bucle de archivos alterna inserciones en S3 y en la base de datos. Cada subida a S3 que tiene éxito se registra en la lista local `uploadedKeys`, que actúa como *journal* de las acciones que requerirían compensación si algo fallase a partir de ese punto. La función auxiliar `cleanupS3` invoca `Storage.DeleteMultiple` con esa lista, aprovechando el endpoint *bulk* de la API S3 para borrar todos los objetos huérfanos en una única petición. Es importante destacar que el **Rollback** automático de PostgreSQL es suficiente para limpiar las filas de `resource_files` insertadas con `qtx`, pero no afecta a S3, motivo por el cual la compensación explícita es necesaria.

Existe una ventana muy pequeña en la que el sistema podría quedar inconsistente: si el **Commit** final tiene éxito en PostgreSQL pero la respuesta no llega al cliente, el cliente reintentará la operación y se crearán archivos duplicados con el mismo contenido pero distinta `s3_key`. Esta posibilidad se acepta como *trade-off* frente a la complejidad de implementar idempotencia mediante claves de cliente, decisión razonable dado que el coste de un duplicado ocasional es bajo y el usuario puede borrarlo manualmente.

3.5.5 Descarga de archivos con redirección y empaquetado en *streaming*

La descarga es la operación dual a la creación y plantea un problema distinto, ya que los recursos pueden contener un único fichero o varios y los ficheros pueden llegar a pesar 100 MB, por lo que volcar todos los bytes en memoria del backend antes de enviarlos al cliente sería tanto un desperdicio de RAM como un cuello de botella de latencia, sobre todo en un VPS con 4 GB compartidos entre todos los servicios. Por ello, los dos endpoints de descarga (`/api/resources/{id}/download` para el paquete completo y `/api/resources/{id}/files/{fileId}/download` para un único archivo) se implementan con dos estrategias complementarias que evitan en todo momento mantener un fichero entero en memoria.

El primer endpoint adapta su comportamiento al número de ficheros del recurso. Si solo hay uno, no tiene sentido envolverlo en un ZIP, así que el *handler* genera una URL prefirmada de S3 con 15 minutos de validez y devuelve un **307 Temporary Redirect** apuntando a esa URL. El navegador del usuario sigue la redirección y descarga el fichero directamente desde el *bucket*, sin que el tráfico atraviese el backend. La cabecera `response-content-disposition` incluida en la URL prefirmada (gestionada en `storage/s3.go`) garantiza que el navegador respete el nombre original del fichero al guardarlo, en lugar de la `s3_key` interna.

```
1  if len(files) == 1 {
2      presigned, err := h.app.Storage.GetPresignedURL(
3          r.Context(), files[0].S3Key, 15*time.Minute,
4          files[0].FileName,
5      )
6      if err != nil {
7          http.Error(w, "download error",
8              http.StatusInternalServerError)
9          return
10     }
11     http.Redirect(w, r, presigned, http.StatusTemporaryRedirect)
12     return
13 }
```

Código 8: Descarga de un único fichero mediante URL prefirmada de S3.

Cuando el recurso contiene varios ficheros, no es posible delegar el empaquetado en S3, así que el backend construye un ZIP al vuelo. El *handler* instancia un `zip.Writer` apuntando directamente al `http.ResponseWriter` y, para cada fichero, abre un *stream* de lectura contra S3 con `Storage.GetObject` y copia los bytes en bloques de 32 KB hacia la entrada correspondiente del ZIP. De este modo, los datos fluyen de S3 a backend y luego al navegador, sin que el proceso retenga más de un buffer de 32 KB en ningún momento. El método de compresión utilizado es `zip.Store` (sin compresión), porque la mayoría de los ficheros que sube el usuario, principalmente PDF y archivos comprimidos, ya están comprimidos y un segundo pase solo añadiría coste de CPU sin reducir tamaño.

El endpoint de descarga individual (`DownloadFile`) sigue una lógica similar pero más sencilla, ya que tras localizar el fichero por su identificador abre el flujo de S3 y lo copia íntegro al `ResponseWriter` con `io.Copy`, dejando que la implementación de la librería estándar maneje los *buffers* internos. Es la operación que sirve, por ejemplo, las previsualizaciones de PDF dentro de la propia interfaz, lo que requiere que la cabecera `Content-Disposition` se configure como `inline` en lugar de `attachment` para que el navegador renderice el fichero en lugar de proponer guardarlo.

Una particularidad de este diseño es que el contador `download_count` solo se incrementa en el endpoint del paquete completo, antes de iniciar la transferencia, y no en `DownloadFile`. Esta decisión refleja la intención del modelo, ya que una descarga representa la intención del usuario de obtener todo el material asociado a un recurso, mientras que la apertura individual de un fichero suele formar parte del flujo de previsualización dentro de la propia interfaz, por lo que contarla como descarga inflaría artificialmente las estadísticas mostradas en el perfil del autor.

```
1  zw := zip.NewWriter(w)
2  defer zw.Close()
3
4  for _, file := range files {
5      obj, _, err := h.app.Storage.GetObject(r.Context(), file.S3Key)
6      if err != nil {
7          return // abandona el ZIP en mitad del envío
8      }
9      writer, err := zw.CreateHeader(&zip.FileHeader{
10         Name: file.FileName, Method: zip.Store,
11     })
12     if err != nil {
13         obj.Close()
14         return
15     }
16
17     buf := make([]byte, 32*1024)
18     for {
19         n, readErr := obj.Read(buf)
20         if n > 0 { writer.Write(buf[:n]) }
21         if readErr != nil { break }
22     }
23     obj.Close()
24 }
```

Código 9: Empaquetado en ZIP construido al vuelo sobre el `http.ResponseWriter`.

3.5.6 Detalles internos de la autenticación

La Figura 24 representa el flujo completo de autenticación, desde que el usuario pulsa el botón de *login* hasta que recibe la *cookie* de sesión, mostrando las cuatro partes que intervienen y la secuencia de mensajes intercambiados.

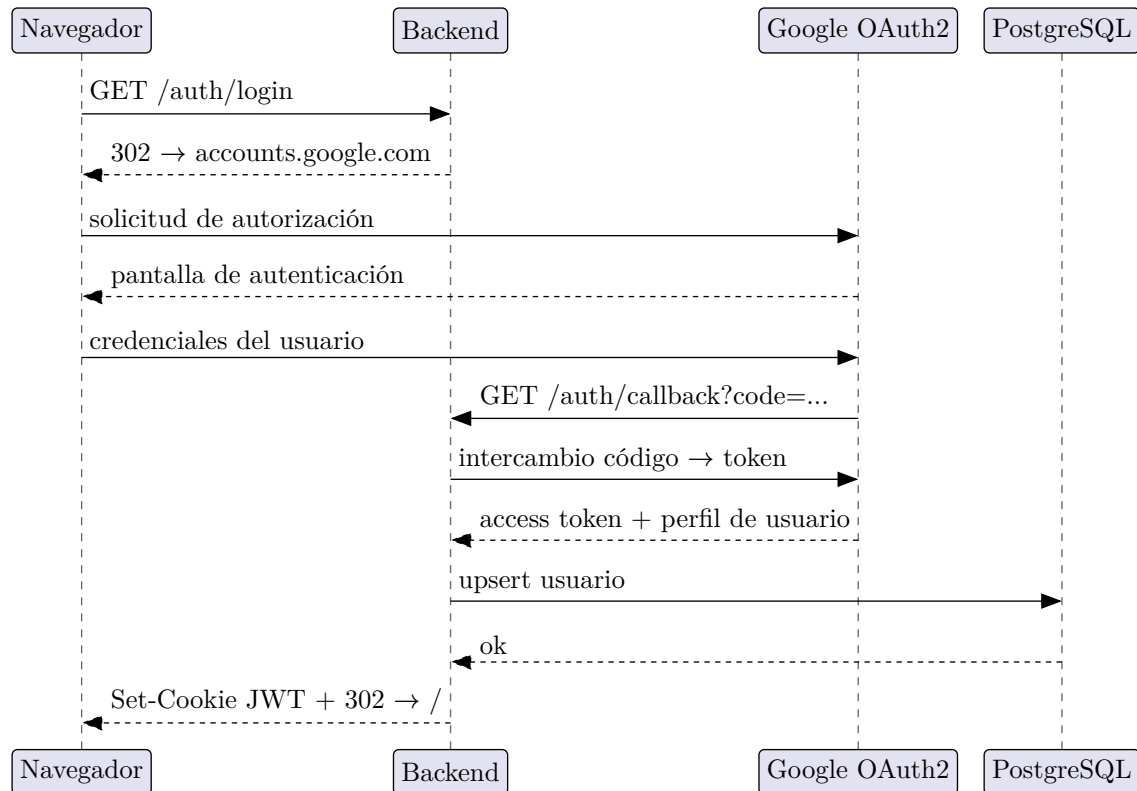


Figura 24: Diagrama de secuencia del flujo de autenticación con Google OAuth2.

Sobre ese flujo, el backend implementa cuatro piezas internas que conviene describir con detalle, ya que cada una resuelve un problema de seguridad concreto. En orden de aparición durante una sesión de usuario son el `StateStore` que protege el flujo OAuth2 frente a ataques CSRF, la generación del token JWT firmado, el `AuthMiddleware` que lo valida en cada petición y el `middleware` adicional `AdminOnly` que restringe las rutas administrativas.

La primera pieza que entra en juego es el `StateStore`. Cuando el usuario pulsa «iniciar sesión», el `LoginHandler` genera una cadena aleatoria de 16 bytes codificada en base64 y la registra en el almacén con un TTL de cinco minutos antes de redirigir al usuario a la pantalla de Google. Esta cadena viaja como parámetro `state` en la URL de autorización y Google la reenvía intacta al `CallbackHandler` cuando el usuario completa el `login`. El `handler` comprueba que el `state` recibido estaba en el almacén y lo elimina inmediatamente, por lo que un atacante no podría inducir a un usuario autenticado a ejecutar una autorización maliciosa simplemente enviándole una URL de `callback` falsa, ya que sin un `state` válido emitido por nosotros la petición se rechaza con `401 Unauthorized` [19]. La implementación del almacén es deliberadamente sencilla, un `map[string]time.Time` protegido por un `sync.Mutex`, suficiente para una sola instancia del backend y sin las complicaciones de un servicio externo como Redis.

```

1  type StateStore struct {
2      m    map[string]time.Time
3      mu   sync.Mutex
4      ttl  time.Duration
5  }
6
7  func (s *StateStore) Put(state string) {
8      s.mu.Lock()
9      defer s.mu.Unlock()
10     s.m[state] = time.Now().Add(s.ttl)
11 }
12
13 func (s *StateStore) Check(state string) bool {
14     s.mu.Lock()
15     defer s.mu.Unlock()
16     exp, ok := s.m[state]
17     if !ok {
18         return false
19     }
20     delete(s.m, state)
21     return time.Now().Before(exp)
22 }

```

Código 10: Almacén de *state* OAuth2 con TTL en `internal/auth/statestore.go`.

Una vez verificado el `state`, el `CallbackHandler` intercambia el código de autorización por un *ID Token* con Google, lo valida criptográficamente con la librería `idtoken` que descarga las claves públicas del servidor de Google, comprueba la firma RS256 y extrae la información del usuario del *payload*. Si es la primera vez que ese correo aparece en la base de datos se crea un nuevo registro intentando un `username` derivado del correo, con sufijos numéricos crecientes en caso de colisión hasta encontrar uno libre. La universidad del usuario se asocia automáticamente buscando el dominio del correo en la tabla `universities`, lo que en la práctica reconoce a un estudiante de `@ucm.es` como miembro de la Complutense sin que tenga que seleccionarla manualmente.

A partir de ese punto la autenticación deja de depender de Google y se pasa a una sesión propia mediante un JSON Web Token [20] firmado con HMAC-SHA256, almacenado en una *cookie* del navegador. Al usar JWT frente a un esquema de sesiones en servidor se mantiene un backend sin estado, ya que cualquier nodo puede validar el token sin necesidad de consultar una base de datos compartida, lo que simplifica la posibilidad de escalar horizontalmente en el futuro. La Tabla 9 recoge los *claims* que se incrustan en el token.

Claim	Tipo	Origen	Propósito
sub	string	ID Token	Identificador estable del usuario en Google, no cambia aunque cambie el correo.
email	string	ID Token	Correo electrónico del usuario.
hd	string	ID Token	<i>Hosted domain</i> del correo (ucm.es, upm.es, ...), usado para asociar la universidad.
user_id	int	Tabla users	Identificador interno del usuario en Nemsy, usado por los <i>handlers</i> para sus consultas.
role	string	Tabla users	Vale <code>user</code> o <code>admin</code> , controla el acceso a las rutas administrativas.
exp	int	Backend (now + 7d)	Marca temporal de expiración del token, comprobada por la librería JWT en cada validación.

Tabla 9: *Claims* incluidos en el JWT de sesión de Nemsy.

La *cookie* que lleva el token se emite con tres atributos defensivos. `HttpOnly` impide que JavaScript lea su contenido, lo que neutraliza el robo de sesiones mediante XSS aunque el atacante consiguiera inyectar código en el frontend. `Secure` obliga a que el navegador la envíe únicamente sobre HTTPS. `SameSite=Lax` la excluye de las peticiones *cross-site* no navegacionales, mitigando ataques CSRF en los *endpoints* autenticados [21]. El alcance se restringe al dominio `.nemsy.org` y la duración a siete días, tras los cuales el `exp` del propio JWT provoca el rechazo del token aunque la *cookie* siguiera presente.

En cada petición a una ruta protegida el `AuthMiddleware` extrae la *cookie*, parsea el token, comprueba que el método de firma es HMAC y, si la validación es satisfactoria, inyecta tres valores en el `context.Context` de la petición, el `UserInfo`, el `user_id` numérico y el `role`. La comprobación explícita del método de firma defiende contra el clásico ataque de confusión de algoritmos [22] en el que un atacante reenvía un token con cabecera `alg: none` o `alg: RS256` con el objetivo de que la librería lo valide con la clave incorrecta. Los tres valores se propagan a los *handlers* subsiguientes a través de la cadena de *context* de Go, que es la forma canónica de transmitir información asociada a una petición sin recurrir a variables globales o parámetros adicionales en cada función.

Por último, el *middleware* `AdminOnly` se anida dentro del grupo protegido por `AuthMiddleware` y constituye una segunda capa que comprueba que el `role` inyectado en el *context* es exactamente `admin`. Cualquier otro valor, incluido el caso improbable de que el *claim* esté ausente, se traduce en un `403 Forbidden`. Los dos *middlewares* actúan en cascada porque responden a preguntas distintas, una de autenticación (quién es el usuario) y otra de autorización (qué puede hacer en cada operación), por lo que tiene sentido mantenerlas como piezas separadas en lugar de mezclarlas en una sola comprobación.

3.5.7 Sistema de reportes y moderación

Cualquier plataforma que aloja contenido subido por usuarios necesita un mecanismo de moderación, tanto por motivos legales como por la propia experiencia de uso, ya que el material académico puede contener errores graves, infringir derechos de autor o incluso, en el peor escenario, aparecer subido como gancho para distribuir un binario malicioso disfrazado de PDF. Nemsy implementa un sistema deliberadamente sencillo en el que cualquier usuario autenticado puede reportar un recurso sospechoso y un grupo reducido de administradores revisa los reportes acumulados, decidiendo en cada caso si descartarlos como falso positivo o eliminar el recurso reportado.

El endpoint `POST /api/resources/{id}/report` recibe un cuerpo JSON con un único campo `reason` que el *frontend* obtiene de un formulario, lo valida (no puede estar vacío) e inserta una fila en la tabla `reports` con el identificador del recurso, el del usuario que reporta y el motivo. La integridad del sistema se apoya en una restricción `UNIQUE(resource_id, reporter_id)` definida en el esquema, que impide que un mismo usuario reporte dos veces el mismo recurso. El segundo intento devuelve un error de violación de unicidad de PostgreSQL que se traduce en un `500 Internal Server Error` desde el *handler*, lo que en la práctica funciona como protección frente a campañas de *spam* de reportes coordinadas, ya que un atacante necesitaría una cuenta distinta por cada reporte y la barrera de entrada (autenticarse con Google) es lo suficientemente alta como para desalentar el abuso.

El paquete `internal/admin` agrupa las tres operaciones disponibles para los administradores. La primera, `ListReports`, devuelve una vista paginada de los reportes pendientes con toda la información que el panel de administración necesita para tomar una decisión sin más consultas, incluyendo el título del recurso reportado, el nombre del autor original y el del usuario que lo ha reportado, y el motivo introducido. Esta agregación se resuelve con una única consulta SQL que une las tablas `reports`, `resources` y `users` (esta última dos veces, una por el reporter y otra por el owner), aprovechando la flexibilidad de sqlc para generar funciones tipadas a partir de *joins* complejos sin las contorsiones que un ORM impondría sobre la misma consulta. Las otras dos operaciones son `DismissReport`, que descarta un reporte concreto sin afectar al recurso, y `DeleteResource`, que elimina el recurso completo cuando el reporte se confirma como legítimo.

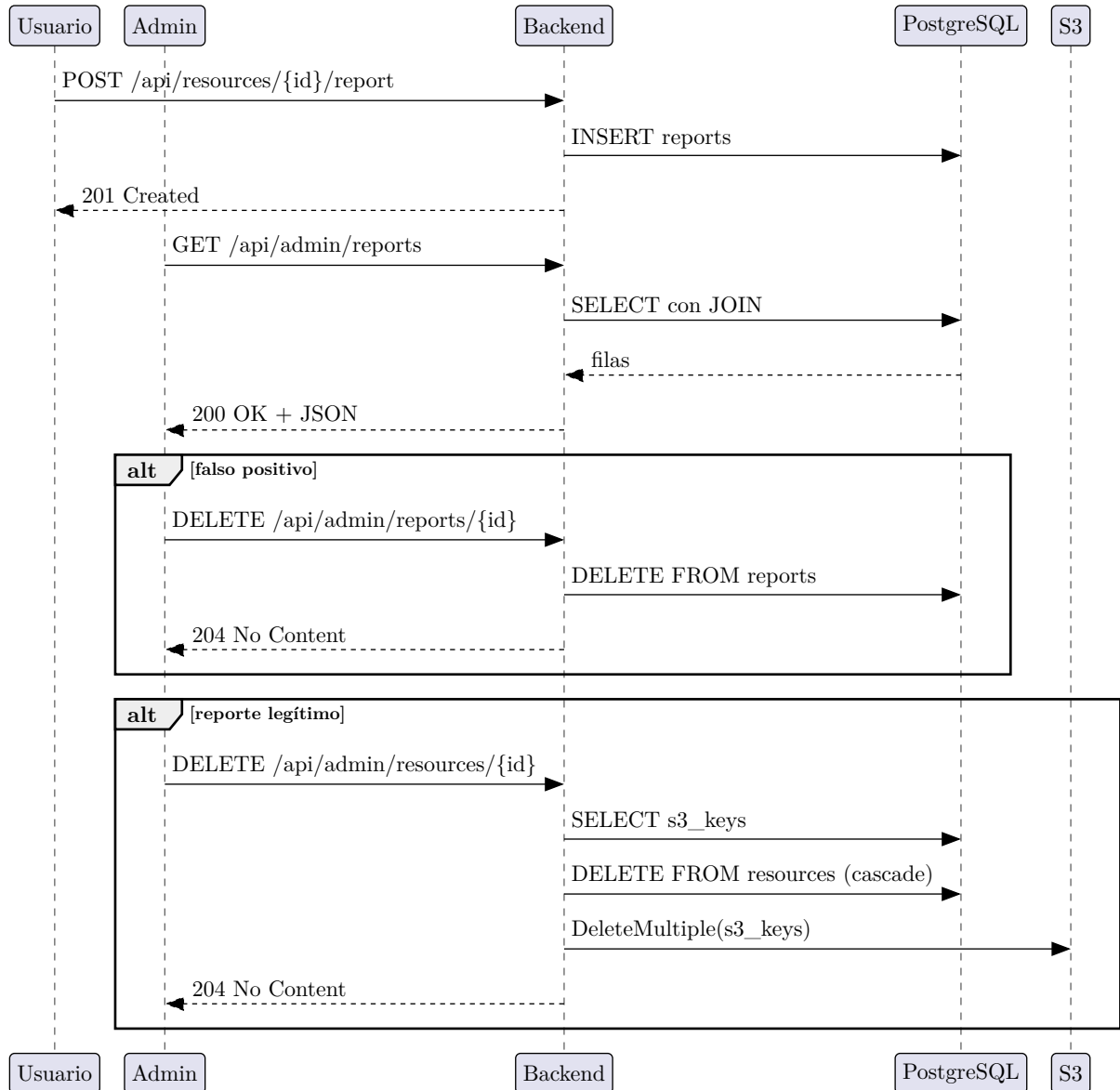


Figura 25: Ciclo de vida de un reporte, desde la denuncia del usuario hasta la resolución por parte de un administrador.

La operación `DeleteResource` del administrador conviene examinarla con detalle porque combina dos mecanismos de limpieza distintos. Antes de borrar la fila de `resources`, el *handler* consulta la lista de claves S3 asociadas al recurso mediante la consulta `ListS3KeysByResource`, que devuelve todos los `s3_key` registrados en la tabla `resource_files`. A continuación borra el recurso de la base de datos, lo que desencadena automáticamente el `ON DELETE CASCADE` declarado en el esquema y arrastra consigo las filas correspondientes de `resource_files` y `reports`. Por último, llama a `Storage.DeleteMultiple` con la lista previamente obtenida para eliminar los objetos del *bucket* en una única petición. El orden importa, ya que invertir las dos primeras llamadas dejaría el sistema en un estado en el que `s3_keys` ya no se podría obtener (porque la fila habría sido borrada con todo lo que cuelga de ella) pero los objetos seguirían en S3.

Esta operación es la versión administrativa del patrón de compensación introducido en Sección 3.5.4, aunque aquí el compromiso es distinto. Un fallo en `DeleteMultiple` después del `DELETE` de PostgreSQL deja objetos huérfanos en S3 sin afectar al funcionamiento del sistema, ya que ninguna fila los referencia, por lo que el *handler* se limita a registrar el error en el *log* y devolver `204 No Content` al administrador. La limpieza tardía de esos objetos podría delegarse en una tarea periódica que enumere el *bucket* y compare con la base de datos, pero por ahora no se ha implementado, ya que el volumen de borrados es reducido y el coste de almacenamiento de los objetos huérfanos resulta despreciable.

3.6 Implementación del frontend

Tras la descripción del backend, esta sección recoge la otra mitad del sistema. El frontend es una aplicación SvelteKit que se encarga de presentar los datos expuestos por la API, gestionar la sesión del usuario y orquestar las interacciones más complejas, en particular la subida de recursos y la búsqueda. Los apartados siguientes recorren la organización del proyecto, el ciclo de vida de una página, el modelo de reactividad de Svelte 5, la comunicación con el backend, los dos flujos de interfaz más representativos (creación y búsqueda) y, por último, la adaptación a móvil y el sistema de componentes reutilizados.

3.6.1 Estructura del proyecto

El proyecto sigue la convención de SvelteKit, que distingue tres ámbitos por la ubicación de los ficheros. El directorio `src/routes/` define el árbol de URL mediante carpetas, donde cada `+page.svelte` es la página renderizada y los `+page.server.ts` que la acompañan ejecutan código exclusivamente en el servidor. El directorio `src/lib/` agrupa todo lo reutilizable entre páginas y es accesible desde cualquier punto mediante el alias `$lib`. Y los ficheros sueltos en la raíz de `src/` son los puntos de entrada de la aplicación: `app.html` es la plantilla HTML base, `app.css` carga Tailwind v4 mediante una sola directiva `@import "tailwindcss"`, y `hooks.server.ts` intercepta cada petición entrante antes de que llegue al *router*. La Figura 26 reproduce la salida de `tree src -L 4 -I node_modules --dirsfirst` ejecutado sobre la raíz del frontend, con los directorios resaltados como en una terminal real.

```

src
├── lib
│   ├── actions
│   │   ├── clickOutside.svelte.test.ts
│   │   └── clickOutside.ts
│   ├── assets
│   │   └── images
│   │       └── screenshot.png
│   ├── components
│   │   ├── HighlightText.svelte
│   │   ├── HighlightText.svelte.test.ts
│   │   ├── ImageViewer.svelte
│   │   ├── MarkdownViewer.svelte
│   │   ├── PdfThumbnail.svelte
│   │   ├── PdfViewer.svelte
│   │   ├── ResourceList.svelte
│   │   ├── ResourceMenu.svelte
│   │   ├── ResourceView.svelte
│   │   └── UserAvatar.svelte
│   ├── index.ts
│   └── types.ts
├── routes
│   ├── admin
│   │   ├── +page.server.ts
│   │   └── +page.svelte
│   ├── auth
│   │   ├── +layout.svelte
│   │   └── +page.svelte
│   ├── cookies
│   │   └── +page.svelte
│   ├── create
│   │   ├── +page.server.ts
│   │   └── +page.svelte
│   ├── privacy
│   │   └── +page.svelte
│   ├── search
│   │   └── +page.svelte
│   ├── tos
│   │   └── +page.svelte
│   ├── user
│   │   └── [slug]
│   │       ├── +page.server.ts
│   │       └── +page.svelte
│   ├── +layout.server.ts
│   ├── +layout.svelte
│   ├── +page.server.ts
│   └── +page.svelte
├── app.css
├── app.d.ts
├── app.html
└── hooks.server.ts
16 directories, 35 files

```

Figura 26: Salida de `tree src -L 4 -I node_modules --dirsfirst` sobre la raíz del frontend.

Dentro de `src/lib/`, los componentes reutilizables se concentran en `components/`, las *custom actions* de Svelte en `actions/`, las imágenes y otros recursos estáticos en `assets/`, y las definiciones compartidas de tipos en `types.ts`. Estas últimas son interfaces TypeScript que reflejan la forma de los DTO que devuelve la API (User, Resource, Subject, ResourceFile), de manera que cada respuesta JSON puede tiparse en el punto de consumo sin duplicar declaraciones. La Tabla 10 resume la responsabilidad de cada directorio.

Directorio	Responsabilidad
<code>src/routes/</code>	Páginas y <i>layouts</i> de la aplicación. Cada carpeta define una URL y los ficheros con prefijo <code>+</code> tienen un rol asignado por SvelteKit (<code>+page.svelte</code> , <code>+page.server.ts</code> , <code>+layout.svelte</code> , <code>+layout.server.ts</code>).
<code>src/lib/components/</code>	Componentes Svelte reutilizables: visores de PDF, imagen y Markdown, lista de recursos, menú contextual, <i>avatar</i> de usuario y resaltado de coincidencias en búsqueda.

Directorio	Responsabilidad
src/lib/actions/	<i>Custom actions</i> aplicables a cualquier elemento mediante <code>use:.</code> Actualmente alberga <code>clickOutside</code> , usada para cerrar menús al pulsar fuera.
src/lib/types.ts	Interfaces TypeScript compartidas que reflejan los DTO de la API.
hooks.server.ts	<i>Middleware</i> ejecutado en cada petición SSR antes del <i>router</i> , encargado de poblar <code>event.locals.user</code> a partir de la cookie de sesión.
static/	Activos servidos tal cual por el servidor, como el <code>favicon.svg</code> y los iconos para PWA.

Tabla 10: Directorios principales del frontend y su responsabilidad.

3.6.2 Ciclo de vida de una página

Una de las características que distinguen a SvelteKit de un SPA tradicional es que la primera carga de cualquier página se renderiza en el servidor. El navegador recibe HTML completo con los datos ya inyectados, y solo después se hidrata la aplicación para tomar el control de las navegaciones siguientes en el cliente. La Figura 27 ilustra las cinco fases de una petición a una ruta cualquiera.

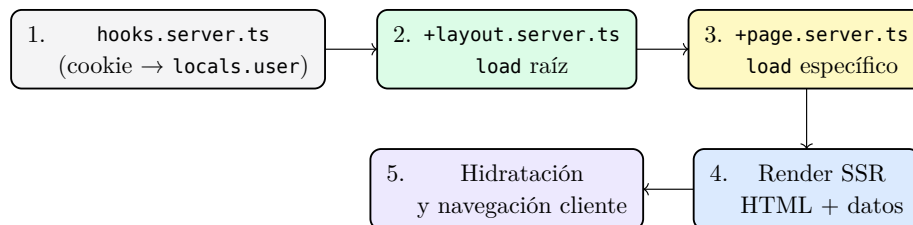


Figura 27: Fases de la primera carga de una página y transición a navegación cliente.

El primer eslabón es el *hook* `handle` definido en `hooks.server.ts`, que se ejecuta antes que cualquier `load`. Este *hook* lee la cookie `session_token` enviada por el navegador y, si existe, la reenvía al backend mediante una llamada interna a `/api/me` para resolver el usuario autenticado. El resultado se deposita en `event.locals.user`, una estructura accesible desde cualquier `load` durante la misma petición. Centralizar esta operación en el *hook* evita que cada página repita la consulta y, lo que es más importante, garantiza una única fuente de verdad sobre la identidad del usuario para toda la cadena de renderizado.

A continuación se ejecuta el `load` del *layout* raíz, que es el que ven todas las páginas que no estén bajo `/auth`. Su misión es exponer `data.me` al árbol de componentes y, de paso, redirigir al usuario hacia `/auth` cuando ha completado el *login* con Google pero todavía no ha elegido estudio. Las páginas hijas obtienen este dato sin volver a pedirlo, gracias al patrón `parent()` de SvelteKit.


```

1  // src/routes/+layout.server.ts
2  export const load: LayoutServerLoad = async ({ fetch, url }) => {
3      let me: User | null = null;
4      const res = await fetch(`${PUBLIC_API_BASE_URL}/api/me`, {
5          credentials: 'include',
6      });
7      if (res.ok) me = await res.json();
8
9      const isAuthPage = url.pathname.startsWith('/auth');
10     if (me && me.studyId == null && !isAuthPage) {
11         redirect(302, '/auth');
12     }
13     return { me };
14 };

```

Código 11: *Load* del *layout* raíz, que expone el usuario y fuerza la elección de estudio antes de continuar.

Las páginas que necesitan datos adicionales declaran su propio `+page.server.ts`. Por ejemplo, `/create` consulta las asignaturas fijadas del usuario para poblar el desplegable del formulario, y aprovecha la llamada a `parent()` para reutilizar el `me` ya resuelto en lugar de pedirlo de nuevo. Si el usuario no está autenticado o no ha elegido estudio, la página redirige antes de renderizar, descargando al cliente de cualquier comprobación.

Concluido el `load`, SvelteKit serializa los datos en el HTML y los envía al navegador. La aplicación se hidrata, los *runes* se inicializan y, a partir de ese momento, cada navegación interna se resuelve en el cliente reemplazando solo la zona de la página afectada. Esta arquitectura SSR-más-hidratación combina lo mejor de ambos mundos. La primera carga es rápida porque el HTML viaja con los datos ya impresos, y las navegaciones posteriores son instantáneas porque no requieren un viaje completo al servidor.

3.6.3 Reactividad con runes de Svelte 5

El frontend se construyó sobre Svelte 5, la versión que sustituyó el sistema de reactividad implícito por un modelo explícito basado en *runes*. En lugar de detectar dependencias mediante la reasignación de variables declaradas con `let`, Svelte 5 introduce funciones especiales que el compilador reconoce y traduce en señales reactivas. La Tabla 11 resume las cuatro empleadas en el proyecto.

Rune	Descripción	Ejemplo de uso en Nemsy
<code>\$state</code>	Declara un valor reactivo. Cualquier lectura desde la	Estado del formulario <code>/create</code> : <code>title</code> , <code>description</code> , <code>selectedFiles</code> .

Rune	Descripción	Ejemplo de uso en Nemsy
	plantilla o desde un <code>\$derived</code> se vuelve dependiente.	
<code>\$derived</code>	Define un valor calculado a partir de otros estados. Se recomputa automáticamente cuando cambian sus dependencias.	<code>filesArray = \$derived(Array.from(selectedFiles))</code> para iterar el Set en la plantilla.
<code>\$props</code>	Recibe las <i>props</i> tipadas que el padre pasa al componente.	<code>let { data } = \$props()</code> en cada <code>+page.svelte</code> para acceder al resultado del load.
<code>\$effect</code>	Ejecuta un bloque cada vez que cambian los estados que lee. Útil para sincronizar estado con APIs externas (DOM, <code>localStorage</code>).	Persistencia del modo de vista compacto/comforte en <code>localStorage</code> .

Tabla 11: Runes de Svelte 5 utilizados en el frontend.

Un caso particularmente ilustrativo es el del formulario de creación, que combina los cuatro mecanismos en pocas líneas. La selección de archivos se almacena en un `Set<File>` reactivo, del que se deriva un array para iterar y, sobre ese array, otro `$derived` calcula el archivo que se mostrará en la previsualización lateral. El usuario puede forzar manualmente cuál ver con un click, lo que actualiza un tercer estado.

```

1  let selectedFiles = $state<Set<File>>(new Set());
2  let manualPreviewFile = $state<File | null>(null);
3
4  const filesArray = $derived(Array.from(selectedFiles));
5  const previewFile = $derived(
6    filesArray.length === 0
7    ? null
8    : manualPreviewFile !== null &&
      filesArray.includes(manualPreviewFile)
9    ? manualPreviewFile
10   : filesArray[0]
11 );

```

svelte

Código 12: Estado y derivaciones del visor de previsualización en `/create`.

La diferencia frente al modelo anterior de Svelte es que aquí no hay magia oculta. El compilador no necesita inferir qué variables son reactivas escaneando reasignaciones, sino que basta con que sigan la convención de los *runes*. Esto facilita razonar sobre el código y permite usar estado reactivo dentro de funciones, *stores* y módulos sin las restricciones del compilador clásico.

3.6.4 Comunicación con el backend

La aplicación se comunica con el backend Go por dos caminos distintos según el momento del ciclo de vida de la página. Durante el SSR, las llamadas las realiza la función `fetch` que SvelteKit inyecta en cada `load`, una variante especial que reescribe las URL relativas y reenvía las cookies de la petición original. Tras la hidratación, las llamadas las hace directamente el navegador con la API `fetch` estándar, enviando la cookie `session_token` automáticamente al ser de primer nivel de dominio. El Figura 28 ilustra el camino de una llamada autenticada.

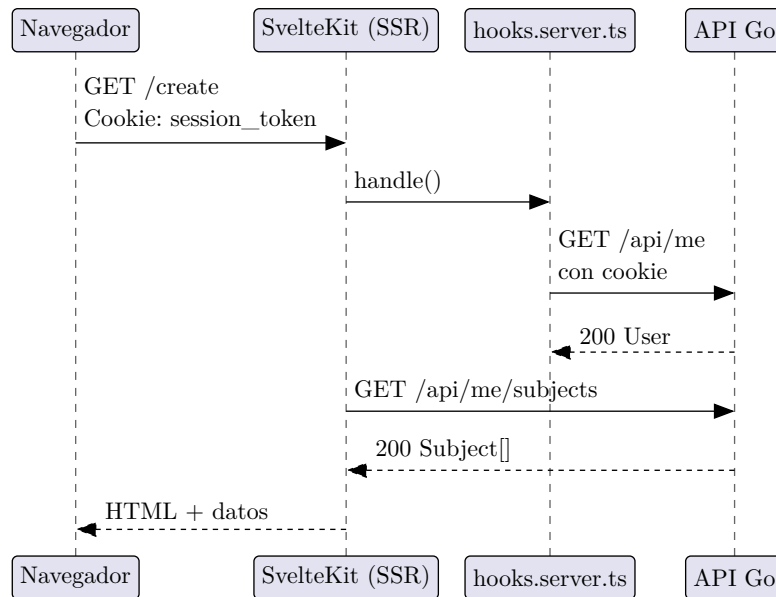


Figura 28: Camino de una petición SSR autenticada hasta la API.

El *endpoint* base se inyecta a través de la variable de entorno `PUBLIC_API_BASE_URL`, que Vite expone tanto al servidor como al cliente con el prefijo `PUBLIC_`. En el cliente apunta al dominio público de la API y en el servidor puede apuntar a una dirección interna de la red de contenedores, lo que evita un salto innecesario por el balanceador en producción. La Tabla 12 resume los *endpoints* que el frontend consume agrupados por dominio.

Dominio	Endpoint	Uso
Identidad	GET /api/me	Resolver el usuario autenticado en <code>hooks.server.ts</code> y en el <i>layout</i> raíz.
	PATCH /api/me	Actualizar universidad, estudio o asignaturas fijadas desde

Dominio	Endpoint	Uso
		la pantalla de perfil.
	GET /api/me/subjects	Poblar el desplegable de asignaturas en /create y la lista lateral en /.
Recursos	POST /api/resources	Crear un recurso con archivos en formato multipart/form-data.
	GET /api/subjects/{id}/resources	Listar los recursos de una asignatura.
	GET /api/resources/search	Búsqueda de texto completo paginada (50 resultados por página).
	GET /api/resources/{id}/download	Descargar el recurso completo empaquetado en ZIP.
	GET /api/resources/{id}/files/{fileId}/download	Descargar un archivo individual del recurso.
	POST /api/resources/{id}/reports	Reportar un recurso desde el menú contextual.
Búsqueda	GET /api/universities/search	Autocompletado de universidad en /auth.
	GET /api/studies/search	Autocompletado de estudio tras elegir universidad.
Admin	GET /api/admin/reports	Listado de reportes en el pa-

Dominio	Endpoint	Uso
		nel de administración.
	POST /api/admin/reports/{id}/resolve	Resolver un reporte sin borrar el recurso.
	DELETE /api/admin/resources/{id}	Eliminar un recurso reportado.

Tabla 12: Endpoints de la API consumidos desde el frontend.

3.6.5 Subida de recursos: formulario multipaso

La pantalla `/create` es la más compleja del frontend y la que mejor refleja la combinación de *runes*, componentes de terceros y comunicación con la API. El usuario debe elegir una asignatura, asignar un título, redactar una descripción opcional, seleccionar uno o varios archivos y aceptar la cesión de derechos antes de poder enviar el formulario. Mientras compone el envío, una mitad de la pantalla muestra una previsualización del archivo seleccionado renderizada localmente con `URL.createObjectURL`, sin necesidad de subir nada al servidor todavía.

La selección de asignatura emplea el componente `Combobox` de `bits-ui`, que permite escribir para filtrar y conserva el último valor escogido en `localStorage` bajo la clave `lastSubject`, de modo que las subidas consecutivas a la misma asignatura no exigen volver a buscarla. La selección de archivos utiliza el componente `FileUpload` de `melt` con *drag and drop* y multiselección, que devuelve un `Set<File>` enlazado al estado reactivo del formulario. La previsualización se delega en tres visores propios alojados en `src/lib/components/`, cada uno especializado en un tipo de archivo, según muestra la Tabla 13.

Componente	Tipos	Tecnología
<code>PdfViewer.svelte</code>	<code>.pdf</code>	<code>pdfjs-dist</code> con renderizado por página y navegación.
<code>ImageViewer.svelte</code>	<code>.jpg .png .gif .webp</code>	Etiqueta <code></code> con encuadre adaptativo.
<code>MarkdownViewer.svelte</code>	<code>.md .markdown</code>	<code>marked</code> con sanitización antes del render.

Tabla 13: Visores de previsualización empleados en `/create` y en la consulta de recursos.

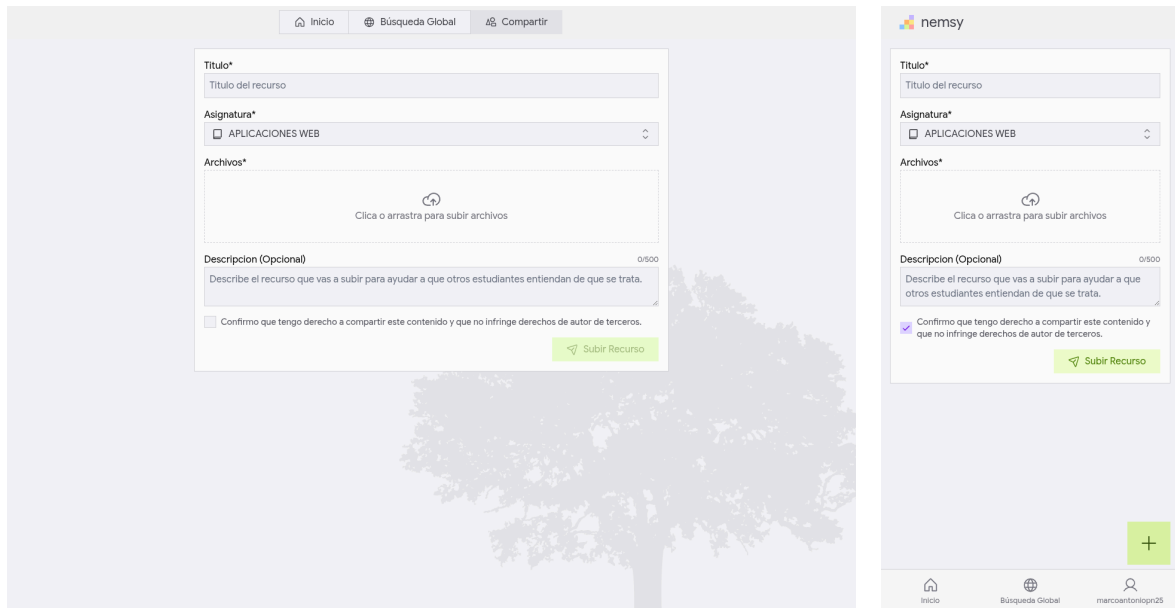


Figura 29: Formulario de creación de un recurso en escritorio y en móvil.

Cuando el usuario pulsa enviar, el formulario construye un `FormData` con los campos textuales y cada archivo, y lo envía al endpoint `POST /api/resources` con `credentials: 'include'`. La respuesta exitosa devuelve el identificador del recurso recién creado, que se utiliza para redirigir al usuario a su perfil mediante `goto`. Los URL de objeto creados durante la previsualización se liberan en un `onDestroy` con `URL.revokeObjectURL` para no fugar memoria al desmontar la página.

3.6.6 Búsqueda con *debounce* y desplazamiento infinito

La búsqueda global en `/search` es una entrada de texto que dispara consultas al *endpoint* FTS del backend descrito en Sección 3.7.1. Para no saturar la API mientras el usuario teclea, cada cambio en el *input* programa una llamada con un retardo de 300 milisegundos, y un nuevo cambio en ese intervalo cancela la programada y arranca otra. Este patrón clásico de *debounce* se implementa con `setTimeout` y `clearTimeout` sobre una variable suelta del módulo, sin necesidad de *runes*.

```

1 let query = $state('');
2 let results = $state<Resource[]>([]);
3 let debounceTimer: ReturnType<typeof setTimeout>;
4
5 function onInput() {
6   clearTimeout(debounceTimer);
7   debounceTimer = setTimeout(() => search(false), 300);
8 }

```

Código 13: Implementación del *debounce* en el *input* de búsqueda.

La paginación es de tipo *scroll* infinito. La función `search` acepta un parámetro `append` que, cuando es cierto, calcula el *offset* a partir de los resultados ya cargados y

concatena los nuevos al estado en lugar de reemplazarlo. Un `IntersectionObserver` montado en `onMount` vigila un elemento centinela colocado al final de la lista, y dispara una nueva llamada con `append = true` cada vez que el centinela entra en el *viewport*, siempre que haya más resultados disponibles y no haya otra petición en curso. Las coincidencias en el título y la descripción se resaltan con el componente `HighlightText`, que parte el texto en torno a los términos buscados y envuelve cada coincidencia en un `<mark>` con clases de Tailwind.

3.6.7 Responsividad

La interfaz se adapta a dos contextos de uso completamente distintos. En escritorio se prioriza la densidad de información y la navegación con el cursor, mientras que en móvil se busca minimizar los *taps* y aprovechar los gestos. Esto ya se introdujo brevemente en la Tabla 4, pero conviene detallar los mecanismos que lo hacen posible. La Tabla 14 recoge los *breakpoints* de Tailwind v4 utilizados en el proyecto.

Prefijo	Mínimo	Uso típico en Nemsy
<i>(default)</i>	0 px	Estilos base, asumidos para móvil.
<code>md:</code>	768 px	Activación de la barra superior y desactivación de la barra inferior.
<code>lg:</code>	1024 px	Disposición a dos columnas en <code>/create</code> y en la consulta de recursos.

Tabla 14: *Breakpoints* de Tailwind utilizados en el frontend.

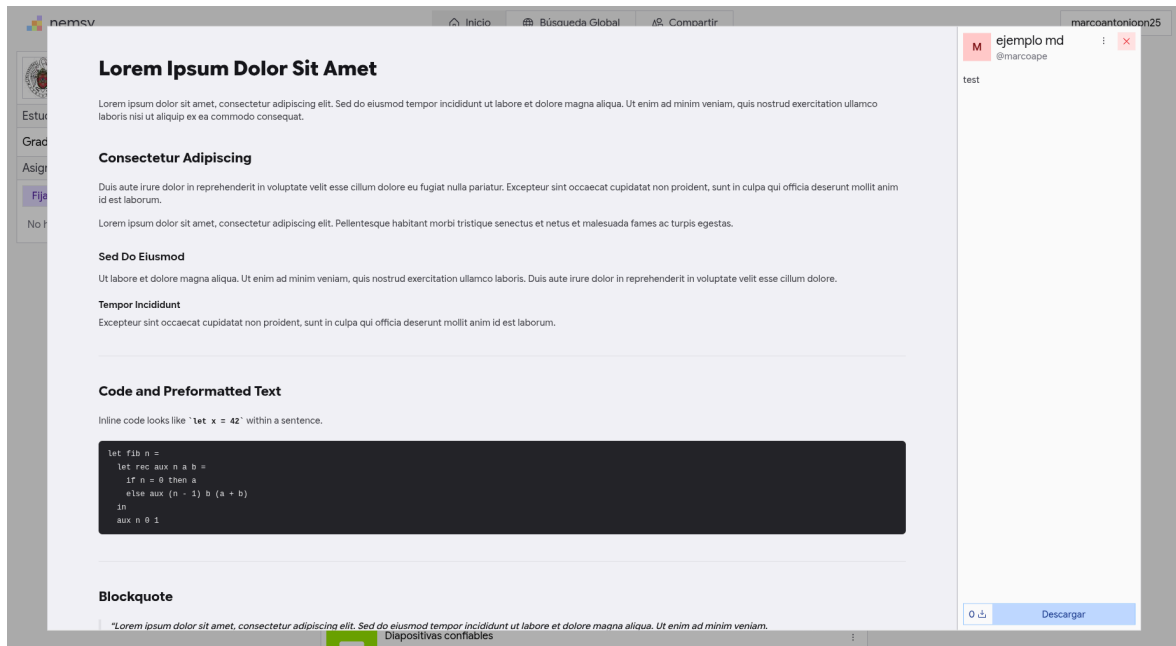


Figura 30: Misma vista de recurso en escritorio y en móvil.

El `+layout.svelte` raíz aplica este principio en su forma más pura. La barra de navegación superior se envuelve en clases `hidden md:flex`, lo que la oculta por defecto y solo la muestra a partir de 768 píxeles. Una segunda barra fija a la parte inferior, con tres iconos grandes para las acciones principales y un botón de acción flotante centrado para subir un recurso, lleva las clases inversas `flex md:hidden`, de manera que ambas se reemplazan limpiamente sin necesidad de *media queries* explícitas en CSS. Las páginas internas siguen el mismo patrón. Por ejemplo, en `/create` la previsualización lateral se oculta en pantallas pequeñas y se muestra en una pestaña aparte, y en `/search` la rejilla de resultados pasa de cuatro columnas a una sola.

3.6.8 Sistema de componentes

El frontend se apoya en una pequeña selección de bibliotecas de componentes *headless* que aportan accesibilidad y comportamiento sin imponer estilos, dejando que toda la apariencia se resuelva con utilidades de Tailwind. La Tabla 15 resume cada una y su rol.

Biblioteca	Rol
<code>bits-ui</code>	Primitivas accesibles de UI: <code>Combobox</code> , <code>Dialog</code> , <code>Checkbox</code> , <code>DropdownMenu</code> . Cubren la mayoría de los controles de formulario y los modales.
<code>melt</code>	Builders de bajo nivel para casos no cubiertos por <code>bits-ui</code> . En el proyecto se usa <code>FileUpload</code> por su soporte nativo de <i>drag and drop</i> con <code>Set<File></code> .
<code>phosphor-svelte</code>	Iconografía completa con <i>tree-shaking</i> por icono, lo que evita cargar el conjunto entero. Cada vista importa solo los iconos que necesita.
<code>tailwindcss</code> (v4)	Sistema de utilidades CSS configurado mediante <code>@import</code> en <code>app.css</code> , sin fichero <code>tailwind.config.js</code> .

Tabla 15: Bibliotecas de UI utilizadas en el frontend.

Por encima de estas primitivas, el directorio `src/lib/components/` aloja diez componentes propios con responsabilidades concretas. `ResourceList` y `ResourceView` son los más usados, pues aparecen tanto en la página de inicio como en los perfiles de usuario y en los resultados de búsqueda, y encapsulan la presentación de un recurso con sus archivos, autor, fecha y estadísticas. `ResourceMenu` despliega el menú contextual de cada recurso sobre la base de `DropdownMenu` de `bits-ui`, e incorpora la *custom action* `clickOutside` para cerrarse al pulsar fuera. `UserAvatar` resuelve la imagen de perfil con un fallback a las iniciales del usuario cuando Google no devuelve avatar. Los tres visores ya mencionados (`PdfViewer`, `ImageViewer`, `MarkdownViewer`) y el `PdfThumbnail` que genera la miniatura de la primera página del PDF para los listados completan el listado de componentes propios. Esta separación entre primitivas externas y componentes de dominio mantiene los ficheros de las páginas concentrados en la lógica de la vista, mientras que los detalles de presentación quedan encapsulados y reutilizables.

3.6.9 Panel de administración

La ruta `/admin` da acceso a una vista reservada a los usuarios con rol de administrador, encargada de gestionar los reportes que los usuarios envían sobre los recursos compartidos. La protección se aplica en dos capas. En el frontend, el `+page.server.ts` comprueba `me.role === 'admin'` durante el `load` y redirige a la página de inicio en caso contrario, evitando renderizar la página por completo a usuarios sin permiso. En el backend, el *middleware* `AdminOnly` introducido en Sección 3.5.6 rechaza cualquier llamada a `/api/admin/*` con un `403 Forbidden` cuando el JWT del solicitante no

contiene el *claim* de administrador, de modo que la restricción se sostiene aunque el cliente intentara saltarse la comprobación del frontend.

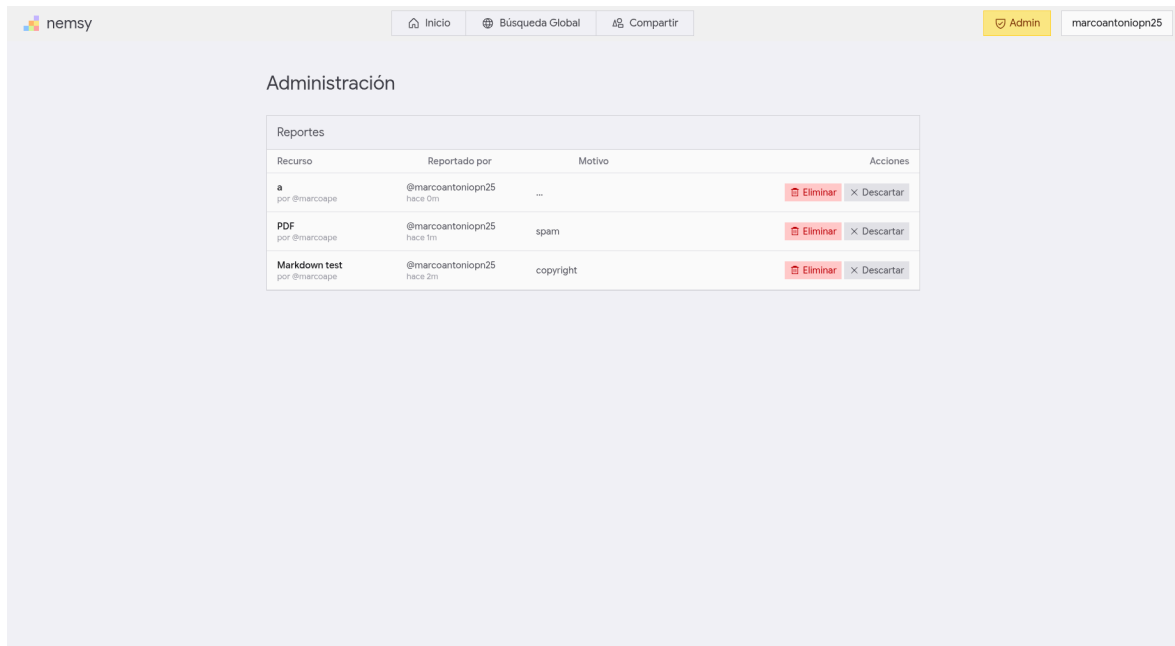


Figura 31: Panel de administración con la lista de reportes pendientes.

La pantalla muestra una tabla con los reportes ordenados por fecha de creación descendente, indicando para cada uno el recurso reportado, el reportante, el motivo y la antigüedad. Cada fila ofrece dos acciones excluyentes. La primera, *descartar*, marca el reporte como atendido sin tocar el recurso, útil cuando el administrador determina que el contenido es legítimo y el reporte infundado. La segunda, *eliminar*, dispara el `DELETE /api/admin/resources/{id}` descrito en Sección 3.5.7 tras una confirmación rápida, que borra el recurso y, en cascada, todos sus archivos en S3 y los reportes asociados.

3.7 Pruebas y validación

Para garantizar la corrección y el rendimiento de la plataforma se aplicaron tres niveles de pruebas complementarios, cubriendo los tests unitarios del backend con el paquete `testing` de Go, los tests unitarios y *end to end* del frontend con Vitest y Playwright, y una prueba de carga con k6 sobre la API desplegada en producción. Esta última no solo valida que el sistema no falla bajo carga, sino que sirve como demostración empírica del rendimiento de la plataforma, aportando los datos que respaldan uno de sus objetivos centrales, ofrecer una experiencia rápida como alternativa a Wuolah.

3.7.1 Tests unitarios del backend

Los tests unitarios del backend están escritos con el paquete `testing` de la librería estándar de Go y `httptest` para simular peticiones HTTP sin levantar un servidor real. Cada paquete define un `mockQuerier` que implementa la interfaz `QuerierWithTx` generada por `sqlc`, de forma que los tests son completamente independientes de la base de datos y se ejecutan de forma determinista.

Los tests del paquete `auth` cubren la generación y verificación de tokens JWT, la extracción de claims y el comportamiento del middleware ante distintos escenarios, incluyendo petición sin cookie, token con firma incorrecta y token válido. Este último caso verifica además que el middleware inyecta correctamente la información del usuario en el contexto de la petición.

```

1  func TestAuthMiddleware_ValidToken(t *testing.T) {
2      secret := []byte("testsecret")
3      tokenStr, _ := GenerateJWTWithUserID(
4          UserInfo{GoogleSub: "123456", Email: "test@example.com"},
5          42, "user", secret,
6      )
7
8      req := httptest.NewRequest("GET", "/protected", nil)
9      req.AddCookie(&http.Cookie{Name: "session_token", Value:
10         tokenStr})
11      rr := httptest.NewRecorder()
12
13      mw := &AuthMiddleware{Secret: secret}
14      handlerCalled := false
15
16      mw.Middleware(http.HandlerFunc(func(w http.ResponseWriter, r
17         *http.Request) {
18         handlerCalled = true
19         w.WriteHeader(http.StatusOK)
20     })).ServeHTTP(rr, req)
21
22     if rr.Code != http.StatusOK { t.Errorf("expected 200, got %d",
23         rr.Code) }
24     if !handlerCalled { t.Error("protected handler was not called") }
25 }
```

Código 14: Test del middleware de autenticación con token válido.

Los tests del paquete `search` verifican la función `PrefixQuery`, que transforma la entrada del usuario en una expresión compatible con el operador `@@` de PostgreSQL Full Text Search. Los casos incluyen búsquedas con una o varias palabras, eliminación de tildes y caracteres especiales, espacios extra y entrada vacía.

```

1  func TestPrefixQuery(t *testing.T) {
2      tests := []struct{ name, input, want string }{
3          {"single word", "matematicas", "matematicas:*"},
4          {"multiple words", "algebra lineal", "algebra:* & lineal:*"},
5          {"strips accents", "programación", "programacion:*"},
6          {"removes specials", "hello! world?", "hello:* & world:*"},
7          {"empty input", "", ""},
8          {"extra whitespace", "  fisica cuantica  ", "fisica:* & cuantica:*"},
9      }
10     for _, tt := range tests {
11         t.Run(tt.name, func(t *testing.T) {
12             got := PrefixQuery(tt.input)
13             if got != tt.want {
14                 t.Errorf("PrefixQuery(%q) = %q, want %q", tt.input, got, tt.want)
15             }
16         })
17     }
18 }

```

Código 15: Tests de la función PrefixQuery para Full Text Search.

Los tests de los *handlers* siguen el mismo patrón: instancian un `mockQuerier` con funciones anónimas que devuelven datos controlados, construyen una petición HTTP con `httptest.NewRequest`, inyectan el contexto de autenticación manualmente y verifican tanto el código de estado como el cuerpo de la respuesta.

3.7.2 Tests unitarios del frontend

Los tests unitarios del frontend se escribieron con Vitest [23] y se centran en los dos tipos de lógica que pueden probarse de forma aislada, los componentes de interfaz y las acciones de Svelte.

El componente `HighlightText` resalta la subcadena buscada dentro de un texto envolviéndola en un elemento `<mark>`. Sus tests verifican los tres casos posibles, que resalta correctamente ignorando mayúsculas y tildes, que no introduce ningún `<mark>` cuando no hay coincidencia, y que tampoco lo hace con una búsqueda vacía.

```

1  it('highlights the matching substring', async () => { typescript
2      render(HighlightText, { text: 'Álgebra Lineal', query:
        'lineal' });
3      const mark = page.getByRole('mark');
4      await expect(element(mark).toHaveTextContent('Lineal'));
5  });
6
7  it('renders plain text when there is no match', async () => {
8      const { container } = render(HighlightText, { text: 'Cálculo',
        query: 'física' });
9      expect(container.querySelectorAll('mark').length).toBe(0);
10 });

```

Código 16: Tests unitarios del componente HighlightText.

La acción `clickOutside` detecta clics fuera de un elemento del DOM y emite un evento `outclick`, usado para cerrar menús desplegables. Los tests verifican tanto que el evento se dispara al hacer clic fuera del nodo, como que no se dispara al hacer clic dentro, y que deja de escuchar tras llamar a `destroy`.

```

1  it('dispatches outclick when clicking outside the node', typescript
    () => {
2      const node = document.createElement('div');
3      const outside = document.createElement('div');
4      document.body.appendChild(node);
5      document.body.appendChild(outside);
6
7      let fired = false;
8      node.addEventListener('outclick', () => { fired = true; });
9
10     const action = clickOutside(node);
11     outside.click();
12
13     expect(fired).toBe(true);
14     action.destroy!();
15 });

```

Código 17: Test unitario de la acción `clickOutside`.

3.7.3 Tests end to end con Playwright

Los tests *end to end* (E2E) verifican los flujos de la aplicación desde el punto de vista del navegador, ejecutando interacciones reales contra el frontend y el backend levantados localmente. Se escribieron con Playwright [24], una herramienta de auto-

matización de navegadores que permite controlar Chrome, Firefox y Safari desde código TypeScript.

El principal reto de los tests E2E de Nemsy es que como la autenticación se delega completamente en Google OAuth2, cuyo flujo no puede automatizarse en un entorno de pruebas, es necesario inyectar directamente una cookie `session_token` con un JWT firmado antes de cada test, usando un helper `generateTestJWT` implementado en TypeScript con la API de criptografía de Node.js. Con esta solución, los tests arrancan ya autenticados sin pasar por el flujo de Google.

```

1  test.beforeEach(async ({ context }) => {                                typescript
2      const jwt = generateTestJWT({
3          sub: 'e2e-test-sub', email: 'e2e@test.edu',
4          hd: 'test.edu',      user_id: 32, role: 'user'
5      });
6      await context.addCookies([
7          name: 'session_token', value: jwt,
8          domain: 'localhost',   path: '/'
9      ]);
10 });

```

Código 18: Inyección de cookie JWT antes de cada test E2E.

Los tests están organizados en dos grupos. El grupo `logged out` verifica que la página de inicio muestra el *hero* y el botón de inicio de sesión cuando no hay sesión activa. El grupo `logged in` cubre los flujos principales, verificando que la página de inicio muestra la barra lateral de asignaturas, que la búsqueda global devuelve resultados al escribir, que el formulario de subida muestra todos sus campos y que el perfil de un usuario es accesible.

```

1  test('search returns results', async ({ page }) => {                    typescript
2      await page.goto('/search');
3      await page.waitForLoadState('networkidle');
4
5      const input = page.getByPlaceholder('Buscar recursos de toda la
6      plataforma...');
7      await input.click();
8      await input.pressSequentially('prueba', { delay: 50 });
9
10     await expect(page.getByText('Recurso de
11     prueba')).toBeVisible({ timeout: 10000 });
12 });

```

Código 19: Test E2E del flujo de búsqueda global.

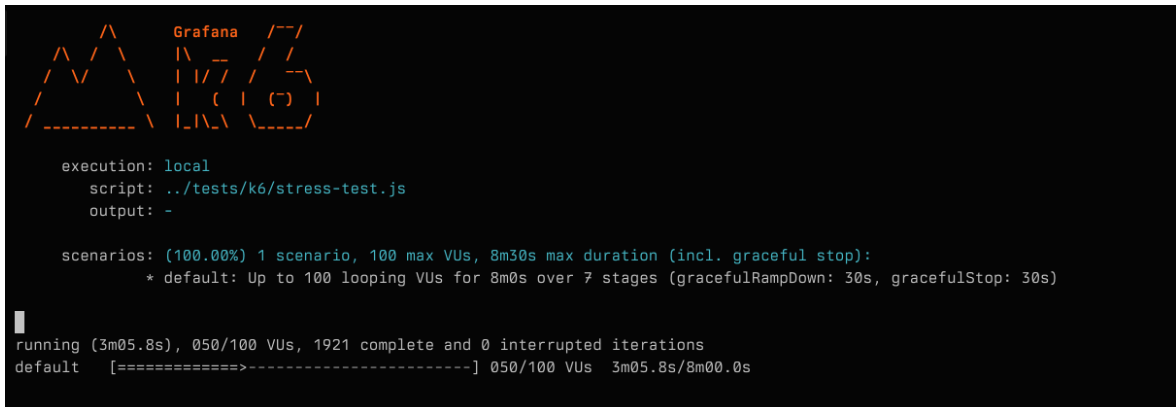
3.7.4 Validación de rendimiento

Más allá de comprobar que el código funciona, conviene medir cómo se comporta la plataforma bajo condiciones realistas y cómo se percibe desde el navegador del usuario. Para cubrir ambos planos se aplicaron dos validaciones complementarias, una prueba de carga con k6 sobre la API en producción que mide la capacidad del servidor frente a tráfico concurrente, y un análisis con Lighthouse que evalúa la experiencia de usuario en métricas de rendimiento, accesibilidad, buenas prácticas y SEO, comparada directamente con la de Wuolah.

3.7.4.1 Prueba de carga con k6

Para validar empíricamente el rendimiento de la API bajo condiciones de uso realistas se realizó una prueba de carga con k6 [25], una herramienta de código abierto orientada a pruebas de rendimiento de APIs HTTP. El escenario simula usuarios virtuales (VUs) navegando por la aplicación, ejecutando en bucle una de tres acciones de forma aleatoria: consultar su perfil y asignaturas, buscar recursos por texto completo o consultar el detalle de un recurso concreto, con pausas de entre 0,5 y 1,5 segundos entre peticiones para simular el tiempo que un usuario real emplea leyendo.

Un VU de k6 no equivale a un usuario real ya que un usuario real hace peticiones de forma esporádica con largos intervalos de inactividad, mientras que un VU las hace de forma continua. Realmente, en la práctica, 100 VUs con pausas de un segundo generan una carga equivalente a varios miles de usuarios activos simultáneamente. La prueba se ejecutó contra la API desplegada en producción en un VPS de Hetzner con 2 vCPU y 4 GB de RAM, aumentando progresivamente la carga desde 10 hasta 100 VUs a lo largo de ocho minutos.



```

Grafana /~/

execution: local
  script: ../tests/k6/stress-test.js
  output: -

scenarios: (100.00%) 1 scenario, 100 max VUs, 8m30s max duration (incl. graceful stop):
  * default: Up to 100 looping VUs for 8m0s over 7 stages (gracefulRampDown: 30s, gracefulStop: 30s)

running (3m05.8s), 050/100 VUs, 1921 complete and 0 interrupted iterations
default [=====>-----] 050/100 VUs 3m05.8s/8m00.0s

```

Figura 32: Ejecución de la prueba de carga con k6 en el VPS de producción.

Endpoint	p50	p90	p95
GET /api/me	1,72 ms	2,27 ms	2,89 ms
GET /api/me/subjects	2,45 ms	3,48 ms	4,18 ms
GET /api/subjects/{id}/resources	1,53 ms	2,22 ms	2,72 ms

Endpoint	p50	p90	p95
GET /api/resources/search	1,75 ms	2,44 ms	2,93 ms
GET /api/resources/{id}	1,96 ms	2,81 ms	3,36 ms
Global	1,83 ms	2,79 ms	3,45 ms

Tabla 16: Latencias por endpoint en la prueba de carga con k6 (100 VUs, 0 % de errores).

El modelo de rendimiento RAIL de Google [26] establece que las respuestas del servidor deben llegar en menos de 100 ms para que el usuario perciba la interacción como inmediata. Como se puede observar en la Figura 32 y en la Tabla 16, la API de Nemsy obtiene un p95 global de 3,45 ms bajo 100 VUs concurrentes, casi treinta veces por debajo de ese umbral, con una tasa de error del 0 %. Cabe destacar que estas latencias corresponden exclusivamente al servidor pero el tiempo que percibe el usuario final incluye además la red, el renderizado del navegador y la ejecución del JavaScript del frontend.

El script de prueba se encuentra en `tests/k6/stress-test.js`. Para ejecutar las rutas protegidas, se incluyó un generador de tokens JWT en `backend/cmd/gen-token/main.go` que firma un token con el mismo secreto que el servidor, evitando la necesidad de pasar por el flujo de Google OAuth2 durante la prueba.

3.7.4.2 Análisis con Lighthouse

Para medir el rendimiento desde la perspectiva del usuario se utilizó Google Lighthouse, ejecutando la auditoría en las mismas condiciones para ambas plataformas mediante Helium [27], un fork ligero de Chromium sin extensiones instaladas y en modo incógnito, accediendo a la página de inicio tras iniciar sesión. El resultado se recoge en la Figura 33.

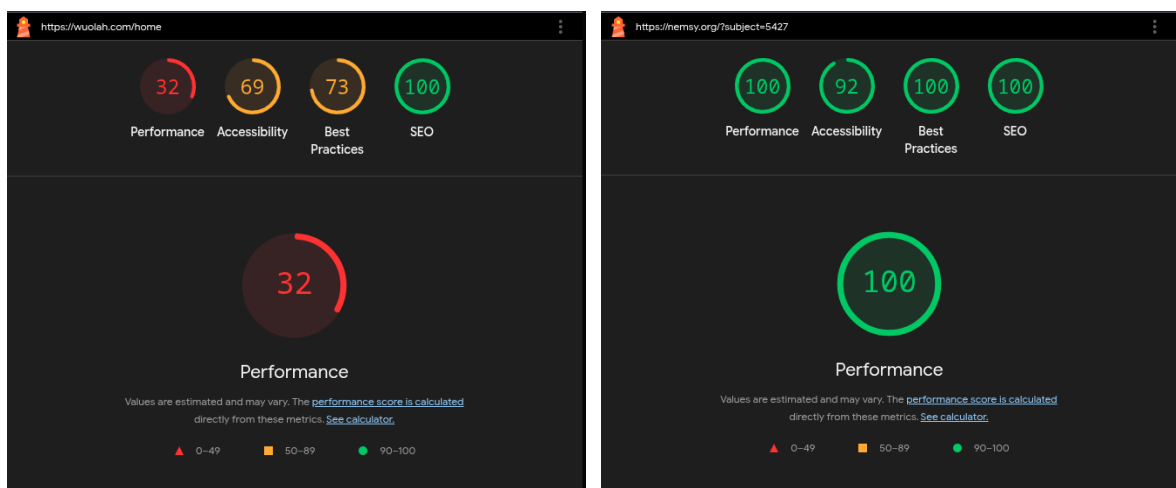


Figura 33: Comparativa de puntuaciones Lighthouse entre Wuolah y Nemsy.

Nemsy obtiene una puntuación de rendimiento de 100 sobre 100, frente al 32 de Wuolah. La diferencia responde principalmente a la ausencia de publicidad y scripts de terceros, que en Wuolah son la principal fuente de bloqueo del hilo principal

del navegador, a un bundle de JavaScript mínimo gracias a que Svelte compila los componentes a JavaScript puro sin necesidad de ningún framework en tiempo de ejecución, y a una API en Go que, como demuestran los resultados de la sección anterior, responde en menos de 4 ms en el percentil 95 independientemente de la carga concurrente.

Capítulo 4 Conclusiones y Trabajo Futuro

Resumen: Este capítulo presenta las conclusiones extraídas del proyecto y las líneas de trabajo futuro.

La perfección no se alcanza cuando ya no hay nada más que añadir, sino cuando ya no queda nada por quitar.
— Antoine de Saint-Exupéry

4.1 Conclusiones

El resultado del proyecto es una plataforma funcional, desplegada en producción en nemsy.org y demostrablemente solvente que cubre el caso de uso planteado, este siendo el de compartir y consultar apuntes universitarios entre estudiantes, sin las concesiones publicitarias y de *paywall* que limitan a las alternativas analizadas en el capítulo de Estado de la Cuestión. La implementación combina un *backend* en Go con `sqlc` y PostgreSQL y un *frontend* en SvelteKit con Svelte 5, dos elecciones que han demostrado ser acertadas tanto en términos de productividad durante el desarrollo como en rendimiento medido en la fase de validación.

Los datos recogidos en la prueba de carga con `k6` y en el análisis con Lighthouse, descritos en Sección 3.7.4.1, muestran que el sistema se comporta de forma estable bajo concurrencia sostenida y obtiene puntuaciones notablemente superiores a Wuolah en las cuatro dimensiones que mide Lighthouse. Esto valida empíricamente uno de los objetivos centrales del proyecto, ofrecer una experiencia rápida como factor diferencial frente a las alternativas existentes. Más allá de las cifras, la arquitectura resultante es deliberadamente simple, está bien delimitada por capas y se apoya en estándares ampliamente adoptados (HTTP, JWT, S3, OAuth2), lo que facilita tanto el mantenimiento como la incorporación de nuevas funcionalidades sin reescrituras estructurales.

Conviene destacar que lo que se ha construido es un prototipo en el sentido estricto del término, un sistema completo y operativo que cubre el camino feliz y los principales caminos alternativos, pero que no pretende competir todavía en variedad de funcionalidades con plataformas que llevan años en el mercado. Dicho esto, la base

sobre la que se asienta es lo bastante sólida para escalar, ya que el *backend* es *stateless* y se puede replicar horizontalmente detrás de un balanceador, el almacenamiento de objetos está abstraído tras una fachada y puede cambiarse de proveedor sin tocar lógica de dominio, la base de datos PostgreSQL admite particionado y *read replicas* cuando llegue el momento, y el *frontend* se sirve como contenido estático con SSR puntual. En otras palabras, las decisiones tomadas no comprometen en absoluto el crecimiento futuro y permiten incorporar tanto más usuarios como más funcionalidades con el coste mínimo posible.

4.2 Objetivos alcanzados

Los siete objetivos enumerados en el capítulo de Introducción se han cumplido en su totalidad. La API REST en Go está implementada y desplegada en producción, exponiendo los *endpoints* recogidos en la Tabla 12 y validados tanto por *tests* unitarios como por la prueba de carga descrita en Sección 3.7.4.1. La interfaz web en SvelteKit cubre la totalidad del flujo de usuario, desde el *onboarding* hasta la subida y consulta de recursos, con dos disposiciones de navegación adaptadas a escritorio y móvil. La autenticación con Google OAuth2 funciona como mecanismo único de acceso, con detección automática del centro educativo a partir del dominio del correo cuando este está registrado en la tabla de universidades. El catálogo de centros y asignaturas se ha poblado de forma totalmente automatizada, combinando los datos de JetBrains SWOT para las universidades y un *scraper* específico de la UCM para los grados y asignaturas, y se carga durante el despliegue inicial mediante los binarios `cmd/seed-universities` y `cmd/seed-studies` introducidos en Sección 3.5.2. El almacenamiento de archivos se delega en un *bucket* de Hetzner Object Storage al que se accede mediante la librería `minio-go`, lo que permite migrar a cualquier otro proveedor compatible con S3 sin cambios de código. La búsqueda global se resuelve con *Full Text Search* nativo de PostgreSQL, con pesos por campo y soporte de tildes mediante la extensión `unaccent`. Por último, la plataforma se ha desplegado de forma portable mediante Docker Compose en un VPS, con un único `docker compose up` capaz de levantar todo el *stack* en cualquier máquina con Docker.

4.3 Trabajo futuro

A partir del prototipo entregado se han existido varias líneas de evolución, todas ellas viables sin reescritura estructural gracias a la arquitectura descrita en el capítulo de Descripción del Trabajo.

La extensión más inmediata es la incorporación de nuevos visores de archivos en el *frontend*. La arquitectura actual delega la previsualización en componentes Svelte especializados por tipo (`PdfViewer`, `ImageViewer`, `MarkdownViewer`), seleccionados a

partir de la extensión del fichero. Añadir soporte para código fuente con resaltado de sintaxis, audio mediante `<audio>`, vídeo mediante `<video>` o documentos ofimáticos mediante una conversión previa a PDF se reduce a crear un componente nuevo y registrarlo en la tabla de despacho del visor genérico, sin tocar el *backend*.

Una segunda línea, más ambiciosa, es ampliar el concepto de recurso para que no se limite a un conjunto de archivos. Cabe imaginar al menos tres variantes nuevas. La primera son los *enlaces*, recursos que apuntan a un URL externo, útiles para compartir vídeos de YouTube, foros o documentación oficial sin necesidad de descargar nada. La segunda son los *recursos de texto*, con un editor enriquecido usando `markdown` que permita redactar resúmenes y apuntes directamente en la plataforma, sin pasar por un fichero subido. La tercera, la más interesante didácticamente, son los *cuestionarios* al estilo de los *tests* de Moodle, con preguntas, respuestas correctas y un modo de *autoevaluación* que se podría enriquecer con métricas agregadas. La estructura actual de la tabla `resources` admite esta extensión añadiendo una columna `kind` discriminadora y tablas auxiliares por tipo, sin alterar el resto del modelo.

Otra evolución natural, en la línea de las plataformas analizadas en el capítulo de Estado de la Cuestión, es la integración de funcionalidades de inteligencia artificial. Algunas son inmediatamente útiles, como la generación automática de un resumen del recurso a partir del texto extraído del PDF, la sugerencia de etiquetas o de la asignatura más probable durante la subida, o la detección automática de contenido inapropiado para complementar el sistema de reportes manuales. Otras requieren más diseño, como un *chatbot* que responda preguntas sobre los apuntes del usuario o la generación de cuestionarios a partir de un recurso existente, similares a los que ya ofrecen Studocu y Docsity. La arquitectura permite incorporarlas como un servicio adicional consumido por el *backend*, sin acoplarlas al núcleo del dominio.

Por último, hay líneas de trabajo más operativas que no introducen funcionalidades nuevas pero amplían el alcance del prototipo. La más evidente es completar el catálogo más allá de la UCM, escribiendo *scrapers* para otras universidades españolas y aprovechando que la tabla `universities` ya contiene los centros del mundo entero gracias a JetBrains SWOT. Otras posibilidades son la implementación de un sistema de comentarios y valoraciones por recurso, la introducción de notificaciones en tiempo real mediante WebSockets cuando se reporta o se descarga un recurso del usuario, una aplicación móvil nativa que reutilice la API existente y la migración del despliegue actual de un único VPS a una infraestructura con balanceador y réplicas cuando el volumen de tráfico lo justifique. Todas estas extensiones encajan limpiamente con las decisiones de diseño tomadas, lo que confirma que el prototipo no solo cumple los objetivos planteados, sino que constituye una base sobre la que seguir construyendo.

Chapter 4 Conclusions and Future Work

Abstract: This chapter presents the conclusions drawn from the project and the future lines of work.

Perfection is achieved, not when there is nothing more to add, but when there is nothing left to take away.

— Antoine de Saint-Exupéry

4.1 Conclusions

The result of the project is a functional platform, deployed in production at nemsy.org and demonstrably solid, that covers the proposed use case, namely sharing and consulting university notes between students, without the advertising and paywall concessions that limit the alternatives analysed in the State of the Art chapter. The implementation combines a Go backend with sqlc and PostgreSQL and a SvelteKit frontend with Svelte 5, two choices that have proven to be sound both in terms of productivity during development and in performance measured during the validation phase.

The data gathered during the k6 load test and the Lighthouse analysis show that the system behaves stably under sustained concurrency and obtains scores noticeably higher than Wuolah across the four dimensions Lighthouse measures. This empirically validates one of the central goals of the project, offering a fast experience as a differentiating factor against the existing alternatives. Beyond the figures, the resulting architecture is deliberately simple, well delimited by layers and built on widely adopted standards (HTTP, JWT, S3, OAuth2), which makes both maintenance and the addition of new features easy without structural rewrites.

It is worth emphasising that what has been built is a prototype in the strict sense of the word, a complete and operational system that covers the happy path and the main alternative paths, but that does not yet aim to compete in feature breadth with platforms that have been on the market for years. That said, the foundation it sits on is solid enough to scale, since the backend is stateless and can be replicated horizontally behind a load balancer, the object storage is abstracted behind a façade and the provider can be swapped without touching domain logic, PostgreSQL supports partitioning and read replicas when the time comes, and the

frontend is served as static content with targeted SSR. In other words, the decisions taken do not compromise future growth and allow incorporating both more users and more features at minimal cost.

4.2 Objectives Achieved

The seven objectives listed in the Introduction chapter have been fully met. The Go REST API is implemented and deployed in production, exposing the endpoints documented throughout the Work Description chapter and validated both by unit tests and by the load test described in the testing section. The SvelteKit web interface covers the entire user flow, from onboarding to uploading and consulting resources, with two navigation layouts adapted to desktop and mobile. Google OAuth2 authentication works as the sole access mechanism, with automatic detection of the educational institution from the email domain when it is registered in the universities table. The catalogue of institutions and subjects has been populated entirely automatically, combining JetBrains SWOT data for universities with a UCM-specific scraper for degrees and subjects, and is loaded during the initial deployment via the `cmd/seed-universities` and `cmd/seed-studies` binaries. File storage is delegated to a Hetzner Object Storage bucket accessed through the `minio-go` library, which makes it possible to migrate to any other S3-compatible provider without code changes. Global search is resolved with PostgreSQL native Full Text Search, with per field weighting and accent support via the `unaccent` extension. Finally, the platform has been deployed in a portable way using Docker Compose on a VPS, with a single `docker compose up` capable of bringing up the entire stack on any machine with Docker installed.

4.3 Future Work

Several lines of evolution have been identified from the delivered prototype, all of them feasible without structural rewrites thanks to the architecture described in the Work Description chapter.

The most immediate extension is the addition of new file viewers in the frontend. The current architecture delegates previewing to Svelte components specialised by type (`PdfViewer`, `ImageViewer`, `MarkdownViewer`), selected from the file extension. Adding support for source code with syntax highlighting, audio via `<audio>`, video via `<video>` or office documents via prior conversion to PDF reduces to creating a new component and registering it in the dispatch table of the generic viewer, without touching the backend.

A second, more ambitious line is to broaden the concept of resource so that it is not limited to a set of files. At least three new variants can be envisaged. The first

is *links*, resources pointing to an external URL, useful for sharing YouTube videos, forums or official documentation without having to download anything. The second is *text resources*, with a Markdown based rich editor that allows writing summaries and notes directly on the platform, without going through an uploaded file. The third, the most interesting from a didactic point of view, is *quizzes* in the style of Moodle tests, with questions, correct answers and a self-assessment mode that could be enriched with aggregated metrics. The current structure of the `resources` table admits this extension by adding a discriminating `kind` column and auxiliary tables per type, without altering the rest of the model.

Another natural evolution, in line with the platforms analysed in the State of the Art chapter, is the integration of artificial intelligence features. Some are immediately useful, such as automatically generating a summary of the resource from text extracted from the PDF, suggesting tags or the most likely subject during upload, or automatic detection of inappropriate content to complement the manual reporting system. Others require more design, such as a chatbot that answers questions about the user's notes or the generation of quizzes from an existing resource, similar to those already offered by Studocu and Docsity. The architecture allows incorporating them as an additional service consumed by the backend, without coupling them to the core domain.

Finally, there are more operational lines of work that do not introduce new features but extend the scope of the prototype. The most obvious one is to complete the catalogue beyond UCM, writing scrapers for other Spanish universities and taking advantage of the fact that the `universities` table already contains institutions worldwide thanks to JetBrains SWOT. Other possibilities are implementing a per resource comments and ratings system, introducing realtime notifications via WebSockets when a user resource is reported or downloaded, a native mobile application that reuses the existing API, and migrating the current single VPS deployment to an infrastructure with a load balancer and replicas when traffic volume justifies it. All these extensions fit cleanly with the design decisions taken, which confirms that the prototype not only meets the proposed objectives but also constitutes a foundation on which to keep building.

Bibliografía

- [1] Stack Overflow, «Stack Overflow Developer Survey 2025». [En línea]. Disponible en: <https://survey.stackoverflow.co/2025/technology>
- [2] E. L. Deci, R. Koestner, y R. M. Ryan, «A meta-analytic review of experiments examining the effects of extrinsic rewards on intrinsic motivation», 1999.
- [3] B. S. Frey y R. Jegen, «Motivation Crowding Theory», 2001.
- [4] The Go Authors, «The Go Programming Language». [En línea]. Disponible en: <https://go.dev/>
- [5] go-chi contributors, «Chi: lightweight, idiomatic and composable router for building Go HTTP services». [En línea]. Disponible en: <https://go-chi.io/>
- [6] Meta Open Source, «React: The library for web and native user interfaces». [En línea]. Disponible en: <https://react.dev/>
- [7] Svelte Contributors, «Svelte: Cybernetically enhanced web apps». [En línea]. Disponible en: <https://svelte.dev/>
- [8] Oracle Corporation, «MySQL: The world's most popular open source database». [En línea]. Disponible en: <https://www.mysql.com/>
- [9] MongoDB, Inc., «MongoDB: The Developer Data Platform». [En línea]. Disponible en: <https://www.mongodb.com/>
- [10] The PostgreSQL Global Development Group, «PostgreSQL: The World's Most Advanced Open Source Relational Database». [En línea]. Disponible en: <https://www.postgresql.org/>
- [11] Caddy Web Server, «Caddy: The Ultimate Server with Automatic HTTPS». [En línea]. Disponible en: <https://caddyserver.com/>
- [12] golang-migrate contributors, «golang-migrate: Database migrations. CLI and Golang library.». [En línea]. Disponible en: <https://github.com/golang-migrate/migrate>
- [13] PostgreSQL Global Development Group, «GIN Indexes». [En línea]. Disponible en: <https://www.postgresql.org/docs/current/gin.html>
- [14] The Go Authors, «Go Modules Reference: Internal packages». [En línea]. Disponible en: <https://go.dev/ref/mod#internal-packages>

- [15] M. Seemann, «Composition Root». [En línea]. Disponible en: <https://blog.ploeh.dk/2011/07/28/CompositionRoot/>
- [16] A. Wiggins, «The Twelve-Factor App». [En línea]. Disponible en: <https://12factor.net/>
- [17] The sqlc Authors, «sqlc: A SQL Compiler». [En línea]. Disponible en: <https://docs.sqlc.dev/>
- [18] C. Richardson, «Pattern: Saga». [En línea]. Disponible en: <https://microservices.io/patterns/data/saga.html>
- [19] IETF, «RFC 6819: OAuth 2.0 Threat Model and Security Considerations, §4.4.1.8». [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc6819#section-4.4.1.8>
- [20] IETF, «RFC 7519: JSON Web Token (JWT)». [En línea]. Disponible en: <https://datatracker.ietf.org/doc/html/rfc7519>
- [21] OWASP Foundation, «Session Management Cheat Sheet». [En línea]. Disponible en: https://cheatsheetseries.owasp.org/cheatsheets/Session_Management_Cheat_Sheet.html
- [22] T. McLean, «Critical vulnerabilities in JSON Web Token libraries». [En línea]. Disponible en: <https://auth0.com/blog/critical-vulnerabilities-in-json-web-token-libraries/>
- [23] Vitest Team, «Vitest: Next Generation Testing Framework ». [En línea]. Disponible en: <https://vitest.dev/>
- [24] Microsoft, «Playwright: Playwright enables reliable web automation for testing, scripting, and AI agents.». [En línea]. Disponible en: <https://playwright.dev/>
- [25] Grafana Labs, «k6: The best developer experience for load testing». [En línea]. Disponible en: <https://k6.io/>
- [26] Google, «RAIL performance model». [En línea]. Disponible en: <https://web.dev/articles/rail>
- [27] inputnet, «Helium: Internet without interruptions». [En línea]. Disponible en: <https://helium.computer/>

Apéndice A Guía de despliegue

Este apéndice describe los pasos necesarios para desplegar Nemsy en una máquina con Docker desde cero. Se asume un sistema GNU/Linux con `git`, `docker` y `docker compose` instalados, una cuenta de Google Cloud Platform para crear las credenciales OAuth2 y un proveedor de almacenamiento compatible con S3, en este caso Hetzner Object Storage.

A.1 Requisitos previos

El despliegue de producción corre sobre una instancia *CAX11* de Hetzner Cloud con procesador Ampere Altra (2 vCPU ARM64 con núcleos Neoverse-N1), 4 GB de RAM y 40 GB de disco, utilizando la imagen oficial de Fedora Linux 42 con Docker Engine y Docker Compose v2. La máquina debe disponer de al menos 1 GB de memoria RAM libre y unos 5 GB de almacenamiento para las imágenes Docker, la base de datos y un margen razonable para los archivos subidos al inicio, y exponer los puertos 80 y 443 al exterior si se va a publicar bajo un dominio propio.

Para la autenticación es necesario crear unas credenciales OAuth2 en Google Cloud Platform. En la consola, dentro del menú *APIs and Services*, se crea un nuevo proyecto, se habilita la *Google People API* y se generan unas credenciales de tipo *OAuth client ID* con la URL de retorno `https://<dominio>/api/auth/google/callback` (o `http://localhost:8080/api/auth/google/callback` para pruebas locales).

A.2 Configuración

Toda la configuración del sistema se concentra en un único fichero `.env` que `docker compose` lee al levantar el *stack*. La Código 20 reproduce las variables que es necesario definir.

```
DB_PASSWORD=<contraseña_aleatoria>

GOOGLE_CLIENT_ID=<id_obtenido_en_GCP>
GOOGLE_CLIENT_SECRET=<secreto_obtenido_en_GCP>
GOOGLE_REDIRECT_URL=https://<dominio>/api/auth/google/callback

JWT_SECRET=<cadena_aleatoria_de_32+_caracteres>
ALLOWED_ORIGIN=https://<dominio>

S3_ENDPOINT=<endpoint_de_hetzner_object_storage>
S3_ACCESS_KEY=<access_key_del_bucket>
S3_SECRET_KEY=<secret_key_del_bucket>
S3_BUCKET=nemsy
S3_USE_SSL=true

PUBLIC_API_BASE_URL=https://<dominio>
```

Código 20: Variables de entorno necesarias para el despliegue.

Las contraseñas y claves deben generarse con un mecanismo criptográficamente seguro, por ejemplo `openssl rand -hex 32` para `JWT_SECRET` y `openssl rand -base64 24` para las contraseñas de servicio.

A.3 Levantamiento del *stack*

Una vez configurado el `.env`, basta con clonar el repositorio y levantar el *stack* con un único comando.

```
git clone https://github.com/DCCXXV/nemsy.git
cd nemsy
cp .env.example .env
# editar .env con los valores reales (vi .env)
docker compose up -d
```

Código 21: Comandos para clonar y arrancar Nemsy.

`docker compose` levanta tres servicios en una red privada, el contenedor del *backend* Go, el del *frontend* SvelteKit servido como SSR Node y el de PostgreSQL con persistencia en un volumen llamado `pgdata`. El almacenamiento de archivos no se levanta como contenedor, sino que se delega externamente en un *bucket* de Hetzner Object Storage al que el *backend* accede mediante la librería `minio-go`, lo que evita ocupar espacio del VPS con los archivos subidos por los usuarios. Las migraciones de base de datos se aplican automáticamente al arrancar el *backend* mediante `golang-migrate`.

A.4 Carga inicial del catálogo

Tras el primer arranque, la base de datos está vacía de universidades y estudios. La imagen Docker del *backend* solo incluye el binario del servidor, así que los

seeders se ejecutan directamente desde el código fuente clonado, aprovechando que el contenedor de PostgreSQL expone el puerto 5432 en 127.0.0.1:5433 para conexiones desde el *host*.

```
cd backend
export DATABASE_URL="postgres://nemsy:$DB_PASSWORD@127.0.0.1:5433/nemsy?
sslmode=disable"
go run ./cmd/seed-universities
go run ./cmd/seed-studies
```

Código 22: Comandos para cargar el catálogo inicial de universidades y estudios.

El primer comando importa el catálogo mundial de centros desde JetBrains SWOT, y el segundo ejecuta el *scraper* de la UCM para poblar los grados y asignaturas. Ambos son idempotentes y pueden volver a ejecutarse para refrescar los datos. Esto requiere tener Go instalado en el *host*, requisito que se cumple sin esfuerzo en Fedora con `dnf install golang`.

A.5 Promoción de un usuario a administrador

El rol de administrador no se asigna desde la interfaz, sino directamente en la base de datos. Una vez que el usuario ha iniciado sesión por primera vez con Google, basta con actualizar su fila como muestra la Código 23.

```
docker compose exec -e PGPASSWORD=$DB_PASSWORD db \
psql -U nemsy -d nemsy \
-c "UPDATE users SET role = 'admin' WHERE email = '<correo>';"
```

Código 23: Promoción de un usuario a administrador.

Apéndice B Glosario y acrónimos

Término	Definición
API	<i>Application Programming Interface</i> . Conjunto de <i>endpoints</i> HTTP que el <i>backend</i> expone para que el <i>frontend</i> y otros clientes consuman.
CORS	<i>Cross-Origin Resource Sharing</i> . Mecanismo del navegador que controla qué orígenes pueden hacer peticiones a un dominio distinto.
DTO	<i>Data Transfer Object</i> . Estructura de datos que se serializa entre cliente y servidor.
FAB	<i>Floating Action Button</i> . Botón de acción flotante típico en interfaces móviles.
FTS	<i>Full Text Search</i> . Búsqueda de texto completo, en este proyecto resuelta con <code>tsvector/tsquery</code> de PostgreSQL.
GIN	<i>Generalized Inverted Index</i> . Tipo de índice invertido genérico de PostgreSQL, adecuado para valores compuestos como arrays, JSONB y <code>tsvector</code> . En Nemsy se utiliza para indexar las columnas <code>tsvector</code> que sostienen la búsqueda de texto completo.
JWT	<i>JSON Web Token</i> . Estándar para representar afirmaciones (<i>claims</i>) firmadas, utilizado para las cookies de sesión.
OAuth2	Estándar de autorización delegada. En Nemsy se usa el flujo <i>authorization code</i> con Google como proveedor.
ORM	<i>Object-Relational Mapping</i> . Capa que traduce entre objetos del lenguaje y filas de la base de datos. En este proyecto se ha evitado a favor de <code>sqlc</code> .
PWA	<i>Progressive Web App</i> . Aplicación web instalable con capacidades cercanas a las nativas.
REST	<i>Representational State Transfer</i> . Estilo arquitectónico para APIs HTTP basado en recursos y verbos.
RUNE	Construcción de Svelte 5 (<code>\$state</code> , <code>\$derived</code> , <code>\$props</code> , <code>\$effect</code>) que declara reactividad de forma explícita.
S3	<i>Simple Storage Service</i> . API de almacenamiento de objetos popularizada por Amazon. En Nemsy se utiliza Hetzner Object Storage como proveedor compatible con esta API.

Término	Definición
Saga	Patrón de coordinación de operaciones distribuidas mediante pasos compensables, aplicado en la creación de recursos.
SGBD	<i>Sistema Gestor de Bases de Datos.</i>
SPA	<i>Single Page Application.</i> Aplicación web que se carga una sola vez y reemplaza el contenido en cliente al navegar.
sqlc	Herramienta que genera código Go fuertemente tipado a partir de consultas SQL escritas a mano.
SSR	<i>Server-Side Rendering.</i> Renderizado de la página en el servidor antes de enviarla al cliente.
SWOT	<i>Student Webmail Origin Tracker</i> , lista mantenida por Jet-Brains que asocia dominios de correo a centros educativos.
TFG	<i>Trabajo de Fin de Grado.</i>
tsquery / tsvector	Tipos de PostgreSQL para consulta y representación tokenizada de texto en búsquedas FTS.
VPS	<i>Virtual Private Server.</i> Servidor virtualizado contratado a un proveedor de <i>hosting</i> , donde está desplegado Nemsy.
VU	<i>Virtual User.</i> Usuario simulado por k6 durante una prueba de carga.